
OpenSTREAM Manual

The OpenSTREAM Team

Aug 10, 2026

USAGE

1	Welcome	1
1.1	What's inside?	1
1.2	Why OpenSTREAM?	1
1.3	About this documentation	1
2	Usage	3
2.1	Getting started	3
2.1.1	Prerequisites and installation	3
2.2	Run sample script	4
3	Packages	7
3.1	InputEnums	7
3.2	Inputs	16
3.3	Session	33
3.4	Solver	36
3.4.1	Four-field solver	36
3.4.2	Mixture solver	53
3.4.3	Three-field solver	75
3.4.4	Two-fluid solver	85
4	ExternalPackages	117
4.1	CoolPropWrapper	117
	Bibliography	119
	MATLAB Module Index	121
	Index	123

WELCOME

OpenSTREAM (**O**pen **S**olvers for **T**wo-phase flow **R**esearch, **E**ngineering **A**nalysis and **M**odeling) is an open-source, object-oriented computational environment designed for simulating one-dimensional, multi-field, two-phase flows in straight geometries (Le Corre *et al.* [2025a]) (Le Corre *et al.* [2025b]). It supports phenomena such as wall boiling, phase interactions and non-equilibrium effects, making it a relevant tool for researchers working in thermal-hydraulics and two-phase flow systems.

1.1 What's inside?

OpenSTREAM offers a suite of solver frameworks tailored to different modeling needs:

- A **mixture solver** – with thermal non-equilibrium capabilities
- A generic **two-fluid solver** – for generic separate-phase modeling
- A **three-field solver** – for annular two-phase flow
- An advanced **four-field solver** – for annular two-phase flow explicitly capturing disturbance waves

These solvers are designed to simulate single-component, thermally expandable, steady-state and transient boiling two-phase flows in straight multi-wall channels, which may be uniformly or non-uniformly heated. The implementation relies on a set of reasonable simplifying assumptions to ensure computational efficiency and stability. Each solver includes a set of baseline closure models, which can be customized by modifying the corresponding class methods in the source code.

1.2 Why OpenSTREAM?

OpenSTREAM is built to support open, transparent research and to lower the barrier to entry for developing and validating fundamental models in two-phase flow simulations. Its goal is to make advanced thermal-hydraulic modeling more accessible to engineers, scientists, and students alike. Whether you're conducting research or learning the fundamentals, OpenSTREAM provides a collaborative and extensible environment for model development, performance evaluation, and cross-institutional validation, advancing the state of the art in thermal-hydraulic simulation.

1.3 About this documentation

This guide will walk you through:

- Installation and setup instructions
- The design philosophy behind OpenSTREAM
- How to run simulations and interpret results
- Example applications and use cases

- How to contribute

Looking for a printable version? You can download the full manual here: [OpenSTREAM.pdf](#)

2.1 Getting started

2.1.1 Prerequisites and installation

OpenSTREAM is a MATLAB program for simulating one-dimensional two-phase flows. The Python version of **CoolProp** is used for calculating the fluid properties. **CoolPropWrapper** is a MATLAB interface that interacts with CoolProp through Python. Both OpenSTREAM and CoolProp are hosted on GitHub. Making a copy (cloning) of these programs will be our first step.

Install using Git and Github

- There are two main ways of interacting with GitHub to use and contribute to the OpenSTREAM project: **Git Command Line Interface (CLI)**, and the **GitHub Desktop** program. Use the links to set up one of the methods.
- After installing and setting up your Git mechanism of choice, if you chose to use the:
 - **Command line interface**
 - * Clone (make a copy of) the repository using: `git clone --recursive git@github.com:OpenSTREAM-solvers/openstream.git` (**SSH authentication**)
 - or
 - `git clone --recursive https://github.com/OpenSTREAM-solvers/openstream.git` (**HTTPS authentication**).
 - (The `--recursive` flag is needed to clone **submodules**, such as the **mfval/CoolPropWrapper**, in one command.)
 - * Then, use `cd openstream` to enter the newly created folder. You should see the newly downloaded files for the project.
 - **GitHub Desktop program**
 - * Open the program, login, and **Clone repository**. Everything should be cloned automatically. This is part of the setup guide linked above.

MATLAB-Python compatibility

Before using OpenSTREAM, we need to make sure Python with the appropriate version is installed, and MATLAB knows the location of the Python executable. The following instructions are from the **CoolPropWrapper** repo.

1. Determine which versions of Python are compatible with your version of MATLAB: **compatibility list**. The second column, *MATLAB Interface MATLAB Engine*, shows the compatible Python versions.
2. Start MATLAB and verify a compatible python version is installed:

- Run `pyenv()`
 - If the results are empty or an incompatible Python version is shown, go to step 3.
 - If the results are satisfactory, continue to step 5.
3. Install compatible Python3
- **Windows:** Go to python.org to download the specific version of Python you need. Follow the setup procedure.
 - **Linux:** Chances are, you already have a version of Python installed. Verify the version using `python3 --version`. If Python is not installed, or an incompatible version is installed, install appropriate version using the OS package manager. For Debian-based Linux distros (such as Ubuntu), use `sudo apt-get install python3.x` to install the *x* version of Python3.
 - **MacOS:** Someone with a Mac, please write how you do this...
4. Specify Python installation location in MATLAB:
- Run `pyenv('Version', 'pathtopython')`, where `pathtopython` is the path to where Python is installed. Here are some typical locations depending on your OS:
 - **Windows:** `C:\Users\Username\AppData\Local\Programs\Python\Python310\python.exe`, for version 3.10.
 - **Linux:** `/usr/bin/python3.10`, for version 3.10. Use `whereis python3` to find possible locations.
 - **MacOS:** Someone with a Mac, please write how you do this...
5. Try creating an instance of `CoolPropWrapper()` in MATLAB:
- Run `cp=CoolPropWrapper()`.
 - If you are using the `CoolPropWrapper` as MATLAB Package in OpenSTREAM, use `cp=CoolPropWrapper.CoolPropWrapper()`.
 - If the `CoolProp` module is not installed, you will be prompted to do so automatically. If you prefer to do this manually, in your terminal, run `python3 -m pip install --user -U CoolProp`, replacing `python3` with the specific python path as necessary.

Next steps

Congrats! Hopefully, at this point, you have successfully made a copy of this repo and installed the required programs. Next, let's run the sample script to make sure everything is working properly.

2.2 Run sample script

After installation, you're all set to launch your first simulation!

As a first exercise, open MATLAB and run the commands listed below from within the `openstream` folder, which will simulate an example of boiling two-phase flow in a uniformly heated tube followed by an adiabatic region. The **OpenSTREAM** mixture model will be used, which defaults to the Homogeneous Equilibrium Model (HEM) when the physical models are not modified by the user.

This is the quickest and easiest way to start exploring what OpenSTREAM can do, and to confirm everything is working smoothly under the hood.

```
% Clear existing variables from workspace
clearvars
```

(continues on next page)

(continued from previous page)

```
% Import the necessary classes for inputs and solvers
import Inputs.*
import Solvers.*
import Solvers.Mixture.*

% Turn off warnings during input setup
warning off

% Create the input set using paths to model, options, geometry, and boundary condition
↪ files
inputSet = InputSet( ...
    modelFilePath      = './inputs/models.inp', modelID      = 'TUTORIAL1', ...
    optionsFilePath    = './inputs/options.inp', optionsID   = 'DEFAULT', ...
    geometryFilePath   = './inputs/geom.inp',   geometryID  = 'TUTORIAL1', ...
    bcFilePath         = './inputs/tutorial1.inp', ...
    sessionParentDir   = fullfile(pwd,'outputs'), ...
    overwriteSessionFiles = true, ...
    LOGMODE            = 'BOTH');

% Create a mixture solver object
mixSolver = MixtureSolver(inputSet);

% Solve
mixSolver.solve();

% Default axial plots
mixSolver.plotz();

% Save results
mixSolver.save();
```

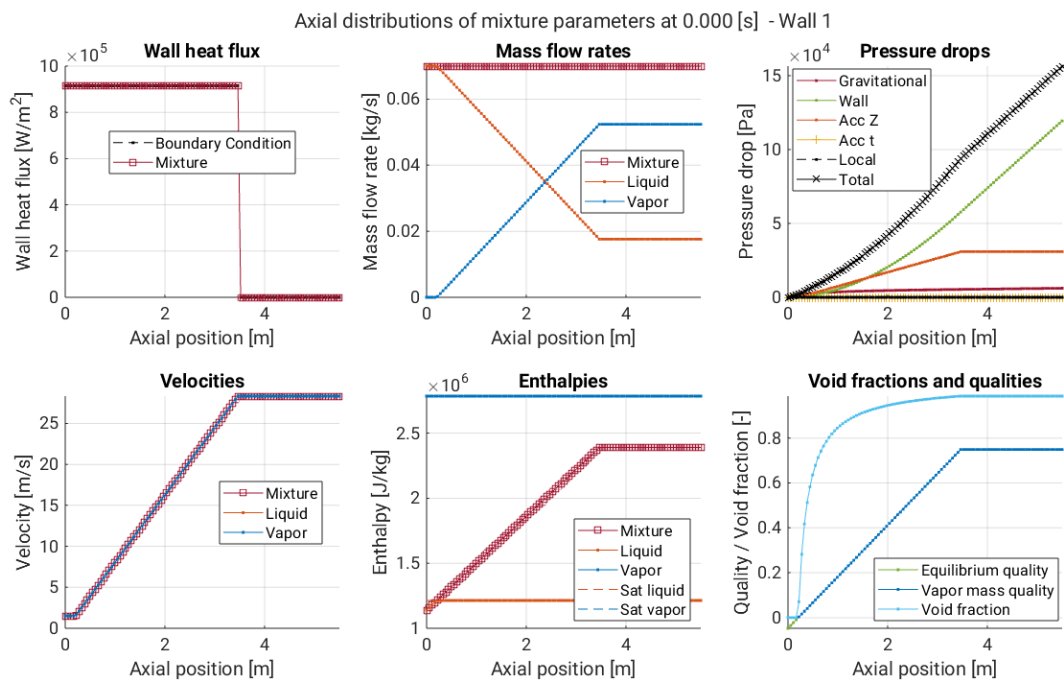


Figure 1: Sample figure of axial distributions of mixture parameters.

PACKAGES

3.1 InputEnums

The *InputEnums* package defines a set of enumeration classes used to specify valid options for physical models in **OpenSTREAM**. These enums help ensure consistency and clarity when selecting model configurations, making the simulation setup more robust and less error-prone.

Typical examples include choosing between void fraction models, interfacial transfer options, or flow regime assumptions, each represented by a clearly defined enum.

class InputEnums.LOCRELVEL

LOCRELVEL Enumeration of local relative velocity models

This class defines the available *Inputs.Model.LOCRELVEL* models for calculating the local relative velocity between liquid and vapor phases, used by the two-fluid solver.

Models:

- AREAMEAN — Difference between area-averaged phase velocities (Walter [2024])
- SCALED — Scaled difference using user-defined *Inputs.Model.RELVELCST* (Walter [2024])
- DRIFT — Uses drift velocity as the local relative velocity (Walter [2024])
- SIMPLE — Simplified assumptions for bubbly and annular flow regime (Walter [2024])

class InputEnums.DEPENHANCEMENT

DEPENHANCEMENT Enumeration of droplet deposition enhancement models

This class defines the available *Inputs.Model.DEPENHANCEMENT* models for calculating local droplet deposition enhancement, used by the three-field and four-field solvers.

Models:

- NONE — No enhancement applied
- WINDECKER — Model adapted from Windecker [1999], as documented in Le Corre [2024], using user defined *Inputs.Model.KTUNING*

class InputEnums.THINFILMFRIC

THINFILMFRIC Enumeration of thin film friction models

This class defines the available *Inputs.Model.THINFILMFRIC* models for calculating wall friction under thin film conditions, used by the three-field and four-field solvers.

Models:

- LAMINAR — Laminar flow model ($16/Re_{film}$)

- TURBULENT — Turbulent flow model

class InputEnums.**THERMALRELAX**

THERMALRELAX Enumeration of MRM time relaxation models

This class defines the available `Inputs.Model.THERMALRELAX` models for calculating relaxation times in the MRM model, used when `Inputs.Model.THERMALNONEQ` is set to `MRM`.

Models:

- FOURIERX — Simple model relating the Fourier number to equilibrium quality using user-defined `Inputs.Model.RELAXX`, `Inputs.Model.RELAXCONDFO` and `Inputs.Model.RELAXEVAPFO`
- TIMEX — Simple model relating relaxation time to equilibrium quality using user-defined `Inputs.Model.RELAXX`, `Inputs.Model.RELAXCONDT` and `Inputs.Model.RELAXTEVAPT`
- VOID — Physical model simplified to Fourier number dependence on phase volumetric fraction and input characteristic length of the dispersed phase at CBT using user-defined `Inputs.Model.RELAXCONDCOEF` and `Inputs.Model.RELAXEVAPCOEF`
- HOMOGENEOUS — Physical model based on homogeneous assumptions and spherical dispersed phase
- FOURIER — Empirical model based on Fourier number dependence on Reynolds number, etc, using user-defined `Inputs.Model.RELAXCONDFOCOEF` and `Inputs.Model.RELAXEVAPFOCOEF`

class InputEnums.**INTNU**

INTNU Enumeration of interfacial Nusselt number models

This class defines the available `Inputs.Model.INTNU` models for calculating the interfacial Nusselt number, used by the two-fluid solver.

Models:

- CONSTANT — Assumes a constant Nusselt number using user-defined `Inputs.Model.INTNUVCST` and `Inputs.Model.INTNULCST` for the dispersed gas and liquid phases, respectively
- RANZMARSHALL — Ranz-Marshall correlation-based model (Ranz and Marshall [1952]) using user-defined `Inputs.Model.RANZMARSHALLVCST` and `Inputs.Model.RANZMARSHALLLCST` for the dispersed gas and liquid phases, respectively
- RELAXATION — Time relaxation-based model (Walter [2024])

class InputEnums.**WAVEBASEINT**

WAVEBASEINT Enumeration of wave to base film interfacial momentum transfer models

This class defines the available `Inputs.Model.WAVEBASEINT` models for calculating the interfacial momentum transfer between wave and base film regions, used by the four-field solver.

Models:

- VAPORSHEAR — Vapor shear across wave area (consistent with Le Corre [2022], Eq. 46).
- VAPORSHEARDROPMASS — Also includes drop deposition momentum transfer, ensuring consistency between wave and base film velocities for thin films.

class InputEnums.**NEARWALLH**

NEARWALLH Enumeration of near-wall equilibrium enthalpy models

This class defines the available `Inputs.Model.NEARWALLH` models for setting the near-wall enthalpy to mixture enthalpy ratio, used by the mixture solvers.

Models:

- RATIO — Constant near-wall enthalpy to mixture enthalpy ratio using user-defined `Inputs.Model.NEARWALLHRATIO`

- FILM — Mass flux-dependent input for annular two-phase flow

class InputEnums.OAFENTRAINED

OAFENTRAINED Enumeration of entrained drop models at onset of annular flow

This class defines the available [Inputs.Model.OAFENTRAINED](#) models for splitting the droplet and film mass flow rates at the onset of annular flow, used by the three-field and four-field solvers.

Models:

- RATIO — Ratio of droplet to liquid mass flow rates using user-defined [Inputs.Model.OAFDROPRATIO](#)
- EQUILIBRIUM — Equilibrium assumption model ($M_{\text{ent}} = M_{\text{dep}}$) (Adamsson and Le Corre [2011], Equations 15 and 16)

class InputEnums.BTMTM

BTMTM Enumeration of post-boiling transition wall momentum transfer models

This class defines the available [Inputs.Model.BTMTM](#) models for calculating the post-boiling transition wall momentum transfer, used by the mixture solver.

Models:

- TPFM — Wall momentum transfer based on two-phase friction multiplier, using multiplier selected in [Inputs.Model.TPFM](#)
- VAPOR — Wall momentum transfer based on vapor phase only, using single-phase momentum transfer model selected in [Inputs.Model.SPMTM](#)

class InputEnums.BASEEQTHICK

BASEEQTHICK Enumeration of base film equilibrium thickness models

This class defines the available [Inputs.Model.BASEEQTHICK](#) models for calculating the equilibrium base film thickness, used by the four-field solver.

Models:

- RISO — Based on RISO dataset (Le Corre [2022])
- COEFS — Generalized RISO model using user-defined [Inputs.Model.BASEEQTHICKCOEF](#)
- YPLUS — Based on dimensionless wall distance (y^+) using user-defined [Inputs.Model.BASEYPLUS](#)
- MFVAL — MFVAL model (under development)

class InputEnums.OAF

OAF Enumeration of onset of annular flow models

This class defines the available [Inputs.Model.OAF](#) models for predicting the onset of annular flow, used by the three-field and four-field solvers.

Models:

- WALLIS — Full Wallis correlation (Wallis [1969], Equation 11.1019)
- WALLIS_SIMP — Simplified Wallis correlation (Sanmiguel Gimeno *et al.* [2015], Equation 6), used for compatibility with codes employing this approach
- LEVITAN — Levitan correlation (Levitan and Borevsky [1989])

class InputEnums.MFBT

MFBT Enumeration of Minimum Film Boiling Transition (MFBT) models

This class defines the available [Inputs.Model.MFBT](#) models for evaluating the Minimum Film Boiling Transition, used by the mixture field.

Models:

- NONE — No MFBT transition applied
- CONSTANT — Constant deviation from the saturation temperature using user-defined `Inputs.Model.DTMFB`

class InputEnums.THERMALNONEQ

THERMALNONEQ Enumeration of thermal non-equilibrium models

This class defines the available `Inputs.Model.THERMALNONEQ` models for calculating phase thermal non-equilibrium effects, used by the mixture solver.

Models:

- EQUILIBRIUM — Assumes thermal equilibrium ($X = \max(0, X_{EQ})$)
- SAHAZUBER — Saha-Zuber subcooled boiling model (Saha and Zuber [1973])
- EPRI — EPRI subcooled boiling model (Lellouche and Zolotar [1982])
- MRM — Homogeneous Relaxation Model for subcooled boiling and post-CHF conditions (MRM)

class InputEnums.OAFFILMSPLIT

OAFFILMSPLIT Enumeration of film mass flow ratio models at onset of annular flow

This class defines the available `Inputs.Model.OAFFILMSPLIT` models for splitting the base film and wave mass flow rates at the onset of annular flow, used by the four-field solver.

Models:

- RATIO — Ratio of base film to film mass flow rates using user-defined `Inputs.Model.OAFBASERATIO`
- EQUILIBRIUM — Equilibrium model (film thickness = equilibrium film thickness) (Le Corre [2022])

class InputEnums.FLUIDPROPERTIES

FLUIDPROPERTIES Enumeration of fluid property calculation assumptions

This class defines the available `Inputs.Model.PROPERTIES` assumptions to calculate fluid properties, applicable to all solvers.

Models:

- SATURATED — Properties based on saturated conditions at system pressure
- PSYSTEM — Properties based on system pressure and local phase enthalpy

class InputEnums.DROPDRAG

DROPDRAG Enumeration of droplet drop drag coefficient models

This class defines the available `Inputs.Model.DROPDRAG` models for calculating the droplet drag coefficient, used by the two-fluid solver.

Models:

- CONSTANT — Constant drag coefficient using user-defined `Inputs.Model.DROPDRAGCOEF`
- STOKES — Stokes flow regime model ($\frac{24}{Re_p}$)
- VISCOUS — Viscous regime model (Walter [2024])
- DISTORTED — Model for distorted (non-spherical) droplets (Walter [2024])

class InputEnums.MOMENTGAS

MOMENTGAS Enumeration of gas momentum conservation models

This class defines the available [Inputs.Model.MOMENTGAS](#) models for solving the momentum conservation equation for the gas phase, used to compute gas velocity in the two-fluid solver.

Models:

- MIXTURE — Uses the same gas velocity as in the mixture model
- SLIP — Slip ratio model (gas/liquid velocity) using % user-defined [Inputs.Model.SLIP](#)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a], Walter [2024])

class InputEnums.DEPOSITION

DEPOSITION Enumeration of droplet deposition models

This class defines the available [Inputs.Model.DEPOSITION](#) models for calculating the droplet deposition mass flux, used by the three-field and four-field solvers.

NOTE: Entrainment and deposition correlations are coupled. It is recommended to use the same model for both entrainment and deposition mass flux to ensure consistency.

Models:

- NONE — No deposition applied
- GOVAN — Hewitt and Govan correlation (Hewitt and Govan [1990])
- OKAWA — Okawa et al. correlation (Okawa *et al.* [2003])

class InputEnums.MOMENTDROP

MOMENTDROP Enumeration of droplet momentum conservation models

This class defines the available [Inputs.Model.MOMENTDROP](#) models for solving the droplet momentum conservation equation, used to compute droplet velocity in the three-field and four-field solvers.

Models:

- ALGEBRAIC — Simple algebraic model consistent with mixture solver
- SLIP — Slip ratio model (droplet/gas velocity)
- EQUILIBRIUM — Equilibrium model
- EQUILIBRIUMS — Simplified equilibrium model based on force balance ($F_{\text{drag}} + F_{\text{gravity}} + F_{\text{buoyancy}} = 0$)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a])

class InputEnums.BUBBLEDRAG

BUBBLEDRAG Enumeration of bubble drag coefficient models

This class defines the available [Inputs.Model.BUBBLEDRAG](#) models for calculating the bubble drag coefficient, used by the two-fluid solver.

Models:

- CONSTANT — Constant drag coefficient using user-defined [Inputs.Model.BUBBLEDRAGCOEF](#)
- STOKES — Stokes flow regime model ($\frac{24}{Re_p}$)
- VISCOUS — Viscous regime model (Walter [2024])
- DISTORTED — Model for distorted bubbles (non-spherical) (Walter [2024])

class InputEnums.CBT

CBT Enumeration of Critical Boiling Transition (CBT) models

This class defines the available [Inputs.Model.CBT](#) models for evaluating the Critical Boiling Transition, used by the mixture field.

Models:

- NONE — No CBT transition applied
- BIASI — Biasi correlation model (Biasi *et al.* [1966])
- BEZRUKOV — Bezrukov correlation model (Bezrukov *et al.* [1976])
- ELEVATION — CBT determined from specified elevation using user-defined CBTELEVATION

class InputEnums.MOMENTLIQUID

MOMENTLIQUID Enumeration of liquid momentum conservation models

This class defines the available [Inputs.Model.MOMENTLIQUID](#) models for solving the momentum conservation equation for the liquid phase, used to compute liquid velocity in the two-fluid solver.

Models:

- MIXTURE — Uses the same velocity as in the mixture model
- SLIP — Slip ratio model (gas/liquid velocity) using user-defined [Inputs.Model.SLIP](#)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a], Walter [2024])

class InputEnums.NEARWALLRELAX

NEARWALLRELAX Enumeration of near-wall energy transfer time relaxation models

This class defines the available [Inputs.Model.NEARWALLRELAX](#) models for calculating the near-wall energy transfer time relaxation, used in mixture solver.

Models:

- QUALITY — Simple model relating relaxation time to equilibrium quality using user-defined [Inputs.Model.NEARWALLRELAXX](#) and [Inputs.Model.NEARWALLRELAXT](#)
- VOID — Physical model based on local phase volumetric fraction using user-defined [Inputs.Model.NEARWALLRELAXCOEF](#)
- FILM — Model applicable to annular two-phase flow

class InputEnums.TPHTM

TPHTM Enumeration of two-phase wall heat transfer models

This class defines the available [Inputs.Model.TPHTM](#) models for calculating the two-phase wall heat transfer, used by the mixture solver.

Models:

- THOM — Thom wall heat transfer model (Rohsenow *et al.* [1998])

class InputEnums.MOMENTWAVE

MOMENTWAVE Enumeration of film momentum conservation models

This class defines the available [Inputs.Model.MOMENTWAVE](#) models for solving the momentum conservation equation for the wave field, used to compute wave velocity in the four-field solver.

Models:

- ALGEBRAIC — Simple algebraic model (Adamsson and Le Corre [2014], Equations 21 or 23)

- EQUILIBRIUM — Equilibrium model
- FULL — Full non-equilibrium momentum model (Le Corre [2022])

class InputEnums.EQSTROUHAL

EQSTROUHAL Enumeration of wave equilibrium Strouhal number models

This class defines the available `Inputs.Model.EQSTROUHAL` models for calculating the wave equilibrium Strouhal number, used by the four-field solver.

Models:

- RISO — Based on RISO dataset (Le Corre [2022])
- SAWAI — Based on SAWAI dataset (Le Corre [2022])
- MFVAL — MFVAL model (under development)
- CUSTOM — Custom model using user-defined `Inputs.Model.EQSTROUHALCOEF`
- CSTFRQ — Custom constant frequency using user-defined `Inputs.Model.CSTWAVEFREQ`

class InputEnums.MOMENTFILM

MOMENTFILM Enumeration of film momentum conservation models

This class defines the available `Inputs.Model.MOMENTFILM` models for solving the momentum conservation equation for the liquid film, used to compute film velocity in the three-field solver.

Models:

- ALGEBRAIC — Simple algebraic model
- EQUILIBRIUM — Equilibrium model
- EQUILIBRIUMS — Simplified equilibrium model based on force balance ($F_{\text{wall}} + F_{\text{vapor}} = 0$)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a])

class InputEnums.VAPORFRIC

VAPORFRIC Enumeration of vapor to film interfacial friction models

This class defines the available `Inputs.Model.VAPORFRIC` models for calculating the vapor-to-film interfacial friction coefficient used by the three-field and four-field solvers.

Models:

- CONSTANT — Uses a fixed, friction coefficient using user-defined `Inputs.Model.VAPORFRICCST`
- WALLIS — Wallis model based on void fraction (Wallis [1969]).
- WALLISTHICK — Wallis model based on film thickness (Adamsson and Le Corre [2011], Equation 35).
- SOLVER_DEPENDENT — Friction model determined by the solver configuration.

class InputEnums.VOID

VOID Enumeration of void fraction models

This class defines the available `Inputs.Model.VOID` models to calculate the void fraction in the mixture solver.

Models:

- HOMOGENEOUS — Assumes vapor and liquid phases move at the same velocity.
- SLIP — Assumes a slip ratio between vapor and liquid phases using user-defined `Inputs.Model.SLIP`.
- BESTION — Drift-flux model developed by Bestion (Bestion [1990]).

- EPRI — Drift-flux model developed by the Electric Power Research Institute (EPRI) (Lellouche and Zolotar [1982]).

class InputEnums.**MOMENTBASE**

MOMENTBASE Enumeration of base film momentum conservation models

This class defines the available `Inputs.Model.MOMENTBASE` models for solving the base film momentum conservation equation, used to compute the base film velocity in the four-field solver.

Models:

- ALGEBRAIC — Simple algebraic model (Adamsson and Le Corre [2014], Equations 21 or 23)
- EQUILIBRIUM — Equilibrium model
- EQUILIBRIUMS — Simplified equilibrium model based on force balance ($F_{\text{wall}} + F_{\text{vapor}} = 0$)
- FULL — Full non-equilibrium model (Le Corre [2022])
- FULLNOP — Full non-equilibrium model excluding pressure terms (gravity and buoyancy)

class InputEnums.**INTLENGTH**

INTLENGTH Enumeration of interfacial length scale models

This class defines the available `Inputs.Model.INTLENGTH` models for calculating the interfacial length scale, used by the two-fluid solver.

Models:

- CONSTANT — Assumes a constant interfacial length scale using user-defined `Inputs.Model.INTLENGTHVCST` and `Inputs.Model.INTLENGTHLCST` for the dispersed gas and liquid phases, respectively

class InputEnums.**BTHTM**

BTHTM Enumeration of post-boiling transition wall heat transfer models

This class defines the available `Inputs.Model.BTHTM` models for calculating the post-boiling transition wall heat transfer, used by the mixture solver.

Models:

- VAPOR — Wall heat transfer to the vapor phase only, based on single-phase heat transfer model selected in `Inputs.Model.SPHTM`
- DOUGALL — Dougall-Rohsenow wall heat transfer model (Dougall and Rohsenow [1963])
- BISHOP — Bishop-Sandberg-Tong wall heat transfer model (Bishop *et al.* [1965])
- MOECK — Groeneveld-Moeck wall heat transfer model (Groeneveld and Moeck [1969])
- DELORME — Groeneveld-Delorme wall heat transfer model (Groeneveld and Delorme [1976])
- CONDIEIV — Condie-Bengston IV wall heat transfer model (Morris *et al.* [1982])

class InputEnums.**ENTRAINMENT**

ENTRAINMENT Enumeration of droplet entrainment models

This class defines the available `Inputs.Model.ENTRAINMENT` models for calculating the droplet entrainment mass flux, used by the three-field and four-field solvers.

NOTE: Entrainment and deposition correlations are coupled. It is recommended to use the same model for both entrainment and deposition mass flux to ensure consistency.

Models:

- NONE — No entrainment

- GOVAN — Hewitt and Govan correlation (Hewitt and Govan [1990])
- OKAWA2003 — Okawa et al. correlation (Okawa *et al.* [2003])
- OKAWA2004 — Okawa et al. correlation (Okawa *et al.* [2004])
- OKAWA2004MOD — Modified Okawa (2004) model from (Adamsson and Le Corre [2011])
- OKAWAGEN — Generic Okawa-based model using user-defined `Inputs.Model.OKAWACOEFS`

class InputEnums.SPHTM

SPHTM Enumeration of single-phase wall heat transfer models

This class defines the available `Inputs.Model.SPHTM` models for calculating the single-phase wall heat transfer (liquid or gas), used by the mixture solver.

Models:

- DITTUSBOELTER — Dittus-Boelter wall heat transfer model for heating ($Nu = 0.023 \cdot Re^{0.8} \cdot Pr^{0.4}$) (Dittus and Boelter [1930], McAdams [1942])
- DITTUSBOELTERGEN — Generalized Dittus-Boelter wall heat transfer model using user-defined `Inputs.Model.DITTUSBOELTERCOEF`
- SIEDERTATE — Sieder-Tate wall heat transfer model ($Nu = 0.027 \cdot Re^{0.8} \cdot Pr^{1/3} \cdot \left(\frac{\mu_b}{\mu_w}\right)^{0.14}$) (Sieder and Tate [1936])
- GNIELINSKI — Gnielinski wall heat transfer model (Gnielinski [1976])

class InputEnums.WAVEFREQUENCY

WAVEFREQUENCY Enumeration of wave number density transport models

This class defines the available `Inputs.Model.WAVEFREQUENCY` models for solving the wave number density (or wave frequency) transport equation, used in the four-field solver.

Models:

- EQUILIBRIUM — Assumes instantaneous equilibrium between wave generation and dissipation.
- RELAXATION — Full transport model including time relaxation effects set by user-defined RELAXTW (Le Corre [2022])

class InputEnums.TPFM

TPFM Enumeration of two-phase wall friction multiplier models

This class defines the available `Inputs.Model.TPFM` models for calculating the two-phase wall friction multiplier used by the mixture solver.

Models:

- HOMOGENEOUS — Assumes no slip between phases; both phases move at the same velocity
- SLIP — Accounts for velocity differences (slip) between liquid and vapor phases using user-defined `Inputs.Model.SLIP`
- EPRI — Empirical model developed by the Electric Power Research Institute (EPRI) (Reddy and others [1982])

class InputEnums.TPKM

TPKM Enumeration of two-phase local loss multiplier models

This class defines the available `Inputs.Model.TPKM` models for calculating the two-phase local loss multiplier used by the mixture model

Models:

- HOMOGENEOUS — Assumes no slip between phases; both phases move at the same velocity.
- SLIP — Accounts for velocity differences (slip) between liquid and vapor phases using user-defined [*Inputs.Model.SLIP*](#).
- ROMIE — Romie model; behaves like the homogeneous model when a homogeneous void fraction model is applied.

class InputEnums.**INTTRANSH**

INTTRANSH Enumeration of interfacial enthalpy transfer models

This class defines the available [*Inputs.Model.INTTRANSH*](#) models for calculating the phase enthalpy associated with interfacial mass transfer, used by the mixture and two-fluid solvers.

Models:

- BULK — Uses bulk phase enthalpy
- SATURATED — Uses saturated phase enthalpy

class InputEnums.**SPMTM**

SPMTM Enumeration of single-phase wall momentum transfer models

This class defines the available [*Inputs.Model.SPMTM*](#) models for calculating the single-phase wall momentum transfer (liquid or gas), used by the mixture solver.

Models:

- BLASIUS — Blasius wall momentum transfer model ($f = C(1)Re^{C(2)} + C(3)$) using user-defined [*Inputs.Model.FRICTION*](#) coefficients

class InputEnums.**INTAREA**

INTAREA Enumeration of volumetric interfacial area calculation models

This class defines the available [*Inputs.Model.INTAREA*](#) models for calculating the volumetric interfacial area, used by the two-fluid solver.

Models:

- DISPGAS2DISPLIQ — Transition from spherical bubbles to spherical droplets (Walter [2024])

class InputEnums.**MRMTIMEINT**

MRMTIMEINT Enumeration of time integration options for the MRM vapor mass/energy conservation equations

This class defines the available [*Inputs.Options.MRMTIMEINT*](#) options for solving the vapor mass and energy conservation equations when using the MRM model in the mixture solver.

Models:

- EULER — Backward Euler
- EXPONENTIAL — Exponential

3.2 Inputs

The [*Inputs*](#) package defines a set of classes used to configure and import all the necessary parameters for running OpenSTREAM simulations. This includes:

- Boundary conditions: [*Inputs.BoundaryConditions*](#)
- System geometry: [*Inputs.Geometry*](#)
- Physical models: [*Inputs.Model*](#)

- Numerical options: [Inputs.Options](#)
- Fluid thermophysical properties: [Inputs.FluidProperties](#)
- Input handling: [Inputs.Input](#)
- Complete input set management: [Inputs.InputSet](#)

These input definitions form the backbone of each simulation setup, ensuring consistency, flexibility, and clarity across different solver frameworks.

class `Inputs.BoundaryConditions`(*filePath*, *geometryObjInput*)

Bases: [Inputs.Input](#)

BOUNDARYCONDITIONS Class for defining and managing boundary condition parameters

This class provides access to boundary condition inputs and saturated fluid properties at system pressure. It is used by the solvers to initialize and manage inlet conditions, power distributions, and fluid state calculations.

Key Features:

- Parses input files and assigns values to boundary condition fields.
- Interfaces with fluid property libraries to compute saturation and inlet thermodynamic states.
- Validates consistency between geometry and power mesh definitions.
- Supports plotting of boundary condition time series.

Constructor Summary

BoundaryConditions(*filePath*, *geometryObjInput*)

BOUNDARYCONDITIONS Constructor for BoundaryConditions class

Parses input file and initializes boundary condition parameters. Validates consistency with geometry and applies default values if needed.

Inputs:

- *filePath* — Path to input file (optional)
- *geometryObjInput* — Geometry object for validation

Property Summary

HIN = []

Inlet enthalpy [J/kg]

MFLOW = []

Mass flow rate [kg/s]

POWER = 0

Total power [W]

PRESSURE = []

System pressure [Pa]

TIME

Simulation time [s]

WMESH = 1

Relative power node size distribution [m]

WPOWER = 1

Relative power distribution(s) [-]

Method Summary

DHIN(*fluidObj*)

DHIN Inlet subcooling enthalpy (HF - HIN) [J/kg]

DTIN(*fluidObj*)

DTIN Inlet subcooling temperature (TSAT - TIN) [K]

HF(*fluidObj*)

HF Liquid saturated enthalpy at system pressure using fluid object [J/kg]

HG(*fluidObj*)

HG Vapor saturated enthalpy at system pressure using fluid object [J/kg]

TIN(*fluidObj*)

TIN Inlet temperature based on inlet enthalpy and pressure [K]

TSAT(*fluidObj*)

TSAT Saturation temperature at system pressure using fluid object [K]

XIN(*fluidObj*)

XIN Inlet equilibrium quality [-]

plot(*fluidObj*, *opt*)

PLOT Generates plots of boundary condition parameters over time

Inputs:

- *fluidObj* — FluidProperties object
- *opt* — Struct with fields:
 - *display* — Cell array of parameters to plot
 - *tIdx* — Time indices to include
 - *unitTemp* — Temperature unit ('K' or 'C')

static writeInputFile(*filePathName*, *TIME*, *PRESSURE*, *HIN*, *MFLOW*, *varargin*)

WRITEINPUTFILE Writes boundary condition data to input file

Inputs:

- *filePathName* — Path to output file
- *TIME* — Time value [s]
- *PRESSURE* — Pressure value [Pa]
- *HIN* — Enthalpy value [J/kg]
- *MFLOW* — Mass flow rate [kg/s]
- *varargin* — Additional name-value pairs

class Inputs.Model(*filePath*, *modelID*)

Bases: [Inputs.Input](#)

MODEL Class for defining and managing physical model configuration

This class encapsulates all physical modeling parameters used in the simulations, including fluid properties, regime transitions, wall and interfacial exchange models for all solvers. It reads model data from input files and handles default values as well as input validation.

Constructor Summary

Model (*filePath*, *modelID*)

MODEL Constructor for Model class

Parses model input file and initializes properties. Applies default values and validates entries.

Inputs:

- *filePath* — Path to model input file
- *modelID* — Identifier for model configuration

Property Summary

ANGLE = 0

Flow axis angle from vertical [deg]

BASEEQTHICK = 'RISO'

Equilibrium base film thickness model selected from *InputEnums.BASEEQTHICK*

BASEEQTHICKCOEF = [5.37E-5-0.641.21]

Equilibrium base film thickness coefficients [-]

BASEYPLUS = 15

Equilibrium base film y^+ value [-]

BTHTM = 'VAPOR'

Boiling transition wall heat transfer model selected from *InputEnums.BTHTM*

BTMTM = 'TPFM'

Boiling transition wall momentum transfer model selected from *InputEnums.BTMTM*

BUBBLEDRAG = 'STOKES'

Bubble drag model selected from *InputEnums.BUBBLEDRAG*

BUBBLEDRAGCOEF = 0.45

Bubble constant drag coefficient [-]

CBT = 'NONE'

Critical Boiling Transition model selected from *InputEnums.CBT*

CBTELEVATION = 1

Critical Boiling Transition Elevation [m]

CBTKEFFECT = [00]

Grid effect coefficients $(1 + C(1) \exp(C(2)z))$ [-]

CBTMULT = @(z)1

Critical boiling Heat flux multiplier function

CSTWAVEFREQ = 100

Imposed constant wave frequency [Hz]

DEPENHANCEMENT = 'NONE'

Drop deposition enhancement model due to local perturbations selected from *InputEnums.DEPENHANCEMENT*

DEPOSITION = 'OKAWA'

Drop deposition model selected from *InputEnums.DEPOSITION*

DITTUSBOELTERCOEF = [0.0230.80.4]

Dittus-Boelter coefficients [-]

DROPDIA = 1E-3

Drop diameter [mm]

DROPDRAG = 'VISCOUS'

Drop drag model selected from *InputEnums.DROPDRAG*

DROPDRAGCOEF = 0.45

Drop constant drag coefficient [-]

DROPSLIP = 1.0

Drop/vapor velocity ratio [-]

DTMFB = 100

Minimum film boiling temperature from saturation [K]

ENTRAINMENT = 'OKAWA2003'

Film entrainment model selected from *InputEnums.ENTRAINMENT*

EQSTROUHAL = 'RISO'

Equilibrium wave Strouhal number model selected from *InputEnums.EQSTROUHAL*

EQSTROUHALCOEF = [1.1236E-40.50.0]

Equilibrium wave Strouhal number coefficients [-]

FLUID = 'WATER'

CoolProp fluid identifier

FRICTION = [0.2-0.20]

Wall friction coefficients [-]

G = 9.81

Gravitational acceleration [m/s^2]

ID

Identifier for the model configuration

INTAREA = 'DISPGAS2DISPLIQ'

Interfacial area model selected from *InputEnums.INTAREA*

INTLENGTH = 'CONSTANT'

Interfacial length scale model selected from *InputEnums.INTLENGTH*

INTLENGTHLCST = 2E-3

Dispersed liquid constant interfacial length scale [m]

INTLENGTHVCST = 2E-3

Dispersed gas constant interfacial length scale [m]

INTNU = 'RELAXATION'

Interfacial heat transfer model selected from *InputEnums.INTNU*

INTNULCST = 2.0

Dispersed liquid constant interfacial Nusselt number [-]

INTNUVCST = 2.0

Dispersed gas constant interfacial Nusselt number [-]

INTTRANSH = 'BULK'

Interfacial enthalpy transfer model selected from *InputEnums.INTTRANSH*

KBLOCKRATIO = [00]

Blockage ratios of local perturbations [-]

KLOC = [00]

Elevation of local perturbations [m]

KLOSS = [00]

Pressure loss coefficients of local perturbations [-]

KTRELAX = NaN

Thermal relaxation time at local perturbations [s]

KTUNING = [00]

Drop deposition enhancement tuning coefficients [-]

LOCRELVEL = 'AREAMEAN'

Local relative phase velocity model selected from *InputEnums.LOCRELVEL*

MFBT = 'NONE'

Minimum Film Boiling Transition model selected from *InputEnums.MFBT*

MOECKCOEF = [1.09E-30.9891.41-1.15]

Groeneveld-Moeck post-boiling transition model coefficients [-]

MOMENTBASE = 'FULLNOP'

Base film momentum conservation model selected from *InputEnums.MOMENTBASE*

MOMENTDROP = 'SLIP'

Drop momentum conservation model selected from *InputEnums.MOMENTDROP*

MOMENTFILM = 'ALGEBRAIC'

Film momentum conservation model selected from *InputEnums.MOMENTFILM*

MOMENTGAS = 'MIXTURE'

Gas momentum conservation model selected from *InputEnums.MOMENTGAS*

MOMENTLIQUID = 'MIXTURE'

Liquid momentum conservation model selected from *InputEnums.MOMENTLIQUID*

MOMENTWAVE = 'FULL'

Wave momentum conservation model selected from *InputEnums.MOMENTWAVE*

NEARWALLEQOAF = false

Near-wall equilibrium at onset of annular two-phase flow

NEARWALLH = 'RATIO'

Near-wall equilibrium enthalpy model selected from *InputEnums.NEARWALLH*

NEARWALLHRATIO = 1.0

Near-wall equilibrium enthalpy ratio [-]

NEARWALLRATIO = 0.5

Near-wall mass flow distribution ratio [-]

NEARWALLRELAX = 'QUALITY'

Near-wall energy transfer time relaxation model selected from *InputEnums.NEARWALLRELAX*

NEARWALLRELAXCOEF = [0.1E-31/30.05]

Near-wall energy transfer time relaxation coefficients for void option

NEARWALLRELAXT = [1.00.50.30.10.1]

Near-wall energy transfer relaxation time array [s]

NEARWALLRELAXX = [-0.5-0.25-0.10.01.0]

Near-wall exchange relaxation time thermodynamic quality [-]

NNODES = 100

Number of axial nodes used in the simulation

OAF = 'WALLIS'

Onset of annular flow model selected from *InputEnums.OAF*

OAFBASERATIO = 0.5

Base/Film mass ratio at onset of annular flow [-]

OAFDROPRATIO = 0.7

Drop/Liquid mass ratio at onset of annular flow [-]

OAFENTRAINED = 'EQUILIBRIUM'

Entrained drop model at onset of annular flow selected from *InputEnums.OAFENTRAINED*

OAFFILMSPLIT = 'EQUILIBRIUM'

Film mass flow split model at onset of annular flow selected from *InputEnums.OAFFILMSPLIT*

OAFTRANSITION = [0.100.0]

Annular flow transition function parameters (sigmoid width/location wrt OAF) [m]

OKAWACOEFS = [3200.1114.79E-41]

Coefficients of Okawa entrainment model [-]

POSFILM = true

Positive film flow rate/thickness model

PROPERTIES = 'SATURATED'

Assumption model for fluid properties selected from *InputEnums.FLUIDPROPERTIES*

RANZMARSHALLCST = [20.61/21/3]

Dispersed liquid Ranz-Marshall coefficients [-]

RANZMARSHALLVCST = [20.61/21/3]

Dispersed gas Ranz-Marshall coefficients [-]

RELAXCONDCOEF = [0.1E-31/30.050.0]

Interfacial condensation time relaxation coefficients for homogeneous models

RELAXCONDFO = [1.01.01.01.01.0].*1E-4

Interfacial condensation Fourier number array [s]

RELAXCONDFOCOEF = [5E-55E500]

Interfacial condensation time relaxation coefficients for empirical Fourier number model

RELAXCONDT = [1.00.50.30.10.1]

Interfacial condensation relaxation time array [s]

RELAXEVAPCOEF = [0.1E-31/31E-50.0]

Interfacial evaporation time relaxation coefficients for homogeneous models

RELAXEVAPFO = [1.01.01.01.01.0].*5E-5

Interfacial evaporation Fourier number array [s]

RELAXEVAPFOCOEF = [1E-45E52210E-31]

Interfacial evaporation time relaxation coefficients for empirical Fourier number model

RELAXEVAPT = [0.30.30.30.30.3]

Interfacial evaporation relaxation time array [s]

RELAXTB = 0.2

Base film / wave mass exchange relaxation time [s]

RELAXTW = 0.2

Wave number density relaxation time [s]

RELAXX = [-0.5-0.25-0.10.01.0]

Interfacial phase change relaxation time thermodynamic quality [-]

RELVELCST = 1.0

Multiplication factor to the local relative velocity [-]

SHAPEFACTORCOEF = [1.325E5210]

Wave shape factor coefficients [-]

SLIP = 1

Phase velocity ratio [-]

SOLVERDEPENDENTPROPS = struct("VAPORFRIC", ...)

Mapping of solver-dependent properties and their values

SPHTM = 'DITTUSBOELTER'

Single-phase wall heat transfer model selected from *InputEnums.SPHTM*

SPMTM = 'BLASIUS'

Single-phase wall momentum transfer model selected from *InputEnums.SPMTM*

THERMALNONEQ = 'EQUILIBRIUM'

Thermal non-equilibrium model selected from *InputEnums.THERMALNONEQ*

THERMALRELAX = 'TIMEX'

Thermal non-equilibrium time relaxation model selected from *InputEnums.THERMALRELAX*

THINFILMFRIC = 'TURBULENT'

Thin film wall friction model selected from *InputEnums.THINFILMFRIC*

THINFILMTHICK = 1E-4

Minimum thin film thickness [m]

THINWAVETHICK = 1E-5

Minimum thin wave thickness [m]

TPFM = 'HOMOGENEOUS'

Two-phase friction multiplier model selected from *InputEnums.TPFM*

TPHTM = 'THOM'

Two-phase wall heat transfer model selected from *InputEnums.TPHTM*

TPKM = 'HOMOGENEOUS'

Two-phase local loss multiplier model selected from *InputEnums.TPKM*

VAPORFRIC = 'SOLVER_DEPENDENT'

Vapor friction model selected from *InputEnums.VAPORFRIC*

VAPORFRICCST = 0.005

Vapor friction constant [-]

VOID = 'HOMOGENEOUS'

Void fraction model selected from *InputEnums.VOID*

WAVEBASEINT = 'VAPORSHEAR'

Wave / base film interfacial momentum transfer model selected from *InputEnums.WAVEBASEINT*

WAVEDRAGCOEF = [0.021.350E50.437]

Wave drag coefficient [-]

WAVEFREQUENCY = 'RELAXATION'

Wave number density model selected from *InputEnums.WAVEFREQUENCY*

WAVEMIXCOEF = 2

Base film / wave turbulent mixing coefficient

WBOILINGN = 2

Wall boiling function exponent [-]

WBOILINGXSAT = 0

Equilibrium thermodynamic quality at onset of saturated wall boiling [-]

WBOILINGXSUB = -0.2

Equilibrium thermodynamic quality at onset of subcooled wall boiling [-]

Method Summary

static writeInputFile(filePathName, ID, varargin)

WRITEINPUTFILE Writes model configuration data to input file

Inputs: - filePathName — Path to output file - ID — Model identifier - varargin — Additional name-value pairs for model properties

class Inputs.Options(filePath, optionsID)

Bases: *Inputs.Input*

OPTIONS Class for defining and managing numerical solver options

This class encapsulates all numerical parameters used in the simulations, including time stepping, convergence criteria, and relaxation factors for all solvers. It reads options data from input files and handles default values as well as input validation.

Constructor Summary

Options(*filePath*, *optionsID*)

MODEL Constructor for Options class

Parses options input file and initializes properties. Applies default values and validates entries.

Inputs:

- *filePath* — Path to options input file
- *optionsID* — Identifier for options configuration

Property Summary

AXIALINTERP = 'next'

Method for interpolating axial power distribution

ERRORFW = 1E-1

Wave frequency error target in inner iterations [Hz]

ERRORH = 1E-1

Enthalpy error target in inner iterations [J/kg]

ERRORP = 1E-1

Pressure error target in inner iterations [Pa]

ERRORU = 1E-4

Velocity error target in inner iterations [m/s]

ERRORUD = 1E-2

Drop velocity error target in inner iterations [m/s]

ERRORUF = 1E-2

Film velocity error target in inner iterations [m/s]

ERRORW = 1E-3

Mass flow rate error target in inner iterations [kg/s]

ERRORWF = 1E-4

Film mass flow rate error target in inner iterations [kg/s/m]

ID

Identifier for the options configuration

MAXITER = 100

Maximum number of inner iterations per time step

MRMTIMEINT = 'EXPONENTIAL'

Time integration for vapor mass/energy conservation equations

RELAXFW = 0.5

Relaxation factor for the wave number conservation equation [-]

RELAXHL = 1

Relaxation factor for the liquid energy conservation equation [-]

RELAXHM = 1.0

Relaxation factor for the mixture energy conservation equation [-]

RELAXHV = 0.7

Relaxation factor for the vapor energy conservation equation [-]

RELAXPM = 1.0

Relaxation factor for the mixture momentum conservation equation [-]

RELAXUB = 0.2

Relaxation factor for the base momentum conservation equation [-]

RELAXUD = 0.2

Relaxation factor for the drop momentum conservation equation [-]

RELAXUF = 0.2

Relaxation factor for the film momentum conservation equation [-]

RELAXUL = 0.5

Relaxation factor for the liquid momentum conservation equation [-]

RELAXUV = 0.5

Relaxation factor for the vapor momentum conservation equation [-]

RELAXUW = 0.2

Relaxation factor for the wave momentum conservation equation [-]

RELAXWB = 0.5

Relaxation factor for the base mass conservation equation [-]

RELAXWF = 0.5

Relaxation factor for the film mass conservation equation [-]

RELAXWL = 1

Relaxation factor for the liquid mass conservation equation [-]

RELAXWM = 1.0

Relaxation factor for the mixture mass conservation equation [-]

RELAXWV = 0.7

Relaxation factor for the vapor mass conservation equation [-]

RELAXWW = 0.5

Relaxation factor for the wave mass conservation equation [-]

SSCONVFW = 1E-1

Wave frequency steady-state convergence criterion [Hz]

SSCONVH = 1E+0

Enthalpy steady-state convergence criterion [J/kg]

SSCONVP = 1E+0

Pressure steady-state convergence criterion [Pa]

SSCONVU = 1E-3

Velocity steady-state convergence criterion [m/s]

SSCONVUD = 1E-2

Drop velocity steady-state convergence criterion [m/s]

SSCONVUF = 1E-2

Film velocity steady-state convergence criterion [m/s]

SSCONVW = 1E-3

Mass flow rate steady-state convergence criterion [kg/s]

SSCONVWF = 1E-4

Film mass flow rate steady-state convergence criterion [kg/s/m]

SSMAXITER = 30

Maximum number of steady-state iterations

SSTSTEP = 1.0

Time step size for steady-state iterations [s]

TIMEINTERP = 'linear'

Method for interpolating time-dependent boundary conditions

TSTEP = 0.1

Time step size for transient simulations [s]

Method Summary

static writeInputFile(filePathName, ID, varargin)

Writes numerical options configuration to input file

Inputs:

- filePathName — Path to output file
- ID — Options identifier
- varargin — Additional name-value pairs for options properties

class Inputs.Geometry(filePath, geometryID)

Bases: [Inputs.Input](#)

GEOMETRY Class for defining and managing geometrical input parameters

This class reads and stores geometrical parameters from an input file. It supports calculations of derived quantities such as hydraulic diameter, area-based diameter, and wall perimeter ratios.

Constructor Summary

Geometry(filePath, geometryID)

GEOMETRY Constructor for Geometry class

Parses geometry input file and initializes properties. Applies default values if no input is provided.

Inputs:

- filePath — Path to geometry input file
- geometryID — Identifier for geometry configuration

Property Summary

ANGLE = 0

Inclination angle [rad]

AREA = 1

Coolant area [m^2]

ID

Channel identifier

LENGTH = 1

Axial length [m]

PERIM = 1

Perimeter(s) of the channel walls [m]

Method Summary

ADIAM()

ADIAM Diameter based on coolant cross-section area [m]

Formula: $2\sqrt{\frac{AREA}{\pi}}$

HDIAM()

HDIAM Hydraulic diameter [m]

Formula: $4\frac{Area}{Totalperimeter}$

NWALL()

NWALL Number of wall segments defined by PERIM

RWALL()

RWALL Relative contribution of each wall segment to total perimeter [-]

static writeInputFile(filePathName, ID, varargin)

Writes geometry input data to file

Inputs:

- filePathName — Path to output file
- ID — Geometry identifier
- varargin — Additional name-value pairs for geometry properties

class Inputs.InputSet(opts)

INPUTSET Class for managing a complete set of input objects

This class initializes and stores all input components required for a simulation run. It handles model, options, geometry, and boundary condition inputs, and sets up logging via a Session object. It also supports applying solver-dependent property modifications.

Constructor Summary

InputSet(opts)

INPUTSET Constructor for InputSet class

Initializes all input objects and sets up logging session. Parses input files and handles warnings.

Inputs:

- opts — Struct with fields:
 - modelFilePath, modelID
 - optionsFilePath, optionsID
 - geometryFilePath, geometryID
 - bcFilePath
 - LOGMODE, sessionName, sessionDirName
 - sessionParentDir, overwriteSessionFiles

Property Summary

bc
BoundaryConditions input object

geometry
Geometry input object

model
Model input object

options
Options input object

session
Session object for logging and file management

Method Summary

applySolverDependentProps(*inputSet, solverName*)
APPLYSOLVERDEPENDENTPROPS Applies solver-dependent property modifications to all input objects

Useful for customizing inputs based on selected solver

Inputs:

- *inputSet* — InputSet object
- *solverName* — Name of the solver to apply dependencies for

class Inputs.Input(*inputFilePath, key, val*)

Bases: dynamicprops, matlab.mixin.Copyable

INPUT Abstract superclass for input-handling classes

Provides core functionality for reading, validating, and managing structured input data. Supports dynamic property creation, default value handling, and solver-dependent customization.

Constructor Summary

Input(*inputFilePath, key, val*)
INPUT Constructor for Input class

Parses input file and optionally filters entries by key-value pair. Adds dynamic property 'inputStruct' to hold parsed data.

Inputs:

- *inputFilePath* — Path to input file (string)
- *key* — Field name to filter by (optional)
- *val* — Value to match for filtering (optional)

Property Summary

extra = struct.empty()
Stores unused input entries for reference or debugging

warnings = struct.empty()
Stores warning messages generated during input parsing

Method Summary

applySolverDependentProperties(*solverName*)

APPLYSOLVERDEPENDENTPROPERTIES Applies solver-specific property values based on dependency flags

Returns a copy of the object with updated properties

static convert2JSON(*inputFilePath*)

CONVERT2JSON Converts input file contents to JSON-formatted string

defaultValueUsedReport(*propNames, propValues*)

DEFAULTVALUEUSEDREPORT Generates report of properties for which default values were used

Can print or return formatted string array

listInputProperties(*opts*)

LISTINPUTPROPERTIES Returns list of protected property names for the object

Optionally excludes specified properties

static validateFunctionHandleInput(*handleString*)

VALIDATEFUNCTIONHANDLEINPUT Validates safety of function handle string

Throws error if restricted keywords are detected

validateInputEntry(*objPropname, opts*)

VALIDATEINPUTENTRY Validates whether a property entry is specified or defaulted

Throws error if required entry is missing or empty Returns flags and default value if applicable

static writeInputFile(*filePathName, fidMode, varargin*)

WRITEINPUTFILE Writes input data to file in standard format

Accepts name-value pairs and optional file mode

class Inputs.FluidProperties(*P, modelObj*)

FLUIDPROPERTIES Class for managing thermophysical properties of a fluid

This class encapsulates saturated and generic fluid properties used in thermal-hydraulic simulations. Saturated properties are computed based on system pressure and stored as class properties. Generic properties (e.g., temperature, density, viscosity) are computed dynamically using enthalpy and pressure.

Constructor Summary**FluidProperties**(*P, modelObj*)

FLUIDPROPERTIES Constructor for FluidProperties class

Initializes fluid properties at given pressure(s) using CoolProp. Saturated properties are computed and assigned to each object instance.

Inputs:

- *P* — Vector of system pressures [Pa]
- *modelObj* — Model object containing fluid name and property assumptions

Property Summary**ALPHA_F = 1**

Saturated liquid thermal diffusivity [m²/s]

ALPHA_G = 1

Saturated vapor thermal diffusivity [m²/s]

CPF = 1
 Saturated liquid constant pressure specific heat [J/kg/K]

CPG = 1
 Saturated vapor constant pressure specific heat [J/kg/K]

FLUID = 'WATER'
 Coolprop fluid identifier

HF = 1
 Saturated liquid enthalpy [J/kg]

HFG = 0
 Latent heat of evaporation [J/kg]

HG = 1
 Saturated vapor enthalpy [J/kg]

KF = 1
 Saturated liquid thermal conductivity [W/m/K]

KG = 1
 Saturated vapor thermal conductivity [W/m/K]

MUF = 1
 Saturated liquid viscosity [Pa.s]

MUG = 1
 Saturated vapor viscosity [Pa.s]

PCRIT = 1
 Critical pressure [Pa]

PRANDTLF = 1
 Saturated liquid Prandtl number [-]

PRANDTLG = 1
 Saturated vapor Prandtl number [-]

PRESSURE = 1
 System pressure [Pa]

PROPERTIES = 'SATURATED'
 Fluid property assumptions from *Inputs.Model.PROPERTIES*

PTRIPLE = 1
 Triple point pressure [Pa]

QK = 1
 Kutateladze characteristic heat flux [W/m²]

RHOF = 1
 Saturated liquid mass density [kg/m³]

RHOG = 1
 Saturated vapor mass density [kg/m³]

SIGMA = 1

Surface tension [N/m]

TSAT = 1

Saturated fluid temperature [K]

UC = 1

Kutateladze vapor critical velocity [m/s]

Method Summary

ALPHAL(H)

ALPHAL Liquid thermal diffusivity [m²/s]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

ALPHAV(H)

ALPHAV Vapor thermal diffusivity [m²/s]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

CPL(H)

CPL Liquid constant pressure specific heat [J/kg/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

CPV(H)

CPV Vapor constant pressure specific heat [J/kg/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

H(T)

H Fluid enthalpy at given temperature and system pressure [J/kg]

KL(H)

KL Liquid thermal conductivity [W/m/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

KV(H)

KV Vapor thermal conductivity [W/m/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

MUL(H)

MUL Liquid dynamic viscosity [Pa.s]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

MUV(H)

MUV Vapor dynamic viscosity [Pa.s]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

PRANDTLL(H)

PRANDTLL Liquid Prandtl number [-]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

PRANDTLV(H)

PRANDTLV Vapor Prandtl number [-]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

RHOL(*H*)

RHOL Liquid mass density [kg/m³]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

RHOV(*H*)

RHOV Vapor mass density [kg/m³]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

T(*H*)

T Fluid temperature at given enthalpy and system pressure [K]

Enthalpy is clamped between HMIN and HMAX to ensure valid CoolProp input

plot(*H*)

PLOT Plot properties for given enthalpy vector

transient(*param, opt*)

TRANSIENT Generate transient distribution array for parameter param

3.3 Session

The *Session* package in **OpenSTREAM** provides tools for managing session environments, logging, and file structure during simulations. It helps ensure organized workflows and consistent runtime behavior. This includes:

- Session logging manager *Session.Log*
- Logging mode: *Session.LogMode*
- Session management: *Session.Session*

class Session.Log(*LOGMODE, opts*)

Bases: `handle`

LOG Console and file-based logging manager

This class manages logging of messages and warnings to console and/or file. It supports multiple logging modes, integrates with session management, and provides diary functionality for persistent logs.

Constructor Summary

Log(*LOGMODE, opts*)

LOG Constructor for Log class

Initializes logging mode, session context, and warning settings. Creates log file and session directory if needed.

Inputs:

- *LOGMODE* — Determines where logs go (console, file, both, or none), using *Session.LogMode*.
- *opt.session* — Session object for directory and naming context
- *opt.LOGFID* — File ID for the open log file.
- *opt.showWarnings* — Whether to display warnings in the console.

Property Summary

LOGFID = -1

File ID for log file

LOGMODE

Logging mode (console only, file only, both, or none)

diaryFilePath

Full path to the diary file

diaryIsOn = false

Flag indicating whether MATLAB diary is active

logFileName

Name of the log file (without extension)

logFilePath

Full path to the log file

showWarnings = true

Flag to control display of warnings

Method Summary**closeLog()**

CLOSELOG Closes log file and resets file ID

delete()

DELETE Destructor for log class

Ensures log file and diary are properly closed

diaryOff()

DIARYOFF Turn off diary logging

diaryOn()

DIARYON Turn on diary logging if applicable

log(varargin)

LOG Logs messages to console and/or file depending on LOGMODE

openLog(opts)

OPENLOG Opens log file for writing

Optionally keeps it open between writes

warning(varargin)

WARNING Logs warnings using MATLAB's built-in warning mechanism

Optionally writes to file if warnings are suppressed

Session.isLegalPath(pathname)

ISLEGALPATH Checks if the given pathname is valid on the current operating system

This function attempts to convert the input pathname to a Java Path object. If the conversion succeeds, the path is considered legal and the function returns true. If an exception is thrown during conversion, the path is considered illegal and the function returns false.

Input:

- **pathname** — String representing the file or folder path to validate

Output:

- **bool** — Logical true if the path is valid, false otherwise

class Session.**Session**(*opt*)

Bases: handle

SESSION Class for managing simulation session metadata and logging

This class handles session naming, directory creation, and logging setup. It ensures that session directories are valid and manages file overwriting behavior. It also integrates with the Log class to control output and warnings.

Constructor Summary

Session(*opt*)

SESSION Constructor for Session class

Initializes session metadata including name, directory, and overwrite behavior.

Inputs:

- *opt.name* — Session name
- *opt.dirName* — Directory name
- *opt.parentDir* — Parent directory path
- *opt.overwriteFiles* — Flag to allow overwriting existing session files

Property Summary

dirName = ""

Name of the session directory

directory

Full path to the session directory

log

Log object for managing output and warnings

name = ""

Name of the session (used for log file naming)

overwriteFiles = **false**

Flag to allow overwriting existing session files

parentDir = ""

Parent directory where session folder will be created

showWarnings = **true**

Flag to control display of warnings

Method Summary

makeSessionDirectory()

MAKESESSIONDIRECTORY Creates the session directory

Validates path legality, handles existing directories, closes open files and diaries if necessary, and creates the directory.

setupLog(*session*, *LOGMODE*, *opts*)

SETUPLOG Initializes the Log object for the session

Inputs:

- *LOGMODE* — Logging mode (console, file, both, none)
- *opts.LOGFID* — Optional file ID for logging

class Session.LogMode

Bases: uint16

LOGMODE Enumeration of supported logging modes

This enumeration defines the available logging configurations for the simulation framework. It is used to control whether messages and warnings are printed to the console, written to a file, both, or suppressed entirely.

3.4 Solver

The *Solvers* package defines the **OpenSTREAM** core solver modules and their associated superclasses. Each solver is tailored to simulate specific regimes of one-dimensional, two-phase flow, with increasing levels of physical detail and complexity.

Explore the available solver frameworks:

3.4.1 Four-field solver

The *Solvers.FourField* module in **OpenSTREAM** provides tools and classes for simulating annular two-phase flows using a four-field approach. In this method, the liquid phase is represented by three distinct fields (base film, disturbance waves, and droplets) each governed by its own set of conservation equations for mass, momentum, and energy. The vapor phase is solved using the *Solvers.Mixture* module. This framework enables detailed modeling of liquid film structures, interfacial interactions, slip velocities, and hydrodynamic non-equilibrium effects, making it particularly suitable for annular boiling two-phase flow regimes where field separation and wave dynamics play a significant role.

This module includes:

- Solver class for four-field-based flow simulations: *Solvers.FourField.FourFieldSolver*
- Base film, wave and drop field classes: *Solvers.FourField.Base*, *Solvers.FourField.Wave*, *Solvers.FourField.Drop*
- Film class: *Solvers.FourField.Film*
- Four-field primary properties: *Solvers.FourField.Base.W*, *Solvers.FourField.Base.U*, *Solvers.FourField.Base.H*, *Solvers.FourField.Wave.W*, *Solvers.FourField.Wave.U*, *Solvers.FourField.Wave.H*
- Key solver class methods: *Solvers.FourField.FourFieldSolver.initializeSolver*, *Solvers.FourField.FourFieldSolver.solve()*, *Solvers.FourField.FourFieldSolver.plotz*
- Field class methods for calculating secondary four-field parameters.

class Solvers.FourField.FourFieldSolver.FourFieldSolver(inputSet, mixSolver)Bases: *Solvers.ThreeField.ThreeFieldSolver*

FOURFIELDSOLVER Solver for initializing, solving, and visualizing four-field flow.

The FourFieldSolver class extends *Solvers.ThreeField.ThreeFieldSolver* to handle thermal-hydraulic simulations using a four-field approach.

Responsibilities:

- Initializes solver parameters from an *Inputs.InputSet* object
- Constructs *Solvers.FourField.Film*, *Solvers.FourField.Wave*, *Solvers.FourField.Base*, and *Solvers.FourField.Drop* objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Liquid film (base and wave) and drop field construction
- Field mass/momentum/energy transport modeling
- Pressure drop gradient obtained from mixture solver
- Vapor solution obtained from mixture solver
- Visualization of results across time and axial domains

Constructor Summary**FourFieldSolver**(*inputSet, mixSolver*)

FOURFIELDSOLVER Constructor for the FourFieldSolver class.

Creates a new instance of the FourFieldSolver class using the provided *Inputs.InputSet*. This constructor initializes the solver by calling the *Solvers.ThreeField.ThreeFieldSolver* constructor and then sets up all necessary solver parameters and internal data structures via *Solvers.FourField.FourFieldSolver.initializeSolver*

Input:

- inputSet — An *Inputs.InputSet* object containing model configuration, geometry, boundary conditions, and solver options.

Notes:

- If mixSolver is unsolved, it will be solved automatically.
- Initialization includes film, wave, base, and droplet objects.

Method Summary**EQUIL**(*ffSolver, flm, drp, mix, zIdx*)

EQUIL Find entrained ratio at film/drop equilibrium state [-]

Iteratively computes the entrained liquid ratio at the onset of annular flow by balancing droplet deposition and film entrainment rates.

Inputs:

- ffSolver — *Solvers.FourField.FourFieldSolver* object
- flm — Film field object (*Solvers.FourField.Film*)
- drp — Droplet field object (*Solvers.FourField.Drop*)
- mix — Mixture object (*Solvers.Mixture.Mixture*)
- zIdx — Axial index for evaluation

Algorithm:

- Starts with an initial guess for droplet mass flow (50% of liquid mass)
- Updates droplet and film flow rates iteratively
- Computes difference between deposition and entrainment rates
- Stops when error < 1e-4 kg/s/m or after 100 iterations

Notes:

- Uses interpolation for improved convergence
- Falls back to ad-hoc update if interpolation fails

initializeSolver(*ffSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs film, wave, base, and drop objects.

Sets up the solver's internal state using the provided *Inputs.InputSet* and mixture solver solutions. Includes initialization of data structures for transient and steady-state simulations.

Operations performed:

- Initializes film, wave, and base arrays for each time step
- Initializes fluid property objects

- Sets up mixture object
- Assigns initial conditions for field mass flow rates, velocities, enthalpies

Notes:

- Assumes uniform axial discretization.
- Wall heat flux computed from interpolated boundary conditions.

plott(*ffSolver, zIdx, opt*)

PLOTT Plots temporal distributions of four-field parameters at a given axial location.

Generates time-series plots of selected four-field parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- *ffSolver* — [Solvers.FourField.FourFieldSolver](#) object containing simulation data
- *zIdx* — Axial index (scalar, positive integer)
- *opt.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opt.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opt.wall* — Wall index(es) to plot
- *opt.tIdx* — Time indices to include in the plot
- *opt.reverseTime* — Logical flag to reverse time axis
- *opt.unitTemp* — Temperature unit: 'K' or 'C'
- *opt.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- *opt.resize* — Resize factor for plot scaling

Notes:

- If *opt.display* is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotz(*ffSolver, tIdx, opts*)

PLOTZ Plots spatial (axial) distributions of four-field parameters.

Generates axial plots of selected four-field parameters at specified time indices. The function supports multiple display modes, wall selections, and optional animation over time.

Inputs:

- *ffSolver* — [Solvers.FourField.FourFieldSolver](#) object containing simulation data
- *tIdx* — Time index or indices (vector of positive integers)
- *opts.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opts.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opts.wall* — Wall index(es) to plot
- *opts.zIdx* — Axial indices to include in the plot
- *opts.obstructions* — Logical flag to display obstruction locations
- *opts.annular* — Logical flag to plot annular two-phase flow parameters only from the onset of annular flow
- *opts.unitTemp* — Temperature unit: 'K' or 'C'
- *opts.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- *opts.resize* — Resize factor for plot scaling

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if *opts.obstructions* is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotzt(*ffSolver, opt*)

PLOTZT Plots 2D time/elevation distributions of four-field parameters.

Generates surface plots of selected four-field related parameters over time and axial (elevation) positions. It supports plotting for base and wave fields, and can handle both real and null (initial) transient data.

Inputs:

- `ffSolver` — [Solvers.FourField.FourFieldSolver](#) object containing simulation data
- `opt.display` — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- `opt.label` — Corresponding labels for display parameters
- `opt.unit` — Units for each parameter
- `opt.field` — Field to plot: 'film' or 'drop'
- `opt.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opt.wall` — Wall index(es) to plot
- `opt.zIdx` — Axial indices (must include at least 2)
- `opt.tIdx` — Time indices (must include at least 2)
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.shading` — Surface shading style: 'faceted', 'flat', or 'interp'
- `opt.view` — View angle for 3D plot [azimuth elevation]

Notes:

- If 'ALL' is passed to `opt.display`, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

`Solvers.FourField.FourFieldSolver.solve(ffSolver)`

SOLVE Executes the four-field solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.FourField.FourFieldSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (`solveINIT = true`)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for phase flows, velocities, and enthalpies
- Supports thermal non-equilibrium modeling
- Logs progress and outputs to session directory

class `Solvers.FourField.Drop`

Bases: [Solvers.ThreeField.Drop](#)

DROP Class for modeling drop field in four-field solver

This class encapsulates the physical and numerical properties of the drop field, including flow variables and phase interactions. It supports multiple solver models and provides methods for computing derived quantities.

All methods are inherited from [Solvers.ThreeField.ThreeFieldSolver](#)

class `Solvers.FourField.Film`(*inputSet*, *fluid*)

Bases: [*Solvers.AbstractFilm*](#)

FILM Class for modeling liquid film in four-field solver

Represents the physical and numerical properties of the liquid film in a four-field thermal-hydraulic solver. The film consists of two sub-fields: base film and wave film, which interact with vapor and droplet phases.

This class manages initialization, mass flow distribution, velocity and enthalpy calculations, and provides methods for splitting film flow at the onset of annular flow (OAF) according to model options.

Key responsibilities:

- Store solver state (time, axial positions, heat flux, evaporation flux)
- Aggregate flow properties from base and wave components
- Initialize base and wave fields and apply OAF split logic
- Support equilibrium calculations for film thickness and flow ratios

Constructor Summary

Film(*inputSet*, *fluid*)

FILM Creates a Film object

Initializes the film object with input set and fluid properties required by the four-field solver.

Inputs:

- *inputSet* — [*Inputs.InputSet*](#) object containing geometry and model options
- *fluid* — [*Inputs.FluidProperties*](#) object containing thermophysical properties

Property Summary

DT = 0

Time step size [s] from `Inputs.options.TSTEP`

H

(:,:) double {mustBeNumeric} = 1E6 % Enthalpy [J/kg]

HFLUX = 1.

Film heat flux [W/m^2]

ITR

Iteration tracking

MEVAP = -1.

Evaporation mass flux [kg/s/m^2]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from [*Inputs.Model.NNODES*](#)

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U
 (:,:) double {mustBeNumeric} = 1. % Velocity [m/s]

W
 (:,:) double {mustBeNumeric} = 1. % Mass flow rate [kg/s]

Z = 1.
 Elevation [m]

base = NaN
Solvers.FourField.Base object

wave = NaN
Solvers.FourField.Wave object

Method Summary

EQUIL(*film*, *Wf*, *zIdx*)

EQUIL Equilibrium base/wave split ratio [-]

Calculates the equilibrium split ratio of film mass flow between base and wave using base equilibrium thickness, base velocity, perimeter, and liquid density.

Inputs:

- *film* — *Solvers.FourField.Film* object
- *Wf* — Film mass flow rate [kg/s]
- *zIdx* — Axial indices to evaluate (optional)

copyFlowProperties(*srcObj*, *targetObj*, *opts*)

COPYFLOWPROPERTIES Copy flow properties between Film objects

Copies flow-related properties (base and wave fields) from a source Film object to one or more target Film objects. Ensures that spatial meshes match before copying.

Inputs:

- *srcObj* — Source *Solvers.FourField.Film* object
- *targetObj* — Target *Solvers.FourField.Film* object(s)
- *opts.all* — Logical flag to copy all properties (optional, default = false)

distributeOAFW(*film*, *Wf*, *zIdx*)

DISTRIBUTEAFW Distribute film mass flow at OAF [kg/s]

Distributes the film mass flow rate at the onset of annular flow (OAF) between base and wave fields according to the *Inputs.Model.OAFFILMSPLIT* model option.

Inputs:

- *film* — *Solvers.FourField.Film* object
- *Wf* — Film mass flow rate to distribute [kg/s] (per wall or per z-index)
- *zIdx* — Axial indices to apply distribution (optional)

initializeBaseAndWave(*film*, *WIN*, *ITR*)

INITIALIZEBASEANDWAVE Initialize base and wave fields

Initializes base and wave arrays given inlet film mass flow rate and iteration settings. Performs OAF film split, computes velocities, enthalpies, and sets initial wave frequency using equilibrium correlations.

Inputs:

- *film* — *Solvers.FourField.Film* object
- *WIN* — Inlet film mass flow rate [kg/s]

- ITR — Iteration control struct

class Solvers.FourField.Base(*film*)

Bases: *Solvers.AbstractFilm*

BASE Class for modeling liquid base film in four-field solver

This class represents the physical and numerical properties of the base film field in a four-field thermal-hydraulic solver. It handles flow variables, heat transfer, and phase interaction mechanisms between the base film and other fields (wave, drop, vapor).

The class supports initialization from an existing film object and provides methods for computing derived quantities such as mass flow fractions, interfacial fractions, and deposition/evaporation fluxes.

Constructor Summary

Base(*film*)

BASE Constructor for Base class

Initializes the base film object using properties from an existing film object. Copies solver configuration and geometry data, and allocates flow property arrays for multiple walls.

Inputs:

- film — *Solvers.AbstractFilm* object representing the parent film

Property Summary

DT = 0

Time step size [s] from `Inputs.options.TSTEP`

H = 1E6

Enthalpy [J/kg]

HFLUX = 1.

Film heat flux [W/m²]

ITR

Iteration tracking

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from `Inputs.Model.NNODES`

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

Method Summary

BETA(*base, zIdx*)

BETA Base film interfacial fraction [-]

Computes the fraction of interfacial area attributed to the base film.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

BETAP(*base, zIdx*)

BETAP Base film heat flux fraction [-]

Determines the fraction of heat flux directed to the base film. Applies suppression in pre-annular region to enforce target base film thickness at onset of annular flow (OAF).

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Applies suppression in pre-annular region to enforce target base film thickness at onset of annular flow (OAF). The entire film evaporation hence occurs in the wave field.

EPSILON(*base, zIdx*)

EPSILON Base film mass flow fraction [-]

Calculates the fraction of total film mass flow carried by the base film at specified axial locations.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

EQTHICK(*base, zIdx*)

EQTHICK Base equilibrium thickness [m]

Computes the equilibrium thickness of the base film based on *Inputs.Model.BASEEQTHICK* model.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

ETA(*base, zIdx*)

ETA Base film deposition fraction [-]

Computes the fraction of drop deposition allocated to the base film based on mass flow and interfacial fractions.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

FBASEVAPOR(*base, zIdx*)

FBASEVAPOR Vapor shear stress on base film [N/m²]

Computes the shear force exerted by the vapor phase on the base film, scaled by interfacial fraction.

Inputs:

- base — *Solvers.FourField.Base* object

- `zIdx` — Axial indices to evaluate (optional)

FDEP(*base, drop, zIdx*)

FDEP Droplet deposition force [N/m²]

Calculates the momentum transfer to the base film from droplet deposition, based on velocity difference and deposition flux.

Inputs:

- `base` — [Solvers.FourField.Base](#) object
- `drop` — [Solvers.FourField.Drop](#) object
- `zIdx` — Axial indices to evaluate (optional)

FDRY(*base, drop, zIdx*)

FDRY Dry fraction [-]

Computes the fraction of the exposure period during which the base film is expected to be dry.

Inputs:

- `base` — [Solvers.FourField.Base](#) object
- `drop` — [Solvers.FourField.Drop](#) object
- `zIdx` — Axial indices to evaluate (optional)

FTOT(*base, drop, zIdx*)

FTOT Total interfacial force on base film [N/m²]

Aggregates all force contributions acting on the base film, including wall shear, wave interaction, vapor shear, buoyancy, gravity, and droplet deposition based on [Inputs.Model.MOMENTBASE](#) model.

Inputs:

- `base` — [Solvers.FourField.Base](#) object
- `drop` — [Solvers.FourField.Drop](#) object
- `zIdx` — Axial indices to evaluate (optional)

FWAVE(*base, drop, zIdx*)

FWAVE Wave interfacial force [N/m²]

Computes the interfacial shear force exerted by the wave field on the base film based on [Inputs.Model.WAVEBASEINT](#) model.

Inputs:

- `base` — [Solvers.FourField.Base](#) object
- `drop` — [Solvers.FourField.Drop](#) object
- `zIdx` — Axial indices to evaluate (optional)

FWAVEMASS(*base, drop, zIdx*)

FWAVEMASS Base-wave mass exchange force [N/m²]

Calculates the momentum exchange force due to mass transfer between base and wave films, based on velocity difference and MWAVE flux.

Inputs:

- `base` — [Solvers.FourField.Base](#) object
- `drop` — [Solvers.FourField.Drop](#) object
- `zIdx` — Axial indices to evaluate (optional)

MDEP(*base, drop, zIdx*)

MDEP Base deposition mass flux [kg/m²/s]

Calculates deposition flux to the base film from drops, scaled by deposition fraction.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

MENT(base, zIdx)

MENT Base entrainment mass flux [kg/m²/s]

Placeholder for future implementation if needed. Currently returns zero entrainment.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

MEVAP(base, zIdx)

MEVAP Base evaporation mass flux [kg/m²/s]

Computes evaporation flux from the base film using heat flux fraction and parent film evaporation rate.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

MTOT(base, drop, zIdx)

MTOT Total mass flux of the base film [kg/m²/s]

Calculates the total mass flux exchange associated with the base film, including evaporation, entrainment, deposition, and wave interaction.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MTURB(base, zIdx)

MTURB Turbulent mass exchange [kg/m²/s]

Estimates turbulent mass exchange between base and wave films using wave mixing coefficient *Inputs.Model.WAVEMIXCOEF*.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

MWAVE(base, drop, zIdx)

MWAVE Base film mass flux interaction with wave [kg/m²/s]

Computes the mass flux exchange between the base film and wave field. Includes net exchange term based on relaxation toward equilibrium thickness and turbulent mixing contribution.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

Outputs:

- Mwave — Mass flux transferred to wave field [kg/m²/s]
- Mnet — Net exchange term before turbulent adjustment [kg/m²/s]

TBASE(base, zIdx)

TBASE Base film exposure time [s]

Calculates the time during which the base film is exposed to vapor between successive waves, based on wave spacing and relative velocity (eq 24 of Le Corre [2022]).

Inputs:

- base — [Solvers.FourField.Base](#) object
- zIdx — Axial indices to evaluate (optional)

TDRY(*base, drop, zIdx*)

TDRY Base film dry-out time [s]

Estimates the time of base film dry out under current evaporation and deposition conditions. Returns zero if dry-out does not occur.

Inputs:

- base — [Solvers.FourField.Base](#) object
- drop — [Solvers.FourField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

THICKMIN(*base, drop, zIdx*)

THICKMIN Minimum base film thickness [m]

Calculates the minimum thickness of the base film at the end of its exposure period, based on mass flow rate, liquid density, velocity, and channel perimeter (eq. 28 of Le Corre [2022]).

Inputs:

- base — [Solvers.FourField.Base](#) object
- drop — [Solvers.FourField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

WMIN(*base, drop, zIdx*)

WMIN Minimum base film mass flow rate [kg/s]

Calculates the minimum mass flow rate of the base film at the end of its exposure period (TBASE), accounting for evaporation and deposition.

Inputs:

- base — [Solvers.FourField.Base](#) object
- drop — [Solvers.FourField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

WMINL(*base, drop, zIdx*)

WMINL Minimum base film mass flow rate per perimeter [kg/m-s]

Computes the minimum base film mass flow rate normalized by channel perimeter.

Inputs:

- base — [Solvers.FourField.Base](#) object
- drop — [Solvers.FourField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

copyFlowProperties(*srcObj, targetObj, opts*)

COPYFLOWPROPERTIES Copy flow property arrays between Base objects

Copies selected flow properties (W, U, H) from a source object to one or more target Base objects. Supports full copy or partial copy (excluding first axial node) based on options.

Inputs:

- srcObj — Source object containing flow properties
- targetObj — Array of [Solvers.FourField.Base](#) objects to update
- opts.all — Logical flag; if true, copies all axial nodes, otherwise skips the first node (default: false)

class Solvers.FourField.Wave(*film*)

Bases: [Solvers.AbstractFilm](#)

WAVE Class for modeling liquid waves in four-field solver

This class represents the physical and numerical properties of the wave field in a four-field thermal-hydraulic solver. It handles flow variables, heat transfer, and phase interaction mechanisms between the waves and other fields (base film, drop, vapor).

The class supports initialization from an existing film object and provides methods for computing derived quantities such as mass flow fractions, interfacial fractions, and deposition/evaporation fluxes.

Constructor Summary

Wave(*film*)

WAVE Constructor for Wave class

Initializes the wave object using properties from an existing film object. Copies solver configuration and geometry data, and allocates flow property arrays for multiple walls.

Inputs:

- *film* — [Solvers.AbstractFilm](#) object representing the parent film

Property Summary

DT = 0

Time step size [s] from `Inputs.options.TSTEP`

FREQUENCY = 1

Wave frequency [Hz]

H = 1E6

Enthalpy [J/kg]

HFLUX = 1.

Film heat flux [W/m²]

ITR

Iteration tracking

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from `Inputs.Model.NNODES`

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

film

Parent *Solvers.AbstractFilm* object

Method Summary**AMPLITUDE**(*wave, zIdx*)

AMPLITUDE Wave amplitude [m]

Computes wave amplitude based on mass flow, shape factor, fluid density, and frequency. Limited by half the hydraulic diameter.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

BETA(*wave, zIdx*)

BETA Wave film interfacial fraction [-]

Calculates the fraction of the interface occupied by waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

BETAP(*wave, zIdx*)

BETAP Wave film heat flux fraction [-]

Determines the fraction of heat flux directed to the waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

DELTA_FREQ(*wave, zIdx*)

DELTA_FREQ Wave frequency source/sink [Hz/m]

Calculates the exchange term for wave frequency relaxation toward equilibrium frequency.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

DRAGCOEF(*wave, zIdx*)

DRAGCOEF Wave drag coefficient [-]

Calculates the drag coefficient for vapor-wave interaction based on Reynolds number and *Inputs.Model.WAVEDRAGCOEF* model coefficients.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EPSILON(*wave, zIdx*)

EPSILON Wave mass flow fraction [-]

Computes the fraction of total film mass flow carried by waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EQFREQUENCY(*wave*, *zIdx*)

EQFREQUENCY Equilibrium wave frequency [Hz]

Computes the equilibrium wave frequency based on Strouhal number and vapor velocity.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EQPERIOD(*wave*, *zIdx*)

EQPERIOD Equilibrium wave period [s]

Calculates the inverse of equilibrium wave frequency.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EQSTROUHAL(*wave*, *zIdx*)

EQSTROUHAL Equilibrium Strouhal number [-]

Calculates the equilibrium Strouhal number using selected *Inputs.Model.EQSTROUHAL* model.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

ETA(*wave*, *zIdx*)

ETA Wave film deposition fraction

Computes the fraction of deposition attributed to waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

FBASE(*wave*, *drop*, *zIdx*)

FBASE Base interfacial force [N/m²]

Computes the interfacial force between wave and base film.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *drop* — *Solvers.FourField.Drop* object
- *zIdx* — Axial indices to evaluate (optional)

FBASEMASS(*wave*, *drop*, *zIdx*)

FBASEMASS Wave mass exchange force [N/m²]

Calculates the force due to mass exchange between wave and base film.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *drop* — *Solvers.FourField.Drop* object
- *zIdx* — Axial indices to evaluate (optional)

FDEP(*wave*, *drop*, *zIdx*)

FDEP Droplet interaction force [N/m²]

Calculates the force due to deposition of droplets onto the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FDRAG(*wave*, *zIdx*)

FDRAG Vapor drag force on wave [N/m²]

Calculates the shear stress due to vapor drag acting on the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

FSHEAR(*wave*, *zIdx*)

FSHEAR Vapor shear stress [N/m²]

Computes the shear stress exerted by vapor on the wave field using friction factor.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

FTOT(*wave*, *drop*, *zIdx*)

FTOT Total interfacial force [N/m²]

Computes the total interfacial force acting on the wave field, including vapor shear, buoyancy, gravity, droplet interaction, and base film forces.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FVAPOR(*wave*, *zIdx*)

FVAPOR Wave vapor shear stress [N/m²]

Computes the total shear stress exerted by vapor on the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

FWALL(*wave*, *zIdx*)

FWALL Wave wall shear stress [N/m²]

Placeholder method. Wave does not implement wall shear stress.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

MBASE(*wave*, *drop*, *zIdx*)

MBASE Mass flux interaction with base film [kg/m²/s]

Computes the net mass exchange between wave and base film, including turbulent mixing.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MDEP(*wave, drop, zIdx*)

MDEP Wave deposition mass flux [kg/m²/s]

Computes the deposition mass flux from waves to droplets.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MENT(*wave, zIdx*)

MENT Wave entrainment mass flux [kg/m²/s]

Computes the mass flux associated with entrainment from the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

MEVAP(*wave, zIdx*)

MEVAP Wave evaporation mass flux [kg/m²/s]

Calculates the evaporation mass flux from the wave field based on heat flux fraction.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

MTOT(*wave, drop, zIdx*)

MTOT Total mass flux [kg/m²/s]

Calculates the total mass flux for the wave field, including evaporation, entrainment, deposition, and interactions with base film.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MTURB(*wave, zIdx*)

MTURB Turbulent mass exchange [kg/m²/s]

Returns the turbulent mixing mass flux between wave and base film.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

N(*wave, zIdx*)

N Wave number density [1/m]

Computes the number of waves per unit length (inverse of wave spacing).

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

PERIOD(*wave, zIdx*)

PERIOD Wave time period [s]

Calculates the inverse of wave frequency.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

REV(*wave*, *zIdx*)

REV Vapor Reynolds number (relative to waves) [-]

Computes the Reynolds number for vapor flow interacting with waves.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

SHAPEFACTOR(*wave*, *zIdx*)

SHAPEFACTOR Wave shape factor [-]

Computes the dimensionless shape factor for waves based on Reynolds number and *Inputs.Model.SHAPEFACTORCOEF* model.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

SPACING(*wave*, *zIdx*)

SPACING Wave spacing [m]

Computes the axial spacing between waves based on velocity and frequency.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

THICK(*wave*, *zIdx*)

THICK Wave thickness [m]

Computes the wave thickness as the product of amplitude and interfacial fraction. This parameter represents the thickness that the wave field would have if fully overlayed on top of the base film.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

WIDTH(*wave*, *zIdx*)

WIDTH Wave width [m]

Calculates wave width as a function of amplitude and shape factor, limited by wave spacing.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

WL(*wave*, *zIdx*)

WL Wave mass flow rate per unit perimeter [kg/s/m]

Calculates the mass flow carried by the wave field at specified axial locations.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

3.4.2 Mixture solver

The `Solvers.Mixture` module in **OpenSTREAM** provides classes and tools for simulating two-phase flows using a mixture approach. In this method, all phases are treated as a single continuum with averaged properties, making it suitable for cases where phase separation is minimal or not explicitly resolved. Additionally, the solver supports a Homogeneous Relaxation Model (HRM), which introduces thermal non-equilibrium between phases by solving separate mass and energy conservation equations for the vapor phase.

This module includes:

- Solver class for mixture-based flow simulations: `Solvers.Mixture.MixtureSolver`
- Mixture field class: `Solvers.Mixture.Mixture`
- Phase classes: `Solvers.Mixture.Liquid`, `Solvers.Mixture.Vapor`
- Mixture primary properties: `Solvers.Mixture.Mixture.W`, `Solvers.Mixture.Mixture.P`, `Solvers.Mixture.Mixture.H`
- Key solver class methods: `Solvers.Mixture.MixtureSolver.initializeSolver`, `Solvers.Mixture.MixtureSolver.solve()`, `Solvers.Mixture.MixtureSolver.plotz`
- Field class methods for calculating secondary mixture and phase parameters.

class `Solvers.Mixture.MixtureSolver.MixtureSolver(inputSet)`

Bases: `Solvers.AbstractSolver`

MIXTURESOLVER Solver for initializing, solving, and visualizing mixture flow.

The MixtureSolver class handles the setup, execution, and visualization of thermal-hydraulic simulations using a two-phase mixture approach or a Mixture Relaxation Model (MRM).

Responsibilities:

- Initializes solver parameters from an `Inputs.InputSet` object
- Interpolates boundary conditions in time and space
- Constructs `Solvers.Mixture.Mixture` objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Mixture construction and phase separation (liquid/vapor)
- Mixture mass/momentum/energy transport modeling
- Support for relaxation models and near-wall phenomena
- Visualization of results across time and axial domains

Constructor Summary

MixtureSolver(`inputSet`)

MIXTURESOLVER Constructor for the MixtureSolver class.

Creates a new instance of the MixtureSolver class using the provided `Inputs.InputSet`. This constructor initializes the solver by calling the abstract `Solvers.AbstractSolver` superclass constructor and then sets up all necessary solver parameters and internal data structures via `Solvers.Mixture.MixtureSolver.initializeSolver`

Input:

- `inputSet` — An *Inputs.InputSet* object containing model configuration, geometry, boundary conditions, and solver options.

Notes:

- This constructor assumes *Inputs.InputSet* is fully validated.
- Solver initialization includes time/space discretization, boundary condition interpolation, and mixture object setup.

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

DZ = 0

Axial step size [m]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIME = 0

Time series [s]

Z = 1.

Elevation [m]

boundaryConditions

Boundary conditions object *Inputs.BoundaryConditions*

fluid

Fluid object *Inputs.FluidProperties*

inputSet

Input set object *Inputs.InputSet*

mixture

Mixture object

mixtureInit

Null transient mixture object

Method Summary

axialInterpolate(*mix, x, y*)

AXIALINTERPOLATE Performs axial interpolation of data using a specified method.

Interpolates the values in *y* at positions defined by *mix.Z* using the interpolation method specified in *Inputs.options.AXIALINTERP*. The interpolation is performed over the domain defined by *x*.

If the interpolation returns NaN (e.g., due to points slightly outside the interpolation range when using methods like 'next'), linear extrapolation is applied to those specific points to ensure continuity.

This function ensures robust handling of out-of-bound values by using extrapolation and fallback strategies.

initializeSolver(*mixSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs mixture objects.

Sets up the solver's internal state using the provided [Inputs.InputSet](#). This includes discretization of time and space, interpolation of boundary conditions, and initialization of data structures for transient and steady-state simulations.

Operations performed:

- Computes time steps and axial node positions
- Interpolates boundary conditions in time and space
- Initializes mixture array for each time step
- Constructs data structures for:
 - Pressure drops (DP, DPSUM)
 - Momentum and energy derivatives (MDER)
 - Mixture Relaxation Model (MRM)
 - Near-wall transport (NEARWALL)
 - Iteration tracking (ITR)
- Initializes fluid property objects
- Sets up phase-specific objects (Liquid, Vapor)
- Assigns initial conditions for mass flow rate, pressure, enthalpy

Notes:

- The function assumes uniform axial discretization.
- Wall heat flux is computed from interpolated boundary conditions.
- MRM fields are initialized after enthalpy to ensure correct vapor quality (XEQ) computation.

interpBoundaryConditions(*mixSolver*)

INTERPBOUNDARYCONDITIONS Interpolates boundary conditions in time and space.

Expands user-defined [Inputs.BoundaryConditions](#) to match the solver's spatial grid and time steps. It performs interpolation of relevant boundary condition parameters both temporally and axially, and computes derived quantities such as wall heat flux.

Operations performed:

- Interpolates time-dependent boundary condition parameters: 'TIME', 'PRESSURE', 'HIN', 'MFLOW', 'POWER'
- Interpolates wall power distribution in space (axially)
- Interpolates wall power in time across all walls
- Computes wall heat flux at each axial node and time step

Notes:

- Uses [Solvers.Mixture.MixtureSolver.timeInterpolate](#) and [Solvers.Mixture.MixtureSolver.axialInterpolate](#) for interpolation.
- Throws an error if any expected boundary condition parameter is missing.
- The calculation of HFLUX is based on normalized wall power and geometric properties.

plott(*mixSolver*, *zIdx*, *opt*)

PLOTT Plots temporal distributions of mixture parameters at a given axial location.

Generates time-series plots of selected mixture parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- *mixSolver* — [Solvers.Mixture.MixtureSolver](#) object containing simulation data
- *zIdx* — Axial index (scalar, positive integer)
- *opt.display* — Parameters to display (e.g., 'HFLUX', 'W', 'DP', etc.)
- *opt.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opt.wall* — Wall index(es) to plot
- *opt.tIdx* — Time indices to include in the plot
- *opt.reverseTime* — Logical flag to reverse time axis
- *opt.unitTemp* — Temperature unit: 'K' or 'C'
- *opt.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'

- `opt.resize` — Resize factor for plot scaling
- `opt.nearWall` — Logical flag to include near-wall parameters

Notes:

- If `opt.display` is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Near-wall data is included if `opt.nearWall` is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.
- In the pressure drop plot, the temporal acceleration includes the slip-induced contribution (if any)

plotz(*mixSolver, tIdx, opts*)

PLOTZ Plots spatial (axial) distributions of mixture parameters.

Generates axial plots of selected mixture parameters at specified time indices. The function supports multiple display modes, wall selections, near-wall data, and optional animation over time.

Inputs:

- `mixSolver` — [Solvers.Mixture.MixtureSolver](#) object containing simulation data
- `tIdx` — Time index or indices (vector of positive integers)
- `opts.display` — Parameters to display (e.g., 'HFLUX', 'W', 'DP', etc.)
- `opts.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opts.wall` — Wall index(es) to plot
- `opts.zIdx` — Axial indices to include in the plot
- `opts.obstructions` — Logical flag to display obstruction locations
- `opts.unitTemp` — Temperature unit: 'K' or 'C'
- `opts.arrangement` — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- `opts.resize` — Resize factor for plot scaling
- `opts.nearWall` — Logical flag to include near-wall parameters

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if `opts.obstructions` is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.
- In the pressure drop plot, the temporal acceleration includes the slip-induced contribution (if any)

plotzt(*mixSolver, opt*)

PLOTZT Plots 2D time/elevation distributions of mixture parameters.

Generates surface plots of selected mixture-related parameters over time and axial (elevation) positions. It supports plotting for mixture, liquid and vapor fields, and can handle both real and null (initial) transient data.

Inputs:

- `mixSolver` — [Solvers.Mixture.MixtureSolver](#) object containing simulation data
- `opt.display` — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- `opt.label` — Corresponding labels for display parameters
- `opt.unit` — Units for each parameter
- `opt.field` — Field to plot: 'mixture', 'liquid', or 'vapor'
- `opt.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opt.wall` — Wall index(es) to plot
- `opt.zIdx` — Axial indices (must include at least 2)
- `opt.tIdx` — Time indices (must include at least 2)
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.shading` — Surface shading style: 'faceted', 'flat', or 'interp'
- `opt.view` — View angle for 3D plot [azimuth elevation]

Notes:

- If 'ALL' is passed to `opt.display`, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

timeInterpolate(*mix*, *y*)

TIMEINTERPOLATE Performs temporal interpolation of data using a specified method.

Interpolates the values in *y* at the time points defined by `mix.TIME` using the interpolation method specified in `Inputs.options.TIMEINTERP`.

If the boundary condition time vector (`mix.inputSet.bc.TIME`) is scalar, interpolation is skipped and the original data *y* is returned unchanged.

This function supports flexible interpolation strategies and handles cases where interpolation is unnecessary.

Solvers.Mixture.MixtureSolver.solve(*mixSolver*)

SOLVE Executes the mixture solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.Mixture.MixtureSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (`solveINIT = true`)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for flow, pressure, enthalpy, and vapor properties
- Supports thermal non-equilibrium modeling via vapor relaxation
- Logs progress and outputs to session directory

class Solvers.Mixture.Vapor(*mix*)

Bases: [Solvers.AbstractPhase](#)

VAPOR Represents the vapor phase in a mixture solver simulation.

The Vapor class provides access to vapor-specific properties and calculations derived from the mixture solution. It encapsulates methods for computing flow, thermodynamic, and transport properties of the vapor phase at each axial node.

Responsibilities:

- Compute vapor mass fraction and void fraction
- Calculate vapor velocity, enthalpy, and temperature
- Evaluate wall heat flux and wall evaporation contributions

- Support derived quantities such as Reynolds number and mass flux

Notes:

- All methods assume access to a valid `Solvers.Mixture.Mixture` object
- Axial indexing is optional; defaults to full axial domain
- Enthalpy calculations adapt to thermal non-equilibrium models
- Velocity and enthalpy calculations include fallback logic to handle single-phase liquid regions and numerical stability

Constructor Summary

Vapor(*mix*)

VAPOR Constructor of the Vapor class

Initializes the Vapor object from one or more instances of the `Solvers.Mixture.Mixture` class. Each Vapor instance stores a reference to its corresponding Mixture object and the number of axial nodes (NZ).

Input:

- *mix* — Array of `Solvers.Mixture.Mixture` objects representing the simulation state

Notes:

- Supports vectorized initialization for transient simulations
- Assumes each `Solvers.Mixture.Mixture` object is fully initialized

Method Summary

C(*vapor*, *zIdx*)

C Vapor mass fraction [-]

FW(*vapor*, *zIdx*)

FW Vapor wall friction factor [-]

Computes the Darcy wall friction factor based on `Inputs.Model.SPMTM` model.

Supported models

- BLASIUS: Blasius model ($f = C(1)Re^{C(2)} + C(3)$) using user-defined `Inputs.Model.FRICTION` coefficients

H(*vapor*, *zIdx*)

H Vapor enthalpy [J/kg]

Calculation depends on `Inputs.Model.THERMALNONEQ` model.

HFLUX(*vapor*, *zIdx*)

HFLUX Wall heat flux to vapor [W/m^2]

HFLUXWALEVAP(*vapor*, *zIdx*)

HFLUX Wall evaporation heat flux [W/m^2]

HWALL(*vapor*, *twall*, *zIdx*)

HWALLLIQ Single-phase vapor wall heat transfer coefficient [W/m^2/K]

Computes the wall heat transfer coefficient using vapor thermal conductivity and Nusselt number.

MFLUX(*vapor*, *zIdx*)

MFLUX Vapor mass flux [kg/m^2-s]

NU(*vapor, twall, zIdx*)

NU Vapor wall Nusselt number [-]

Computes the Nusselt number for wall heat transfer to vapor based on [Inputs.Model.SPHTM](#) model.

Q(*vapor, zIdx*)

Q Vapor volumetric flow rate [m³/s]

RE(*vapor, zIdx*)

RE Vapor Reynolds number [-]

T(*vapor, zIdx*)

T Vapor temperature [K]

TAUW(*vapor, zIdx*)

TAUW Vapor wall shear stress [N/m²]

TIME(*vapor*)

TIME Time series [s]

U(*vapor, zIdx*)

U Vapor velocity [m/s]

VF(*vapor, zIdx*)

VF Void fraction [-]

W(*vapor, zIdx*)

W Vapor mass flow rate [kg/s]

X(*vapor, zIdx*)

X Vapor mass flow fraction [-]

Z(*vapor, zIdx*)

Z Axial nodes [m]

class `Solvers.Mixture.Mixture`(*inputSet, fluid*)

Bases: [Solvers.AbstractField](#)

MIXTURE Class for modeling two-phase mixture field in mixture solver

This class encapsulates the physical and numerical properties of a fluid mixture, including flow variables, phase interactions, pressure losses, heat transfer, and relaxation models. It supports multiple solver models and provides methods for computing derived quantities and handling transitions such as boiling and annular flow.

Constructor Summary

Mixture(*inputSet, fluid*)

MIXTURE Constructor for Mixture class

Initializes the mixture object with input configuration and fluid properties. Sets up flow property tracking for simulation.

Inputs:

- *inputSet* — [Inputs.InputSet](#) object containing model, geometry, and boundary conditions
- *fluid* — [Inputs.FluidProperties](#) object containing thermophysical fluid data

Property Summary

DP
Saved detailed pressure drops [Pa]

DPSUM
Saved detailed cumulative pressure drops [Pa]

DT = 0
Time step size [s] from `Inputs.options.TSTEP`

DZ = 0
Axial step size [m]

H = 1E6
Enthalpy [J/kg]

HFLUX = 1.
Wall heat flux [W/m^2]

ITR
Iteration tracking

MDER
Saved detailed material derivative terms

MRM
Mixture relaxation terms

NEARWALL
Near-wall terms

NTIME = 0
Number of time steps [-]

NZ = 0
Number of axial steps [-] from `Inputs.Model.NNODES`

P = 7E6
Pressure [Pa]

TIDX = 1
Time step index [-]

TIME = 0
Time series [s]

W = 1.
Mass flow rate [kg/s]

Z = 1.
Elevation [m]

cbt = false
Critical Boiling Transition flag [-]

fluid
`Inputs.FluidProperties` object

inputSet*Inputs.InputSet* object**liquid**

Liquid phase object

mfbt = false

Minimum Film Boiling Transition flag [-]

vapor

Vapor phase object

Method Summary**AFDISTR**(*mix, param1, param2, zIdx*)

AFDISTR Computes annular flow distribution over axial positions.

Evaluates a weighted distribution between param1 and param2 using the annular flow fraction defined in *Solvers.Mixture.Mixture.AFFNC*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing axial grid and flow fractions
- *param1* — First flow property
- *param2* — Second flow property
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- The annular flow fraction is extracted from *Solvers.Mixture.Mixture.AFFNC* at *zIdx*
- The output is a linear interpolation: $(1 - \text{affnc}) * \text{param1} + \text{affnc} * \text{param2}$
- *mix(1).NZ* is used for default indexing, assuming *mix* may be an array

AFFNC(*mix, zIdx*)

AFFNC Computes annular flow fraction over axial positions.

Evaluates the annular flow fraction using a sigmoid function parameterized by the transition model and geometry. The sigmoid is evaluated via the cached *sigm* method.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing model and geometry data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Sigmoid parameters are scaled by node density (NNODES / LENGTH)
- The center of the transition is offset by *mix.OAFIDX()*
- Uses *mix.sigm(...)* to evaluate the sigmoid function

CBT(*mix, zIdx*)

CBT Critical Boiling Transition flag [-], wall dependent

Retrieves precomputed CBT flags by *Solvers.Mixture.Mixture.CBT_CALC*, updated when *mix.H* is set.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to retrieve (optional)

CBTIDX(*mix*)

CBTIDX Index of first CBT occurrence [-]

Compute the index of first CBT occurrence. If no occurrence is found, NaN is returned.

Inputs

- `mix` — `Solvers.Mixture.Mixture` object

CBTZ(*mix*, *zIdx*)

CBTZ Length from upstream CBT [m]

Compute the distance from the upstream critical boiling transition. Support multiple transitions.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to evaluate (optional)

CHF(*mix*, *zIdx*)

CHF Critical Heat Flux [W/m²], wall dependent

Retrieves precomputed CHF values by `Solvers.Mixture.Mixture.CBT_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

DPACCS(*mix*, *MCTold*, *zIdx*)

DPACCS Slip-induced temporal acceleration pressure drop [Pa]

Computes the pressure drop due to the slip-induced temporal acceleration between time steps.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `MCTold` — Previous slip-induced momentum correction [kg/s]
- `zIdx` — Axial indices to evaluate (optional)

DPACCT(*mix*, *UPold*, *zIdx*)

DPACCT Temporal acceleration pressure drop [Pa]

Computes the pressure drop due to temporal acceleration between time steps.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `UPold` — Previous axial (advection) velocity [m/s]
- `zIdx` — Axial indices to evaluate (optional)

DPACCZ(*mix*, *zIdx*)

DPACCZ Spatial acceleration pressure drop [Pa]

Computes the pressure drop due to spatial acceleration of the flow.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to evaluate (optional)

DPGRAV(*mix*, *zIdx*)

DPGRAV Gravitational pressure loss [Pa]

Computes the pressure loss due to gravity along the axial direction.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to evaluate (optional)

DPK(*mix*, *zIdx*)

DPK Local pressure loss [Pa]

Computes the pressure loss due to localized effects (e.g., flow mixers).

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

DPPARTS(*mix*, *UPold*, *MCTold*, *zIdx*)

DPPARTS All pressure loss components [Pa]

Returns a structure containing all individual pressure loss components.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *UPold* — Previous axial (advection) velocity [m/s]
- *zIdx* — Axial indices to evaluate (optional)

Returns:

- *dpparts* — Struct with fields:
 - GRAV : Gravitational loss [Pa]
 - WALL : Wall friction loss [Pa]
 - ACCZ : Spatial acceleration loss [Pa]
 - ACCT : Temporal acceleration loss [Pa]
 - ACCS : Slip-induced temporal acceleration loss [Pa]
 - K : Local loss [Pa]
 - TOT : Total pressure loss [Pa]

DPTOT(*mix*, *UPold*, *MCTold*, *zIdx*)

DPTOT Total pressure loss [Pa]

Computes the total pressure loss by summing all contributing components.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *UPold* — Previous axial (advection) velocity [m/s]
- *MCTold* — Previous slip-induced momentum correction [kg/s]
- *zIdx* — Axial indices to evaluate (optional)

DPWALL(*mix*, *zIdx*)

DPWALL Wall friction pressure drop [Pa]

Computes the pressure drop due to wall friction.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

ECT(*mix*, *zIdx*)

ECT Slip-induced energy correction term [J/m]

Computes the slip-induced energy correction term per unit length.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

FOURIERCOND(*mix*, *zIdx*)

FOURIERCOND Calculates the non-dimensional relaxation time for wall-dependent interfacial condensation.

Computes the non-dimensional relaxation time (akin to a Fourier number) associated with interfacial condensation at the specified axial indices, based on the thermal relaxation model defined in [Inputs.Model.THERMALRELAX](#).

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object containing fluid and model data
- `zIdx` — Axial indices to evaluate (optional; defaults to full axial domain)

Supported Models:

- FOURIERX: Interpolate Fourier number from pre-defined quality-time pairs
- TIMEX: Interpolate relaxation time from pre-defined quality-time pairs
- VOID: Computes Fourier number based on void fraction and fluid properties
- HOMOGENEOUS: Computes Fourier number based on homogeneous approach
- FOURIER: Empirical Fourier number correlation

FOURIEREVAP(*mix*, *zIdx*)

FOURIEREVAP Calculates the non-dimensional relaxation time for wall-dependent interfacial evaporation.

Computes the non-dimensional relaxation time (akin to a Fourier number) associated with interfacial evaporation at the specified axial indices, based on the thermal relaxation model defined in [Inputs.Model.THERMALRELAX](#).

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object containing fluid and model data
- `zIdx` — Axial indices to evaluate (optional; defaults to full axial domain)

Supported Models:

- FOURIERX: Interpolate Fourier number from pre-defined quality-time pairs
- TIMEX: Interpolate relaxation time from pre-defined quality-time pairs
- VOID: Computes relaxation time based on void fraction and fluid properties
- HOMOGENEOUS: Computes Fourier number based on homogeneous approach
- FOURIER: Empirical Fourier number correlation

FW(*mix*, *zIdx*)

FW Darcy wall friction factor [-]

Computes the Darcy wall friction factor based on [Inputs.Model.SPMTM](#) model.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object
- `zIdx` — Axial indices to evaluate (optional)

Supported models

- BLASIUS: Blasius model ($f = C(1)Re^{C(2)} + C(3)$) using user-defined [Inputs.Model.FRICTION](#) coefficients

FWL(*mix*, *zIdx*)

FWL Liquid-equivalent Darcy wall friction factor [-]

Computes the Darcy friction factor, using liquid-equivalent Reynolds number, based on [Inputs.Model.SPMTM](#) model.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object
- `zIdx` — Axial indices to evaluate (optional)

Supported models

- BLASIUS: Blasius model ($f = C(1)Re^{C(2)}$) using user-defined [Inputs.Model.FRICTION](#) coefficients

HINT(*mix*, *zIdx*)

HINT Linear interfacial heat transfer rate [W/m]

Computes the interfacial heat transfer rates due to condensation and evaporation, based on interfacial mass transfer and enthalpy differences.

Inputs:

- *mix* — `Solvers.Mixture.Mixture` object with flow and enthalpy data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Uses `Solvers.Mixture.Mixture.MINT` for mass transfer and `Solvers.Mixture.Mixture.MRM.HV` for vapor enthalpy
- Behavior depends on `Inputs.Model.INTTRANSH`:
 - ‘*BULK*’: Consideration of bulk enthalpy
 - ‘*SATURATED*’: Consideration of saturated enthalpy

HINTCOND(*mix*, *zIdx*)

HINTCOND Linear interfacial heat condensation rate [W/m]

Extracts the condensation component from `Solvers.Mixture.Mixture.HINT`.

Inputs:

- *mix* — `Solvers.Mixture.Mixture` object
- *zIdx* — Axial indices to evaluate (optional)

HINTEVAP(*mix*, *zIdx*)

HINTEVAP Linear interfacial heat evaporation rate [W/m]

Extracts the evaporation component from `Solvers.Mixture.Mixture.HINT`.

Inputs:

- *mix* — `Solvers.Mixture.Mixture` object
- *zIdx* — Axial indices to evaluate (optional)

HM(*mix*, *zIdx*)

H Static enthalpy [J/kg]

Computes the mass-centered mean mixture enthalpy.

Inputs:

- *mix* — `Solvers.Mixture.Mixture` object
- *zIdx* — Axial indices to evaluate (optional)

HTOT(*mix*, *zIdx*)

HTOT Total linear vapor heat rate [W/m]

Computes the total vapor heat transfer rate by summing interfacial and wall contributions.

Inputs:

- *mix* — `Solvers.Mixture.Mixture` object
- *zIdx* — Axial indices to evaluate (optional)

HVTOT(*mix*, *zIdx*)

HVTOT Total linear vapor specific enthalpy transfer [J/kg/m]

Computes the specific enthalpy input to the vapor per unit mass flow. Uses a lumped wall approach for now.

Inputs:

- *mix* — `Solvers.Mixture.Mixture` object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Uses lumped wall approach: sums WV and Htot across walls
- Set to zero when WV is negligible

HWALEVAP(*mix, zIdx*)

HWALEVAP Linear wall heat evaporation (boiling) rate [W/m]

Computes the heat transfer rate associated with wall boiling.

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object with boiling and enthalpy data
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Behavior depends on [Inputs.Model.INTTRANSH](#):
 - BULK: Consideration of bulk enthalpy
 - SATURATED: Consideration of saturated enthalpy

HWALHEAT(*mix, zIdx*)

HWALHEAT Linear wall heat to vapor rate [W/m]

Returns the wall heat generation rate (LHGR) at boiling transition nodes.

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Applies only at nodes flagged by [Solvers.Mixture.Mixture.CBT](#) or [Solvers.Mixture.Mixture.MFBT](#).

HWALL(*mix, twall, zIdx*)

HWALL Wall heat transfer coefficient [W/m²/K]

Computes the effective wall heat transfer coefficient by combining single-phase, boiling and post-CBT models. Supports multiple walls

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object
- *zIdx* — Axial indices to evaluate (optional)

HWALLBOIL(*mix, zIdx*)

HWALLBOIL Boiling wall heat transfer coefficient [W/m²/K]

Computes the boiling wall heat transfer coefficient using selected [Inputs.Model.TPHTM](#) model.

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- THOM: Thom's correlation

HWALLPOSTCBT(*mix, twall, zIdx*)

HWALLPOSTCBT Post-CBT wall heat transfer coefficient [W/m²/K]

Computes the effective wall heat transfer coefficient under post-CBT conditions. based on [Inputs.Model.BTHTM](#) model.

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object
- *twall* — Wall temperature
- *zIdx* — Axial indices to evaluate (optional)

Supported wall heat transfer models

- VAPOR: Heat transfer to vapor phase
- DOUGALL: Dougall-Rohsenow model (Dougall and Rohsenow [1963])
- BISHOP: Bishop-Sandberg-Tong model (Bishop *et al.* [1965])
- MOECK: Groeneveld-Moeck model (Groeneveld and Moeck [1969])
- DELORME: Groeneveld-Delorme model (Groeneveld and Delorme [1976])
- CONDIEIV: Condie-Bengston IV model (Morris *et al.* [1982])

JG(*mix, zIdx*)

JG Superficial vapor velocity [m/s]

Computes the superficial velocity of the vapor phase.

Inputs:

- mix — *Solvers.Mixture.Mixture* object
- zIdx — Axial indices to evaluate (optional)

JL(*mix, zIdx*)

JL Superficial liquid velocity [m/s]

Computes the superficial velocity of the liquid phase.

Inputs:

- mix — *Solvers.Mixture.Mixture* object
- zIdx — Axial indices to evaluate (optional)

KDIST(*mix, zIdx*)

KDIST Distance from upstream obstruction [m]

Computes the axial distance from the nearest upstream obstruction or inlet, based on *Inputs.Model.KLOC*.

Inputs:

- mix — *Solvers.Mixture.Mixture* object with axial grid and obstruction location
- zIdx — Axial indices to evaluate (optional)

KIDX(*mix, zIdx*)

KIDX Obstruction index

Returns the axial node index of the obstruction closest to *Inputs.Model.KLOC*.

Inputs:

- mix — *Solvers.Mixture.Mixture* object with axial grid
- zIdx — Axial indices to evaluate (optional)

KLOCZ(*mix, zIdx*)

KLOCZ Obstruction positions [m]

Returns the axial position of the obstruction closest to *Inputs.Model.KLOC*.

Inputs:

- mix — *Solvers.Mixture.Mixture* object with axial grid
- zIdx — Axial indices to evaluate (optional)

KLOSS(*mix, zIdx*)

KLOSS Local pressure loss coefficient [-]

Returns the pressure loss coefficient at the elevation closest to *Inputs.Model.KLOC*.

Inputs:

- mix — *Solvers.Mixture.Mixture* object

- `zIdx` — Axial indices to evaluate (optional)

KTRELAX(*mix*, *zIdx*)

KTRELAX Local relaxation time [s]

Returns the relaxation time at the elevation closest to `Inputs.Model.KLOC`. All other axial positions are set to NaN.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object with axial grid and model parameters
- `zIdx` — Axial indices to evaluate (optional; defaults to full axial range)

LHGR(*mix*, *zIdx*)

LHGR Linear heat generation rate [W/m]

Computes the linear heat generation rate by multiplying wall perimeter with local heat flux.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object with geometry and heat flux data
- `zIdx` — Axial indices to evaluate (optional)

MCT(*mix*, *zIdx*)

MCT Slip-induced momentum correction term [kg/s]

Computes the slip-induced momentum correction term per unit length.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to evaluate (optional)

MFBT(*mix*, *zIdx*)

MFBT Minimum Film Boiling Transition flag [-], wall dependent

Retrieves precomputed MFBT flags by `Solvers.Mixture.Mixture.MFBT_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

MFLUX(*mix*, *zIdx*)

MFLUX Mass flux [kg/m²/s]

Retrieves precomputed mass flux values by `Solvers.Mixture.Mixture.MFLUX_CALC`, updated when `mix.W` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

MINT(*mix*, *zIdx*)

MINT Linear interfacial mass transfer rates [kg/s/m]

Computes condensation and evaporation mass transfer rates based on deviation from equilibrium vapor flow and time relaxation.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object with flow and relaxation data
- `zIdx` — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Uses `mix.MRM.WV` for equilibrium deviation

- U_{Veq} is the approximated vapor velocity

MINTCOND(*mix*, *zIdx*)

MINTCOND Linear interfacial condensation rate [kg/s/m]

Extracts the condensation component from *Solvers.Mixture.Mixture.MINT*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

MINTEVAP(*mix*, *zIdx*)

MINTEVAP Linear interfacial evaporation rate [kg/s/m]

Extracts the evaporation component from *Solvers.Mixture.Mixture.MINT*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

MTOT(*mix*, *zIdx*)

MTOT Total linear vapor mass transfer rate [kg/s/m]

Computes the total vapor mass transfer rate by summing interfacial condensation, evaporation, and wall boiling contributions.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

MTRANSV(*mix*, *zIdx*)

MTRANSV Linear transversal mass exchange [kg/s/m]

Models transverse vapor mass exchange between wall regions using a time relaxation approach. Currently not used.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Uses a fixed relaxation time for now (RELAXT = 0.01 s)

MU(*mix*, *zIdx*)

MU Dynamic viscosity [Pa·s]

Computes the mixture dynamic viscosity using vapor and liquid phase contributions weighted by vapor quality.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with fluid properties
- *zIdx* — Axial indices to evaluate (optional)

MWALEVAP(*mix*, *zIdx*)

MWALEVAP Linear wall mass evaporation rate [kg/s/m]

Computes the wall boiling mass transfer rate using latent heat and wall heat generation rate.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with heat transfer and boiling data
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Latent heat is computed based on *Inputs.Model.INTTRANSH* setting

OAFIDX(*mix*)

OAFIDX Onset of annular flow node.

Returns the axial node index corresponding to the onset of annular flow, determined by comparing the equilibrium quality to the onset threshold. The result is cached in *mix.oafidx_const* to avoid recomputation.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing flow quality and onset threshold

Notes:

- If no node satisfies the condition, the most upstream node (1) is returned

OAFWL(*mix*)

OAFWL Liquid mass flow rate at onset of annular flow.

Returns the liquid mass flow rate at the axial index corresponding to the onset of annular flow, as defined by *Solvers.Mixture.Mixture.OAFIDX*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing liquid flow data

OAFX(*mix*, *zIdx*)

OAFX Onset of annular flow equilibrium quality.

Computes the equilibrium quality at the onset of annular flow using a model selected in *Inputs.Model.OAF*. The quality is evaluated over the specified axial indices.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing model, geometry, and fluid data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Supported Models:

- WALLIS: Full Wallis model
- WALLIS_SIMP: Simplified Wallis model
- LEVITAN: Levitan model

OAFZ(*mix*)

OAFZ Onset of annular flow elevation.

Returns the axial elevation (in meters) corresponding to the onset of annular flow, as defined by *Solvers.Mixture.Mixture.OAFIDX*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing axial grid data

PHI2F(*mix*, *zIdx*)

PHI2F Two-phase wall friction multiplier [-]

Computes the two-phase wall friction multiplier using selected *Inputs.Model.TPFM* model.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- HOMOGENEOUS: Homogeneous model
- SLIP: Constant velocity slip ratio
- EPRI: EPRI multiplier

PHI2K(*mix*, *zIdx*)

PHI2K Two-phase local pressure drop multiplier [-]

Computes the multiplier for local pressure drop in two-phase flow using selected *Inputs.Model.TPKM* model.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- HOMOGENEOUS: Homogeneous model
- SLIP: Constant velocity slip ratio
- ROMIE: Romie multiplier

RE(*mix*, *zIdx*)

RE Reynolds number [-]

Computes the Reynolds number based on total mass flow rate and mixture viscosity.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

REL(*mix*, *zIdx*)

REL Liquid-equivalent Reynolds number [-]

Computes the Reynolds number using liquid viscosity and total mass flow.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

RELAXT(*mix*, *zIdx*)

RELAXT Interfacial time relaxation [s]

Computes interfacial time relaxation based on sign of deviation from equilibrium vapor flow.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with flow and relaxation data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Uses *mix.MRM.WV* for equilibrium deviation
- Vapor mass deviation from equilibrium, *Wint*, must be consistent with MINT method

RELAXTCOND(*mix*, *zIdx*)

RELAXTCOND Time relaxation for interfacial condensation [s]

Retrieves precomputed time relaxation values for condensation by *Solvers.Mixture.Mixture.RELAXTCOND_CALC*, updated when *mix.H* is set.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to retrieve (optional)

RELAXTEVAP(*mix*, *zIdx*)

RELAXTEVAP Time relaxation for interfacial evaporation [s]

Retrieves precomputed time relaxation values for evaporation by *Solvers.Mixture.Mixture.RELAXTEVAP_CALC*, updated when *mix.H* is set.

Inputs:

- mix — [Solvers.Mixture.Mixture](#) object
- zIdx — Axial indices to retrieve (optional)

RHO(mix, zIdx)

RHO Density [kg/m³]

Retrieves precomputed mixture density values by `Solvers.Mixture.Mixture.RHO_CALC`, updated when `mix.H` is set.

Inputs:

- mix — [Solvers.Mixture.Mixture](#) object
- zIdx — Axial indices to retrieve (optional)

T(mix, zIdx)

T Temperature [K]

Returns the fluid temperature corresponding to the local enthalpy.

Inputs:

- mix — [Solvers.Mixture.Mixture](#) object with enthalpy data
- zIdx — Axial indices to evaluate (optional)

TAUW(mix, zIdx)

TAUW Wall shear stress [N/m²], wall dependent

Computes the wall shear stress for each wall segment using the friction factor, mass flux and two-phase multiplier. Supports multiple walls and adjusts for boiling transition conditions using [Inputs.Model.BTMTM](#) model.

Inputs:

- mix — [Solvers.Mixture.Mixture](#) object containing flow and geometry data
- zIdx — Axial indices to evaluate (optional)

Supported boiling transition models

- TPFM: Two-phase friction multiplier
- VAPOR: Shear stress to vapor phase

TWALL(mix, zIdx)

TWALL Wall temperature [K]

Computes the wall temperature based on heat flux and wall heat transfer coefficient. Uses liquid bulk temperature pre-CBT and vapor bulk temperature post-CBT.

Inputs:

- mix — [Solvers.Mixture.Mixture](#) object
- zIdx — Axial indices to evaluate (optional)

U(mix, zIdx)

U Static velocity [m/s]

Computes the mass-centered mean mixture velocity.

Inputs:

- mix — [Solvers.Mixture.Mixture](#) object
- zIdx — Axial indices to evaluate (optional)

UP(mix, zIdx)

UP Advection velocity [m/s]

Retrieves precomputed mixture (advection) velocity values by `Solvers.Mixture.Mixture.UP_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

VF(*mix*, *zIdx*)

VF Void fraction [-]

Retrieves precomputed void fraction values by `Solvers.Mixture.Mixture.VF_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

WALEVAPRATIO(*mix*, *zIdx*)

WALEVAPRATIO Wall mass evaporation ratio [-]

Computes the fraction of liquid mass undergoing wall boiling based on equilibrium quality and boiling onset thresholds `Inputs.Model.WBOILINGXSUB` and `Inputs.Model.WBOILINGXSAT`.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object with boiling model parameters
- `zIdx` — Axial indices to evaluate (optional)

Notes:

- Ratio is clamped between 0 and 1
- CBT and MFBT flags suppress boiling at transition nodes

WWALL(*mix*, *zIdx*)

WWALL Mixture mass flow distribution per wall [kg/s]

Computes the wall-distributed mass flow rate by scaling the total flow with the wall perimeter ratio.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object with geometry and flow data
- `zIdx` — Axial indices to evaluate (optional; defaults to full axial range)

X(*mix*, *zIdx*)

X Vapor quality [-]

Retrieves precomputed vapor quality values by `Solvers.Mixture.Mixture.X_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

XEQ(*mix*, *zIdx*)

XEQ Equilibrium quality [-]

Retrieves precomputed equilibrium quality values by `Solvers.Mixture.Mixture.XEQ_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

class `Solvers.Mixture.Liquid`(*mix*)

Bases: `Solvers.AbstractPhase`

LIQUID Represents the liquid phase in a mixture solver simulation.

The Liquid class provides access to liquid-specific properties and calculations derived from the mixture solution. It encapsulates methods for computing flow, thermodynamic, and transport properties of the liquid phase at each axial node.

Responsibilities:

- Compute liquid mass fraction and volumetric fraction
- Calculate liquid velocity, enthalpy, and temperature
- Evaluate wall heat flux contributions to the liquid phase
- Support derived quantities such as Reynolds number and mass flux

Notes:

- All methods assume access to a valid `Solvers.Mixture.Mixture` object
- Axial indexing is optional; defaults to full axial domain
- Velocity and enthalpy calculations include fallback logic to handle single-phase vapor regions and numerical stability

Constructor Summary

Liquid(*mix*)

LIQUID Constructor

Initializes the Liquid object from one or more instances of the `Solvers.Mixture.Mixture` class. Each Liquid instance stores a reference to its corresponding Mixture object and the number of axial nodes (NZ).

Input:

- *mix* — Array of `Solvers.Mixture.Mixture` objects representing the simulation state

Notes:

- Supports vectorized initialization for transient simulations
- Assumes each `Solvers.Mixture.Mixture` object is fully initialized

Method Summary

C(*liquid*, *zIdx*)

C Liquid mass fraction [-]

H(*liquid*, *zIdx*)

H Liquid enthalpy [J/kg]

Calculated based on mixture and vapor enthalpies, and vapor quality.

HFLUX(*liquid*, *zIdx*)

HFLUX Wall heat flux to liquid phase [W/m²]

HWALL(*liquid*, *twall*, *zIdx*)

HWALLLIQ Single-phase liquid wall heat transfer coefficient [W/m²/K]

Computes the wall heat transfer coefficient using liquid thermal conductivity and Nusselt number.

MFLUX(*liquid*, *zIdx*)

MFLUX Liquid mass flux [kg/m²/s]

NU(*liquid*, *twall*, *zIdx*)

NU Liquid wall Nusselt number [-]

Computes the Nusselt number for wall heat transfer to liquid based on `Inputs.Model.SPHTM` model.

Q (<i>liquid</i> , <i>zIdx</i>)	Q Liquid volumetric flow rate [m ³ /s]
RE (<i>liquid</i> , <i>zIdx</i>)	RE Liquid Reynolds number [-]
T (<i>liquid</i> , <i>zIdx</i>)	T Liquid temperature [K]
TIME (<i>liquid</i>)	TIME Time series [s]
U (<i>liquid</i> , <i>zIdx</i>)	U Liquid velocity [m/s]
VF (<i>liquid</i> , <i>zIdx</i>)	VF Liquid volumetric fraction [-]
W (<i>liquid</i> , <i>zIdx</i>)	W Liquid mass flow rate [kg/s]
X (<i>liquid</i> , <i>zIdx</i>)	X Liquid mass flow fraction [-]
Z (<i>liquid</i> , <i>zIdx</i>)	Z Axial node positions [m]

3.4.3 Three-field solver

The *Solvers.ThreeField* module in **OpenSTREAM** provides classes and tools for simulating annular two-phase flows using a three-field approach. In this method, the two liquid fields (film and droplets) are treated separately, each governed by its own set of conservation equations for mass, momentum, and energy. The vapor phase is solved using the *Solvers.Mixture* module. This framework enables explicit modeling of liquid–liquid interactions, slip velocities, and hydrodynamic non-equilibrium effects, making it well-suited for applications involving annular boiling two-phase flow regimes where field separation and interfacial dynamics are significant.

This module includes:

- Solver class for three-field-based flow simulations: *Solvers.ThreeField.ThreeFieldSolver*
 - Film and drop field classes: *Solvers.ThreeField.Film*, *Solvers.ThreeField.Drop*
 - Three-field primary properties: *Solvers.ThreeField.Film.W*, *Solvers.ThreeField.Film.U*, *Solvers.ThreeField.Film.H*, *Solvers.ThreeField.Drop.W*, *Solvers.ThreeField.Drop.U*, *Solvers.ThreeField.Drop.H*
 - Key solver class methods: *Solvers.ThreeField.ThreeFieldSolver.initializeSolver*, *Solvers.ThreeField.ThreeFieldSolver.solve()*, *Solvers.ThreeField.ThreeFieldSolver.plotz*
 - Field class methods for calculating secondary three-field parameters.
-

class `Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver`(*inputSet*, *mixSolver*)

Bases: `Solvers.AbstractSolver`

THREEFIELDSOLVER Solver for initializing, solving, and visualizing three-field flow.

The `ThreeFieldSolver` class handles the setup, execution, and visualization of thermal-hydraulic simulations using a two-phase three-field approach.

Responsibilities:

- Initializes solver parameters from an `Inputs.InputSet` object
- Constructs `Solvers.ThreeField.Film` and `Solvers.ThreeField.Drop` objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Liquid field construction (film/drop)
- Field mass/momentum/energy transport modeling
- Pressure drop gradient obtained from mixture solver
- Vapor solution obtained from mixture solver
- Visualization of results across time and axial domains

Constructor Summary

ThreeFieldSolver(*inputSet*, *mixSolver*)

THREEFIELDSOLVER Constructor for the `ThreeFieldSolver` class.

Creates a new instance of the `ThreeFieldSolver` class using the provided `Inputs.InputSet`. This constructor initializes the solver by calling the abstract `Solvers.AbstractSolver` superclass constructor and then sets up all necessary solver parameters and internal data structures via `Solvers.ThreeField.ThreeFieldSolver.initializeSolver`

Input:

- *inputSet* — An `Inputs.InputSet` object containing model configuration, geometry, boundary conditions, and solver options.

Notes:

- This constructor assumes `Inputs.InputSet` is fully validated.
- Solver initialization includes liquid and vapor objects setup.
- Time/space discretization and boundary condition interpolation initialized by `Solvers.Mixture.MixtureSolver` are used

Property Summary

DT = 0

Time step size [s] from `Inputs.Options.TSTEP`

DZ = 0

Axial step size [m]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from `Inputs.Model.NNODES`

TIME = 0

Time series [s]

Z = 1.

Elevation [m]

boundaryConditions

Boundary conditions object *Inputs.BoundaryConditions*

drop

Drop object

dropInit

Null transient drop object

film

Film object

filmInit

Null transient film object

fluid

Fluid object *Inputs.FluidProperties*

inputSet

Input set object *Inputs.InputSet*

mixSolver

Mixture object *Solvers.Mixture.Mixture*

Method Summary

EQUIL (*tfSolver, flm, drp, mix, zIdx*)

EQUIL Find entrained ratio at film/drop equilibrium state [-]

Iteratively computes the entrained liquid ratio at the onset of annular flow by balancing droplet deposition and film entrainment rates.

Inputs:

- *tfSolver* — *Solvers.ThreeField.ThreeFieldSolver* object
- *flm* — Film field object (*Solvers.ThreeField.Film*)
- *drp* — Droplet field object (*Solvers.ThreeField.Drop*)
- *mix* — Mixture object (*Solvers.Mixture.Mixture*)
- *zIdx* — Axial index for evaluation

Algorithm:

- Starts with an initial guess for droplet mass flow (50% of liquid mass)
- Updates droplet and film flow rates iteratively
- Computes difference between deposition and entrainment rates
- Stops when error < 1e-4 kg/s/m or after 100 iterations

Notes:

- Enhanced drop deposition is disabled
- Uses interpolation for improved convergence
- Falls back to ad-hoc update if interpolation fails
- Method update film and drop mass flow rates. Use *flm.copy()* and *drp.copy()* as input arguments if used in post-process

initializeSolver(*tfSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs film/drop objects.

Sets up the solver's internal state using the provided *Inputs.InputSet* and *Solvers.ThreeField.ThreeFieldSolver*. This includes initialization of data structures for transient and steady-state simulations based on the mixture solver solutions.

Operations performed:

- Initializes film and drop arrays for each time step
- Initializes fluid property objects
- Sets up mixture object
- Assigns initial conditions for phase mass flow rates, velocities, enthalpies

Notes:

- The function assumes uniform axial discretization.
- Wall heat flux is computed from interpolated boundary conditions.

plott(*tfSolver*, *zIdx*, *opt*)

PLOTT Plots temporal distributions of three-field parameters at a given axial location.

Generates time-series plots of selected three-field parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- *tfSolver* — *Solvers.ThreeField.ThreeFieldSolver* object containing simulation data
- *zIdx* — Axial index (scalar, positive integer)
- *opt.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opt.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opt.wall* — Wall index(es) to plot
- *opt.tIdx* — Time indices to include in the plot
- *opt.reverseTime* — Logical flag to reverse time axis
- *opt.unitTemp* — Temperature unit: 'K' or 'C'
- *opt.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- *opt.resize* — Resize factor for plot scaling

Notes:

- If *opt.display* is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotz(*tfSolver*, *tIdx*, *opts*)

PLOTZ Plots spatial (axial) distributions of three-field parameters.

Generates axial plots of selected three-field parameters at specified time indices. The function supports multiple display modes, wall selections, and optional animation over time.

Inputs:

- *tfSolver* — *Solvers.ThreeField.ThreeFieldSolver* object containing simulation data
- *tIdx* — Time index or indices (vector of positive integers)
- *opts.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opts.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opts.wall* — Wall index(es) to plot
- *opts.zIdx* — Axial indices to include in the plot
- *opts.obstructions* — Logical flag to display obstruction locations
- *opts.annular* — Logical flag to plot annular two-phase flow parameters only from the onset of annular flow
- *opts.unitTemp* — Temperature unit: 'K' or 'C'
- *opts.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'

- `opts.resize` — Resize factor for plot scaling

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if `opts.obstructions` is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotzt(*tfSolver*, *opt*)

PLOTZT Plots 2D time/elevation distributions of three-field parameters.

Generates surface plots of selected three-field related parameters over time and axial (elevation) positions. It supports plotting for film and drop fields, and can handle both real and null (initial) transient data.

Inputs:

- `tfSolver` — [Solvers.ThreeField.ThreeFieldSolver](#) object containing simulation data
- `opt.display` — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- `opt.label` — Corresponding labels for display parameters
- `opt.unit` — Units for each parameter
- `opt.field` — Field to plot: 'film' or 'drop'
- `opt.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opt.wall` — Wall index(es) to plot
- `opt.zIdx` — Axial indices (must include at least 2)
- `opt.tIdx` — Time indices (must include at least 2)
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.shading` — Surface shading style: 'faceted', 'flat', or 'interp'
- `opt.view` — View angle for 3D plot [azimuth elevation]

Notes:

- If 'ALL' is passed to `opt.display`, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

Solvers.ThreeField.ThreeFieldSolver.solve(*tfSolver*)

SOLVE Executes the three-field solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.ThreeField.ThreeFieldSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (`solveINIT = true`)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for phase flows, velocities, and enthalpies
- Supports thermal non-equilibrium modeling
- Logs progress and outputs to session directory

class `Solvers.ThreeField.Drop(inputSet, fluid)`

Bases: `Solvers.AbstractField`

DROP Class for modeling drop field in three-field solver

This class encapsulates the physical and numerical properties of the drop field, including flow variables and phase interactions. It supports multiple solver models and provides methods for computing derived quantities.

Constructor Summary

Drop(inputSet, fluid)

DROP Constructor for Drop class

Initializes the drop field object with input configuration and fluid properties.

Inputs:

- inputSet — `Inputs.InputSet` object containing model, geometry, and boundary conditions
- fluid — `Inputs.FluidProperties` object containing thermophysical fluid data

Property Summary

DT = 0

Time step size [s] from `Inputs.Options.TSTEP`

H = 1E6

Enthalpy [J/kg]

ITR

Iteration tracking

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from `Inputs.Model.NNODES`

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

mix = NaN

`Solvers.Mixture.Mixture` object

Method Summary

AI(*drop*, *zIdx*)

AI Drop volumetric interfacial area [m^{-1}]

Calculates the interfacial area per unit volume for drops.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

AREA(*drop*, *zIdx*)

AREA Drop interfacial area [m^2]

Calculates the surface area of a spherical drop.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

CONC(*drop*, *zIdx*)

CONC Drop concentration [kg/m^3]

Calculates the local drop-field concentration based on mass flow and vapor-phase properties.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

DENSITY(*drop*, *zIdx*)

DENSITY Drop number density [m^{-3}]

Calculates the number of drops per unit volume based on flow properties and geometry.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

DIAM(*drop*, *zIdx*)

DIAM Drop diameter [m]

Returns the drop diameter based on [Inputs.Model.DROPDIAM](#).

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

DRAG(*drop*, *zIdx*)

DRAG Drop drag coefficient [-]

Calculates drag coefficient using selected correlation based on [Inputs.Model.DROPDRAG](#).

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

FBUOY(*drop*, *zIdx*)

FBUOY Drop buoyancy force [N/m^3]

Calculates buoyancy force based on pressure gradient.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

FDRAG(*drop, zIdx*)

FDRAG Vapor drag force on drops [N/m³]

Calculates shear force exerted by vapor on drops.

Inputs:

- drop — [Solvers.ThreeField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

FENT(*drop, film, zIdx*)

FENT Film entrainment shear force [N/m³]

Calculates shear force due to film entrainment acting on drops.

Inputs:

- drop — [Solvers.ThreeField.Drop](#) object
- film — [Solvers.ThreeField.Film](#) object
- zIdx — Axial indices to evaluate (optional)

FGRAV(*drop, zIdx*)

FGRAV Gravitational force on drops [N/m³]

Calculates gravity force acting on drops based on channel inclination angle.

Inputs:

- drop — [Solvers.ThreeField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

FTOT(*drop, film, zIdx, opt*)

FTOT Total force acting on drops [N/m³]

Calculates combined forces (drag, buoyancy, gravity, entrainment) based on selected option.

Inputs:

- drop — [Solvers.ThreeField.Drop](#) object
- film — [Solvers.ThreeField.Film](#) object
- zIdx — Axial indices to evaluate (optional)
- opt — Option flag (0: full balance, 1: simplified)

KENH(*drop, zIdx*)

KENH Drop deposition enhancement factor [-]

Calculates axial variation of deposition enhancement using Windecker correlation or other models based on [Inputs.Model.DEPENHANCEMENT](#).

Inputs:

- drop — [Solvers.ThreeField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

MDEP(*drop, zIdx, enhanced*)

MDEP Drop deposition mass flux [kg/m²/s]

Implements various deposition models selected based on [Inputs.Model.DEPOSITION](#). Applies enhancement factors, and restricts deposition to annular flow region. The enhancement flag allow to enable/disable deposition enhancement

Inputs:

- drop — [Solvers.ThreeField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)
- enhanced — Flag indicating enhanced deposition downstream obstructions (optional, default = true)

RE(*drop*, *zIdx*)

RE Drop Reynolds number [-]

Calculates the Reynolds number for the drop field based on mass flow rate, viscosity, and channel perimeter.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

REV(*drop*, *zIdx*)

REV Drop Reynolds number with respect to vapor properties [-]

Calculates Reynolds number using vapor density, viscosity, and relative phase velocity.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

UALGEBR(*drop*, *film*, *zIdx*)

UALGEBR Algebraic drop velocity [m/s]

Calculates drop velocity consistent with mixture model based on void fraction and geometry.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *film* — [Solvers.ThreeField.Film](#) object
- *zIdx* — Axial indices to evaluate (optional)

UEQUIL(*drop*, *film*, *zIdx*, *opt*)

UEQUIL Drop velocity based on equilibrium model ($F_{tot} = 0$) [m/s]

Iteratively solves for drop velocity such that the total force acting on the drop phase is zero (force balance). Uses secant method with relaxation for convergence.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *film* — [Solvers.ThreeField.Film](#) object
- *zIdx* — Axial indices to evaluate (optional)
- **opt** — **Option flag for force balance:**
0 = full force balance (drag, buoyancy, gravity, entrainment) 1 = simplified (drag + buoyancy only)

USLIP(*drop*, *zIdx*)

USLIP Slip velocity model for drops [m/s]

Calculates drop velocity using slip ratio model based on [Inputs.Model.DROPSLIP](#).

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

VOLUME(*drop*, *zIdx*)

VOLUME Drop volume [m³]

Calculates the volume of a spherical drop.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

VR(*drop*, *zIdx*)

VR Drop local relative velocity [m/s]

Calculates the velocity difference between vapor and drop fields.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Only AREAMEAN option is implemented.

X(*drop*, *zIdx*)

X Vapor quality in the bulk (drop) region [-]

Vapor quality in the bulk region, considering only the droplet for the liquid phase. The bulk region is complementary of the near-wall region, defined in the mixture solver.

Inputs:

- *drop* — [Solvers.ThreeField.Drop](#) object
- *zIdx* — Axial indices to evaluate (optional)

class [Solvers.ThreeField.Film](#)Bases: [Solvers.AbstractFilm](#)

FILM Class for modeling liquid film in three-field solver

This class encapsulates the physical and numerical properties of the film field, including flow variables and phase interactions.

Property Summary**DT = 0**Time step size [s] from [Inputs.Options.TSTEP](#)**H = 1E6**

Enthalpy [J/kg]

HFLUX = 1.

Film heat flux [W/m^2]

ITR

Iteration tracking

MEVAP = -1.

Evaporation mass flux [kg/s/m^2]

NTIME = 0

Number of time steps [-]

NZ = 0Number of axial steps [-] from [Inputs.Model.NNODES](#)**TIDX = 1**

Time step index [-]

TIME = 0

Time series [s]

U = 1.
Velocity [m/s]

W = 1.
Mass flow rate [kg/s]

Z = 1.
Elevation [m]

3.4.4 Two-fluid solver

The *Solvers.TwoFluid* module in **OpenSTREAM** provides classes and tools for simulating two-phase flows using a two-fluid approach. In this method, each phase (liquid and vapor) is modeled as an interpenetrating continuum with its own set of conservation equations for mass, momentum, and energy. This framework enables explicit representation of phase interactions, slip velocities, and non-equilibrium effects, making it well-suited for applications where phase separation, interfacial dynamics, and transient boiling phenomena are significant.

This module includes:

- Solver class for two-fluid-based flow simulations: *Solvers.TwoFluid.TwoFluidSolver*
- Liquid and vapor field classes: *Solvers.TwoFluid.Liquid*, *Solvers.TwoFluid.Vapor*
- Mixture class: *Solvers.TwoFluid.Mixture*
- Two-fluid primary properties: *Solvers.TwoFluid.Liquid.W*, *Solvers.TwoFluid.Liquid.U*, *Solvers.TwoFluid.Liquid.H*, *Solvers.TwoFluid.Vapor.W*, *Solvers.TwoFluid.Vapor.U*, *Solvers.TwoFluid.Vapor.H*
- Key solver class methods: *Solvers.TwoFluid.TwoFluidSolver.initializeSolver*, *Solvers.TwoFluid.TwoFluidSolver.solve()*, *Solvers.TwoFluid.TwoFluidSolver.plotz*
- Field class methods for calculating secondary two-fluid parameters.

class *Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver*(*inputSet*, *mixSolver*)

Bases: *Solvers.AbstractSolver*

TWOFLUIDSOLVER Solver for initializing, solving, and visualizing two-fluid flow.

The *TwoFluidSolver* class handles the setup, execution, and visualization of thermal-hydraulic simulations using a two-phase two-fluid approach.

Responsibilities:

- Initializes solver parameters from an *Inputs.InputSet* object
- Constructs *Solvers.TwoFluid.Liquid* and *Solvers.TwoFluid.Vapor* objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Phase construction (liquid/vapor)
- Phase mass/momentum/energy transport modeling
- Pressure drop gradient obtained from mixture solver
- Support for relaxation models

- Visualization of results across time and axial domains

Constructor Summary

TwoFluidSolver(*inputSet*, *mixSolver*)

TWOFLUIDSOLVER Constructor for the TwoFluidSolver class.

Creates a new instance of the TwoFluidSolver class using the provided *Inputs.InputSet*. This constructor initializes the solver by calling the abstract *Solvers.AbstractSolver* superclass constructor and then sets up all necessary solver parameters and internal data structures via *Solvers.TwoFluid.TwoFluidSolver.initializeSolver*

Input:

- *inputSet* — An *Inputs.InputSet* object containing model configuration, geometry, boundary conditions, and solver options.

Notes:

- This constructor assumes *Inputs.InputSet* is fully validated.
- Solver initialization includes liquid and vapor objects setup.
- Time/space discretization and boundary condition interpolation initialized by *Solvers.Mixture.MixtureSolver* are used

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

DZ = 0

Axial step size [m]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIME = 0

Time series [s]

Z = 1.

Elevation [m]

boundaryConditions

Boundary conditions object *Inputs.BoundaryConditions*

fluid

Fluid object *Inputs.FluidProperties*

inputSet

Input set object *Inputs.InputSet*

liquid

Liquid object

liquidInit

Null transient liquid object

mixSolver

Mixture object *Solvers.Mixture.Mixture*

vapor

Vapor object

vaporInit

Null transient vapor object

Method Summary**initializeSolver**(*twfSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs liquid/vapor objects.

Sets up the solver's internal state using the provided [Inputs.InputSet](#) and [Solvers.TwoFluid.TwoFluidSolver](#). This includes initialization of data structures for transient and steady-state simulations based on the mixture solver solutions.

Operations performed:

- Initializes liquid and vapor arrays for each time step
- Initializes fluid property objects
- Sets up mixture object
- Assigns initial conditions for phase mass flow rates, velocities, enthalpies

Notes:

- The function assumes uniform axial discretization.
- Wall heat flux is computed from interpolated boundary conditions.

plott(*twfSolver*, *zIdx*, *opt*)

PLOTT Plots temporal distributions of two-fluid parameters at a given axial location.

Generates time-series plots of selected two-fluid parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- twfSolver — [Solvers.TwoFluid.TwoFluidSolver](#) object containing simulation data
- zIdx — Axial index (scalar, positive integer)
- opt.display — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- opt.solveMode — Solve mode: 'REAL' or 'NULL'
- opt.wall — Wall index(es) to plot
- opt.tIdx — Time indices to include in the plot
- opt.reverseTime — Logical flag to reverse time axis
- opt.unitTemp — Temperature unit: 'K' or 'C'
- opt.arrangement — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- opt.resize — Resize factor for plot scaling

Notes:

- If opt.display is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotz(*twfSolver*, *tIdx*, *opts*)

PLOTZ Plots spatial (axial) distributions of two-fluid parameters.

Generates axial plots of selected two-fluid parameters at specified time indices. The function supports multiple display modes, wall selections, and optional animation over time.

Inputs:

- twfSolver — [Solvers.TwoFluid.TwoFluidSolver](#) object containing simulation data
- tIdx — Time index or indices (vector of positive integers)
- opts.display — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- opts.solveMode — Solve mode: 'REAL' or 'NULL'

- `opts.wall` — Wall index(es) to plot
- `opts.zIdx` — Axial indices to include in the plot
- `opts.obstructions` — Logical flag to display obstruction locations
- `opts.unitTemp` — Temperature unit: 'K' or 'C'
- `opts.arrangement` — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- `opts.resize` — Resize factor for plot scaling

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if `opts.obstructions` is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotzt(*twfSolver*, *opt*)

PLOTZT Plots 2D time/elevation distributions of two-fluid parameters.

Generates surface plots of selected two-fluid related parameters over time and axial (elevation) positions. It supports plotting for liquid and vapor fields, and can handle both real and null (initial) transient data.

Inputs:

- `twfSolver` — [Solvers.TwoFluid.TwoFluidSolver](#) object containing simulation data
- `opt.display` — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- `opt.label` — Corresponding labels for display parameters
- `opt.unit` — Units for each parameter
- `opt.field` — Field to plot: 'liquid' or 'vapor'
- `opt.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opt.wall` — Wall index(es) to plot
- `opt.zIdx` — Axial indices (must include at least 2)
- `opt.tIdx` — Time indices (must include at least 2)
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.shading` — Surface shading style: 'faceted', 'flat', or 'interp'
- `opt.view` — View angle for 3D plot [azimuth elevation]

Notes:

- If 'ALL' is passed to `opt.display`, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

Solvers.TwoFluid.TwoFluidSolver.solve(*twfSolver*)

SOLVE Executes the two-fluid solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.TwoFluid.TwoFluidSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (`solveINIT = true`)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for phase flows, velocities, and enthalpies
- Supports thermal non-equilibrium modeling
- Logs progress and outputs to session directory

class `Solvers.TwoFluid.Vapor(inputSet, fluid)`

Bases: `Solvers.AbstractField`

VAPOR Class for modeling vapor field in two-fluid solver

This class encapsulates the physical and numerical properties of the vapor phase, including flow variables, phase interactions, heat transfer, and relaxation models. It supports multiple solver models and provides methods for computing derived quantities and handling flow regime transitions.

Constructor Summary

Vapor(*inputSet, fluid*)

VAPOR Constructor for Vapor class

Initializes the vapor field object with input configuration and fluid properties.

Inputs:

- *inputSet* — `Inputs.InputSet` object containing model, geometry, and boundary conditions
- *fluid* — `Inputs.FluidProperties` object containing thermophysical fluid data

Property Summary

DT = 0

Time step size [s] from `Inputs.Options.TSTEP`

H = 1E6

Enthalpy [J/kg]

ITR

Iteration properties

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from `Inputs.Model.NNODES`

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

mix = NaN

Solvers.TwoFluid.Mixture object

Method Summary

AREA(*vapor, liquid, zIdx*)

AREA Vapor cross-section area [m²]

Computes the effective cross-sectional area occupied by the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

CD(*vapor, liquid, zIdx*)

CD Drag coefficient for dispersed gas [-]

Computes the drag coefficient for dispersed vapor based on *Inputs.Model.DROPDRAG* model.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Values are clamped to a maximum of 1

FBUOY(*vapor, liquid, zIdx*)

FBUOY Vapor buoyancy force [N/m]

Computes the buoyancy force acting on the vapor phase due to pressure gradient.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- The pressure gradient from the mixture solver is used

FDRAg(*vapor, liquid, zIdx*)

FDRAg Interfacial drag force [N/m]

Computes the interfacial drag force acting on the vapor phase by reversing the sign of the liquid-phase drag (*Solvers.TwoFluid.Liquid.FDRAg()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FGRAV(*vapor, liquid, zIdx*)

FGRAV Vapor gravity force [N/m]

Computes the gravitational force acting on the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FINTEVAP(*vapor, liquid, zIdx*)

FINTEVAP Momentum exchange through interfacial evaporation [N/m]

Computes the momentum exchange due to interfacial evaporation.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FLOWREGIME(*vapor, liquid, zIdx*)

FLOWREGIME Categorical two-phase flow regimes

Returns the flow regime classification based on liquid-phase criteria (*Solvers.TwoFluid.Liquid.FLOWREGIME*)().

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- flowregime — Flow regime category (from *Solvers.TwoFluid.REGIMES*)

FTOT(*vapor, liquid, zIdx*)

FTOT Total force [N/m]

Computes the total force acting on the vapor phase, including gravity, buoyancy, wall shear, drag, and interfacial momentum exchanges.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FWALEVAP(*vapor, liquid, zIdx*)

FWALEVAP Momentum exchange through wall evaporation [N/m]

Computes the momentum exchange due to wall boiling evaporation.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FWALL(*vapor, liquid, zIdx*)

FWALL Wall shear force [N/m]

Computes the wall shear force acting on the vapor phase, weighted by void fraction.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HFLUX(*vapor, zIdx*)

HFLUX Wall heat flux to vapor phase [W/m^2]

Computes the net wall heat flux transferred to the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object

- `zIdx` — Axial indices to evaluate (optional)

Notes:

- Uses `Solvers.TwoFluid.Vapor.HWALHEAT` divided by wall perimeter

HFLUXWALEVAP(*vapor, liquid, zIdx*)

HFLUXWALEVAP Wall boiling heat flux [W/m²]

Computes the portion of wall heat flux attributed to boiling (evaporation) for the vapor phase, based on liquid-phase calculations (`Solvers.TwoFluid.Liquid.HFLUXWALEVAP()`).

Inputs:

- `vapor` — `Solvers.TwoFluid.Vapor` object
- `liquid` — `Solvers.TwoFluid.Liquid` object
- `zIdx` — Axial indices to evaluate (optional)

HINT(*vapor, liquid, zIdx*)

HINT Linear interfacial heat transfer rate [W/m]

Computes the interfacial heat transfer rate for the vapor phase based on mass transfer and enthalpy difference (based on `Inputs.Model.INTTRANS` model).

Inputs:

- `vapor` — `Solvers.TwoFluid.Vapor` object
- `liquid` — `Solvers.TwoFluid.Liquid` object
- `zIdx` — Axial indices to evaluate (optional)

Returns:

- `Hcond` — Condensation heat transfer rate [W/m]
- `Hevap` — Evaporation heat transfer rate [W/m]

HINTCOND(*vapor, liquid, zIdx*)

HINTCOND Linear interfacial heat condensation rate [W/m]

Extracts the condensation component from `Solvers.TwoFluid.Vapor.HINT`.

Inputs:

- `vapor` — `Solvers.TwoFluid.Vapor` object
- `liquid` — `Solvers.TwoFluid.Liquid` object
- `zIdx` — Axial indices to evaluate (optional)

HINTEVAP(*vapor, liquid, zIdx*)

HINTEVAP Linear interfacial heat evaporation rate [W/m]

Extracts the evaporation component from `Solvers.TwoFluid.Vapor.HINT`.

Inputs:

- `vapor` — `Solvers.TwoFluid.Vapor` object
- `liquid` — `Solvers.TwoFluid.Liquid` object
- `zIdx` — Axial indices to evaluate (optional)

HTOT(*vapor, liquid, zIdx*)

HTOT Total linear vapor heat rate [W/m]

Computes the total heat transfer to the vapor phase, including interfacial and wall contributions.

Inputs:

- `vapor` — `Solvers.TwoFluid.Vapor` object
- `liquid` — `Solvers.TwoFluid.Liquid` object
- `zIdx` — Axial indices to evaluate (optional)

HWALEVAP(*vapor, liquid, zIdx*)

HWALEVAP Linear wall heat evaporation (boiling) rate [W/m]

Computes the wall boiling heat transfer rate using latent heat (based on *Inputs.Model.INTTRANSH* model) and wall mass evaporation.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HWALHEAT(*vapor, zIdx*)

HWALHEAT Linear wall heat to vapor rate [W/m]

Returns the wall heat generation rate transferred to the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Set to zero before CBT

INTAREA(*vapor, liquid, zIdx*)

INTAREA Volumetric interfacial area [m⁻¹]

Computes the interfacial area concentration using liquid-phase logic (from *Solvers.TwoFluid.Liquid.INTAREA()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

INTHFLUX(*vapor, liquid, zIdx*)

INTHFLUX Interfacial heat flux [W/m²]

Computes the interfacial heat flux due to evaporation and condensation (delegated to liquid-phase calculation in *Solvers.TwoFluid.Liquid.INTHFLUX()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- inthflux_evap — Evaporation heat flux [W/m²]
- inthflux_cond — Condensation heat flux [W/m²]

Notes:

- Positive toward liquid phase

INTNU(*vapor, liquid, zIdx*)

INTNU Interfacial Nusselt number for dispersed vapor [-]

Computes the interfacial Nusselt number based on *Inputs.Model.INTNU* model.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

J(*vapor*, *zIdx*)

J Vapor superficial velocity [m/s]

Computes the superficial velocity of the vapor phase.

Inputs:

- *vapor* — [Solvers.TwoFluid.Vapor](#) object
- *zIdx* — Axial indices to evaluate (optional)

L(*vapor*, *zIdx*)

L Dispersed vapor interfacial length scale [m]

Returns the characteristic length scale for dispersed vapor (e.g., bubble diameter).

Inputs:

- *vapor* — [Solvers.TwoFluid.Vapor](#) object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Currently uses a constant model

MINT(*vapor*, *liquid*, *zIdx*)

MINT Linear interfacial mass transfer rates [kg/s/m]

Computes condensation and evaporation mass transfer rates for the vapor phase by reversing the sign of liquid-phase values (attr:[Solvers.TwoFluid.Liquid.MINT](#)).

Inputs:

- *vapor* — [Solvers.TwoFluid.Vapor](#) object
- *liquid* — [Solvers.TwoFluid.Liquid](#) object
- *zIdx* — Axial indices to evaluate (optional)

Returns:

- *Mcond* — Condensation mass transfer rate [kg/s/m]
- *Mevap* — Evaporation mass transfer rate [kg/s/m]

MINTCOND(*vapor*, *liquid*, *zIdx*)

MINTCOND Linear interfacial condensation mass transfer [kg/s/m]

Extracts the condensation component from [Solvers.TwoFluid.Vapor.MINT](#).

Inputs:

- *vapor* — [Solvers.TwoFluid.Vapor](#) object
- *liquid* — [Solvers.TwoFluid.Liquid](#) object
- *zIdx* — Axial indices to evaluate (optional)

MINTEVAP(*vapor*, *liquid*, *zIdx*)

MINTEVAP Linear interfacial evaporation mass transfer [kg/s/m]

Extracts the evaporation component from [Solvers.TwoFluid.Vapor.MINT](#).

Inputs:

- *vapor* — [Solvers.TwoFluid.Vapor](#) object
- *liquid* — [Solvers.TwoFluid.Liquid](#) object
- *zIdx* — Axial indices to evaluate (optional)

MTOT(*vapor*, *liquid*, *zIdx*)

MTOT Total linear mass transfer [kg/s/m]

Computes the total vapor mass transfer rate by reversing the sign of the liquid-phase total ([Solvers.TwoFluid.Liquid.MTOT\(\)](#)).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

MWALEVAP(*vapor, liquid, zIdx*)

MWALEVAP Linear wall mass evaporation rate [kg/s/m]

Computes the wall boiling mass transfer rate for vapor by reversing the sign of the liquid-phase value (meth:*Solvers.TwoFluid.Liquid.MWALEVAP*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

RE(*vapor, zIdx*)

RE Vapor Reynolds number [-]

Computes the vapor-phase Reynolds number based on flow rate and viscosity.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

REL(*vapor, liquid, zIdx*)

REL Dispersed vapor Reynolds number [-]

Computes the Reynolds number for dispersed vapor using liquid properties and relative velocity.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

S(*vapor, liquid, zIdx*)

S Phase slip ratio [-]

Returns the slip ratio between vapor and liquid phases.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

T(*vapor, zIdx*)

T Vapor temperature [K]

Computes the vapor temperature from enthalpy.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

TAUW(*vapor, zIdx*)

TAUW Wall shear stress [N/m²]

Returns the wall shear stress acting on the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Uses mixture-level shear stress for consistency

USLIP(*vapor, liquid, zIdx*)

USLIP Vapor velocity based on input phase slip [m/s]

Computes the vapor velocity using slip ratio and liquid velocity.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

VF(*vapor, liquid, zIdx*)

VF Vapor volumetric fraction [-]

Computes the vapor volumetric (i.e., void) fraction using slip ratio and densities.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

VISC(*vapor, liquid, zIdx*)

VISC Viscosity number for dispersed gas [-]

Computes the dimensionless viscosity number using liquid and vapor properties.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

WALEVAPRATIO(*vapor, liquid, zIdx*)

WALEVAPRATIO Wall evaporation mass ratio [-]

Returns the fraction of liquid mass undergoing wall boiling.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- walevapratio — Wall boiling mass ratio [-]

X(*vapor, zIdx*)

X Vapor mass fraction [-]

Computes the vapor mass fraction relative to the mixture flow.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

XTR_ANN(*vapor, liquid*)

XTR_ANN Quality at onset of annular flow region

XTR_CBT(*vapor, liquid*)

XTR_CBT Quality at Critical Boiling Transition [-]

Returns the equilibrium quality threshold for the onset of critical boiling transition.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

XTR_ITM(*vapor, liquid*)

XTR_ITM Quality at onset of intermediate region [-]

Returns the equilibrium quality threshold for the onset of the intermediate flow regime.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SAT(*vapor, liquid*)

XTR_SAT Quality at end of subcooled boiling transition [-]

Returns the equilibrium quality threshold for the onset of saturated boiling.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SUB(*vapor, liquid*)

XTR_SUB Quality at onset of subcooled boiling transition [-]

Returns the equilibrium quality threshold for the onset of subcooled boiling.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

class *Solvers.TwoFluid.Mixture*(*mixture, liquid, vapor*)

Bases: *Solvers.AbstractPhase*

MIXTURE Class for two-fluid mixture properties in thermal-hydraulic solver

Represents combined properties of liquid and vapor phases in a two-fluid solver. Provides methods to compute mixture-level quantities such as mass flow, density, velocity, heat flux, and thermodynamic quality.

Responsibilities:

- Aggregate liquid and vapor properties into mixture-level results
- Compute derived quantities (void fraction, equilibrium quality, CHF)
- Provide access to geometry and fluid properties for calculations

Key features:

- Supports axial indexing for spatially resolved calculations
- Interfaces with liquid and vapor sub-objects for phase-specific data

Constructor Summary

Mixture(*mixture, liquid, vapor*)

MIXTURE Constructor for Mixture class

Initializes the mixture object by linking liquid and vapor phase objects and copying solver configuration and fluid properties.

Inputs:

mixture — *Solvers.Mixture.Mixture* object liquid — *Solvers.TwoFluid.Liquid* object
vapor — *Solvers.TwoFluid.Vapor* object

Method Summary**AFDISTR**(*mix*, *param1*, *param2*, *zIdx*)

AFDISTR Annular flow distribution function [-]

Computes the annular flow distribution function for given parameters.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *param1* — First parameter for distribution function
param2 — Second parameter for distribution function *zIdx* — Axial indices to evaluate (optional)**CBT**(*mix*, *zIdx*)

CBT Critical Boiling Transition flag [-]

Returns the CBT flag for each axial location.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)**CHF**(*mix*, *zIdx*)CHF Critical Heat Flux [W/m²]

Returns the CHF value for each axial location.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)**DPDZ**(*mix*, *zIdx*)

DPDZ Pressure gradient [Pa/m]

Computes the axial pressure gradient from mixture pressure drop.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)**H**(*mix*, *zIdx*)

H Mixture enthalpy [J/kg]

Calculates the mixture enthalpy as a mass-weighted average of phase enthalpies.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)**HFLUX**(*mix*, *zIdx*)HFLUX Wall heat flux [W/m²]

Returns the wall heat flux distribution along the axial domain.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)**LHGR**(*mix*, *zIdx*)

LHGR Linear heat generation rate [W/m]

Computes the linear heat generation rate from wall heat flux and perimeter.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)**MFLUX**(*mix*, *zIdx*)MFLUX Mixture mass flux [kg/m²/s]

Calculates the mixture mass flux using total flow and channel area.

Inputs:*mix* — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

OAFX(*mix*, *zIdx*)

OAFX Onset of annular flow equilibrium quality [-]

Returns the OAF quality from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

RELAXTCOND(*mix*, *zIdx*)

RELAXTCOND Time relaxation for interfacial evaporation [s]

Returns the relaxation time applied to interfacial evaporation processes in the two-fluid mixture model.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

RELAXTEVAP(*mix*, *zIdx*)

RELAXTEVAP Time relaxation for interfacial condensation [s]

Returns the relaxation time applied to interfacial condensation processes in the two-fluid mixture model.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

RHO(*mix*, *zIdx*)

RHO Mixture density [kg/m³]

Computes the mixture density based on void fraction and phase densities.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

TAUW(*mix*, *zIdx*)

TAUW Wall shear stress [N/m²]

Returns the wall shear stress from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

TIME(*mix*)

TIME Time series [s]

Returns the time array from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object

U(*mix*, *zIdx*)

U Mixture velocity [m/s]

Calculates the mixture velocity as a mass-weighted average of phase velocities.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

VF(*mix*, *zIdx*)

VF Void fraction [-]

Computes the vapor volume fraction in the mixture.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

W(*mix*, *zIdx*)

W Mixture mass flow rate [kg/s]

Computes the total mass flow rate as the sum of liquid and vapor flows.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

WALEVAPRATIO(mix, zIdx)

WALEVAPRATIO Wall mass evaporation ratio [-]

Returns the ratio of wall heat flux converted to evaporation.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

XEQ(mix, zIdx)

XEQ Equilibrium thermodynamic quality [-]

Returns the equilibrium quality from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

Z(mix, zIdx)

Z Axial nodes [m]

Returns the axial positions for the mixture domain.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

class *Solvers.TwoFluid.Liquid*(inputSet, fluid)

Bases: *Solvers.AbstractField*

LIQUID Class for modeling liquid field in two-fluid solver

This class encapsulates the physical and numerical properties of the liquid phase, including flow variables, phase interactions, heat transfer, and relaxation models. It supports multiple solver models and provides methods for computing derived quantities and handling flow regime transitions.

Constructor Summary**Liquid**(inputSet, fluid)

LIQUID Constructor for Liquid class

Initializes the liquid field object with input configuration and fluid properties.

Inputs:

- inputSet — *Inputs.InputSet* object containing model, geometry, and boundary conditions
- fluid — *Inputs.FluidProperties* object containing thermophysical fluid data

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

H = 1E6

Enthalpy [J/kg]

ITR

Iteration tracking

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

mix = NaN

Solvers.TwoFluid.Mixture object

Method Summary

AREA(*liquid, vapor, zIdx*)

AREA Liquid cross-section area [m²]

Computes the effective cross-sectional area occupied by the liquid phase.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

CD(*liquid, vapor, zIdx*)

CD Drag coefficient for dispersed liquid [-]

Computes the drag coefficient for dispersed liquid based on *Inputs.Model.DROPDRAG* model.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Values are clamped to a maximum of 1

FBUOY(*liquid, vapor, zIdx*)

FBUOY Liquid buoyancy force [N/m]

Computes the buoyancy force acting on the liquid phase due to pressure gradient.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- The pressure gradient from the mixture solver is used

FDRAG(*liquid, vapor, zIdx*)

FDRAG Interfacial drag force [N/m]

Computes the interfacial drag force acting on the liquid phase.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Uses smooth transition between dispersed gas and liquid drag based on *Inputs.Model.DISPGAS2DISPLIQ* model.

FGRAV(*liquid, vapor, zIdx*)

FGRAV Liquid gravity force [N/m]

Computes the gravitational force acting on the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

FINTCOND(*liquid, vapor, zIdx*)

FINTCOND Momentum exchange through interfacial condensation [N/m]

Computes the momentum exchange due to interfacial condensation.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

FLOWID(*liquid, zIdx*)

FLOWID Two-phase flow regime ID

Returns the integer ID corresponding to the flow regime classification.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FLOWREGIME(*liquid, zIdx*)

FLOWREGIME Categorical two-phase flow regimes

Classifies flow regime based on equilibrium quality and boiling transition flags.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- flowregime — Categorical flow regime (from *Solvers.TwoFluid.REGIMES*)
- id — Integer regime ID

FTOT(*liquid, vapor, zIdx*)

FTOT Total force [N/m]

Computes the total force acting on the liquid phase, including gravity, buoyancy, wall shear, drag, and interfacial condensation.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

FWALL(*liquid, vapor, zIdx*)

FWALL Liquid wall shear force [N/m]

Computes the wall shear force acting on the liquid phase, weighted by void fraction.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HFLUX(*liquid, zIdx*)

HFLUX Wall heat flux to liquid phase [W/m^2]

Computes the net wall heat flux transferred to the liquid phase, combining direct wall heating and boiling contributions.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Combines *Solvers.TwoFluid.Liquid.HWALHEAT* and *Solvers.TwoFluid.Liquid.HFLUXWALEVAP*
- Uses wall perimeter from geometry

HFLUXWALEVAP(*liquid, zIdx*)

HFLUXWALEVAP Wall evaporation (boiling) heat flux [W/m^2]

Computes the portion of wall heat flux attributed to evaporation (e.g., boiling), based on the wall evaporation mass ratio.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HINT(*liquid, vapor, zIdx*)

HINT Linear interfacial heat transfer rate [W/m]

Computes the interfacial heat transfer rate using mass transfer and enthalpy difference (based on *Inputs.Model.INTTRANSH* model).

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- Hcond — Condensation heat transfer rate [W/m]
- Hevap — Evaporation heat transfer rate [W/m]

HINTCOND(*liquid, vapor, zIdx*)

HINTCOND Linear interfacial heat condensation rate [W/m]

Extracts the condensation component from *Solvers.TwoFluid.Liquid.HINT*.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HINTEVAP(*liquid, vapor, zIdx*)

HINTEVAP Linear interfacial heat evaporation rate [W/m]

Extracts the evaporation component from *Solvers.TwoFluid.Liquid.HINT*.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HTOT(*liquid, vapor, zIdx*)

HTOT Total linear vapor heat rate [W/m]

Computes the total heat transfer to the vapor phase, including interfacial and wall contributions.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HWALEVAP(*liquid, vapor, zIdx*)

HWALEVAP Linear wall heat evaporation (boiling) rate [W/m]

Computes the wall boiling heat transfer rate using latent heat (based on *Inputs.Model.INTTRANSH* model) and wall mass evaporation.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HWALHEAT(*liquid, zIdx*)

HWALHEAT Linear wall heat to liquid rate [W/m]

Returns the wall heat generation rate transferred to the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Set to zero beyond CBT

INTAREA(*liquid, vapor, zIdx*)

INTAREA Volumetric interfacial area [m⁻¹]

Computes the interfacial area concentration based on flow regime and phase topology.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Uses a smooth transition between dispersed gas and dispersed liquid models

INTHFLUX(*liquid, vapor, zIdx*)

INTHFLUX Interfacial heat flux [W/m²]

Computes the interfacial heat flux due to evaporation and condensation based on *Inputs.Model.INTAREA* interfacial area model.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- inthflux_evap — Evaporation heat flux [W/m²]
- inthflux_cond — Condensation heat flux [W/m²]

Notes:

- Uses interfacial Nusselt number and length scale
- Driven by deviation from saturated temperature

INTNU(*liquid, vapor, zIdx*)

INTNU Interfacial Nusselt number for dispersed liquid [-]

Computes the interfacial Nusselt number based on *Inputs.Model.INTNU* model.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

J(*liquid, zIdx*)

J Liquid superficial velocity [m/s]

Computes the superficial velocity of the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

L(*liquid, zIdx*)

L Dispersed liquid interfacial length scale [m]

Returns the characteristic length scale for dispersed liquid (e.g., drop Sauter mean diameter).

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Currently uses a constant model

MINT(*liquid, vapor, zIdx*)

MINT Linear interfacial mass transfer rates [kg/s/m]

Computes condensation and evaporation mass transfer rates using either heat flux-based or relaxation-based models.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- Mcond — Condensation mass transfer rate [kg/s/m]
- Mevap — Evaporation mass transfer rate [kg/s/m]

Notes:

- Supports ‘CONSTANT’, ‘RANZMARSHALL’, and ‘RELAXATION’ models based on *Inputs.Model.INTNU* model
- Applies wall-wise distribution

MINTCOND(*liquid, vapor, zIdx*)

MINTEVAP Linear interfacial condensation mass transfer [kg/s/m]

Extracts the evaporation component from *Solvers.TwoFluid.Liquid.MINT*.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

MINTEVAP(*liquid, vapor, zIdx*)

MINTEVAP Linear interfacial evaporation mass transfer [kg/s/m]

Extracts the evaporation component from *Solvers.TwoFluid.Liquid.MINT*.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

MTOT(*liquid, vapor, zIdx*)

MTOT Total linear mass transfer [kg/s/m]

Computes the total vapor mass transfer rate by summing interfacial condensation, evaporation, and wall boiling contributions.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

MWALEVAP(*liquid, vapor, zIdx*)

MWALEVAP Linear wall mass evaporation rate [kg/s/m]

Computes the wall boiling mass transfer rate using latent heat (based on *Inputs.Model.INTTRANSH* model) and wall heat flux.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

RE(*liquid, zIdx*)

RE Liquid Reynolds number [-]

Computes the liquid-phase Reynolds number based on flow rate and viscosity.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

REV(*liquid, vapor, zIdx*)

REV Dispersed liquid Reynolds number [-]

Computes the Reynolds number for dispersed liquid using vapor properties and relative velocity.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

S(*liquid, vapor, zIdx*)

S Phase vapor/liquid slip ratio [-]

Computes the slip ratio between vapor and liquid velocities.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

T(*liquid, zIdx*)

T Liquid temperature [K]

Computes the temperature of the liquid phase from enthalpy.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

TAUW(*liquid, zIdx*)

TAUW Liquid wall shear stress [N/m²]

Returns the wall shear stress acting on the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

USLIP(*liquid, vapor, zIdx*)

USLIP Liquid velocity based on input phase slip [m/s]

Computes the liquid velocity using a slip-based void fraction model.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

VF(*liquid, vapor, zIdx*)

VF Liquid volumetric fraction [-]

Computes the liquid volumetric fraction as the complement of vapor VF.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

VISC(*liquid, vapor, zIdx*)

VISC Viscosity number for dispersed liquid [-]

Computes the dimensionless viscosity number using vapor properties and interfacial tension.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

VR(*liquid, vapor, zIdx*)

VR Local relative velocity [m/s]

Computes the relative velocity between vapor and liquid phases using various models (AREAMEAN, SCALED, DRIFT, FLOWREGIME) based on *Inputs.Model.LOCRELVEL* model.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

WALEVAPRATIO(*liquid*, *zIdx*)

WALEVAPRATIO Wall evaporation mass ratio [-]

Returns the fraction of liquid mass undergoing wall boiling.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

X(*liquid*, *zIdx*)

X Liquid mass fraction [-]

Computes the liquid mass fraction relative to the mixture flow.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

XTR_ANN(*liquid*)

XTR_ANN Quality at onset of annular flow region [-]

Returns the equilibrium quality threshold for the onset of annular flow.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object

Notes:

- Uses minimum value from mix.OAFX

XTR_ITM(*liquid*)

XTR_ITM Quality at onset of intermediate region [-]

Returns the equilibrium quality threshold for the onset of the intermediate flow regime.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SAT(*liquid*)

XTR_SAT Quality at saturated boiling transition [-]

Returns the equilibrium quality threshold for the onset of saturated boiling.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SUB(*liquid*)

XTR_SUB Quality at subcooled boiling transition [-]

Returns the equilibrium quality threshold for the onset of subcooled boiling.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object

class Solvers.TwoFluid.REGIMES

Bases: double

REGIMES Enumeration of two-phase flow regimes

This class defines the available two-phase flow regimes for constitutive model selection, used by the two-fluid solver.

The flow regimes are defined as an enum class to increase computational efficiency while increasing readability of code, particularly when different calculations are carried out for different flow regimes.

Flow regimes:

- LIQUID — Single-phase liquid
- BUBBLY_SUBCOOLED — Bubbly flow in subcooled region
- BUBBLY_SATURATED — Bubble flow in saturated region
- INTERMEDIATE — Intermediate flow region
- ANNULAR — Annular flow
- DFFB — Dispersed Flow Film Boiling

The following is a list of associated superclasses, each with its respective properties and methods. These foundational components provide shared functionality and structure across solver implementations.

- Abstract solver: *Solvers.AbstractSolver*
 - Abstract field: *Solvers.AbstractField*
 - Abstract phase: *Solvers.AbstractPhase*
 - Abstract film: *Solvers.AbstractFilm*
 - Solver plotter: *Solvers.SolverPlotter*
 - Solver state: *Solvers.SolverState*
-

class Solvers.SolverState

Bases: uint16

SOLVERSTATE Enumeration of solver execution states

This enumeration defines the possible states a solver can be in during or after execution. It is used to track solver execution and convergence status.

States:

- UNSOLVED — Solver has not yet run
- SOLVEDCONVERGED — Solver completed and converged successfully
- SOLVEDNOTCONVERGED — Solver completed but did not converge
- INITIALSTEPCONVERGED — Initial steady-state step completed and converged
- INITIALSTEPNOTCONVERGED — Initial steady-state step completed but did not converge

class Solvers.**SolverPlotter**(*titles, WallIdxs, opts*)

Bases: handle

SOLVERPLOTTER Visualization framework for solver results.

This class provides a flexible plotting interface for visualizing solver results. It supports tiled layouts, animated series, and customizable plot styles. It also includes UI controls for animation playback and export.

Constructor Summary

SolverPlotter(*titles, WallIdxs, opts*)

SOLVERPLOTTER Constructor method to initialize plotter objects and layout

Property Summary

FontSize = 13

Font size of text in plots

Grid = 'on'

Grid display option

Title = ""

Title of the figure

Ts = []

Time series [s]

WallIdx = 1

Wall index for multi-wall simulations

Zs = []

Axial positions [m]

currentAhIdx = NaN

Index of current active axes

Method Summary

addTile(*plotters, opts*)

ADDTILE Method to add a new tile to the layout or retrieve existing one by title

static fieldName2plotStyle(*fieldName*)

FIELDNAME2PLOTSTYLE Static method to map field names to plot styles (color, line, marker)

gca(*plotters*)

GCA Get Method to retrieve current axes handle

gcf(*plotters*)

GCF Method to retrieve current plotter handle

legend(*plotters, varargin*)

LEGEND Control legends on current tile, a specific tile, or all tiles

Usage:

plotter.legend('Location','northwest') plotter.legend('Tile',2,'Location','best') plot-
ter.legend('AllTiles',true,'Location','south') plotter.legend('Axes',ah,'off')

newTile(*plotters, opts*)

NEWTILE Method to create a new tile with labels and formatting

plotK(*plotters, klocZ, ah*)

PLOTK Method to plot obstruction elevation indicators

plotOAF(*plotters, oafZ*)

PLOTTOAF Method to plot onset of annular flow boundary indicator

plotz(*plotters, YData, fieldName, opts*)

PLOTZ Method to plot spatial or temporal data (YData) on the current axes

resizeFigure(*plotters, scaleFactor*)

resizeFigure Resizes the figure based on tiledlayout rows/columns Keeps top-left fixed and goes full-screen if size exceeds screen resolution.

Usage:

- `plotters.resizeFigure()` — Default `scaleFactor` = 1
- `plotters.resizeFigure(1.2)` — Tiles 20% larger
- `plotters.resizeFigure(0.8)` — Tiles 20% smaller

setZs(*plotters, Zs*)

SETZS Method to assign the parameter(space or time) on the plotter x-axis

xlim(*plotters, newLim*)

xlim Method to get or set x-axis limits for current axes

ylabel(*plotters, labels*)

YLABEL Method to set y-axis tick labels

ylim(*plotters, newLim*)

YLIM Method to get or set y-axis limits for current axes

class `Solvers.AbstractFilm`(*inputSet, fluid*)

Bases: `Solvers.AbstractField`

ABSTRACTFILM Base class for liquid film models in three- and four-field solvers

This abstract class defines shared methods and properties for liquid film modeling. It includes calculations for film thickness, entrainment, shear forces, and velocity models. Specific entrainment models (e.g., Okawa, Govan) and friction models are implemented.

Constructor Summary

AbstractFilm(*inputSet, fluid*)

AbstractFilm Constructor for AbstractFilm class

Initializes `Inputs.InputSet` and `Inputs.FluidProperties`

Property Summary

DZ = 0

Axial step size [m]

fluid

`Inputs.FluidProperties` object containing fluid thermophysical properties

inputSet

`Inputs.InputSet` object containing geometry, model, and boundary conditions

mix = NaN

Solvers.Mixture.Mixture object of the mixture solver

Method Summary

CV(*absfilm*, *zIdx*)

CV Film/vapor interfacial friction factor [-]

Based on selected *Inputs.Model.VAPORFRIC* model.

CV_WALLISTHICK_CALC(*absfilm*, *thick*, *C*)

CV_WALLISTHICK_CALC Private method to calculate the interfacial shear factor [-]

Based on the *InputEnums.VAPORFRIC* = *WALLISTHICK* model.

CW(*absfilm*, *zIdx*)

CW Film wall friction factor [-]

Based on selected *Inputs.Model.THINFILMFRIC* model.

CW_LAM_CALC(*absfilm*, *zIdx*, *C*)

CW_LAM_CALC Private method to calculate the laminar wall friction factor [-]

CW_TURB_CALC(*absfilm*, *zIdx*, *C*)

CW_TURB_CALC Private method to calculate the turbulent wall friction factor [-]

ENTNUM(*absfilm*, *zIdx*)

ENTNUM Private method to calculate the entrainment number [-]

Based on Okawa model assumptions.

FBUOY(*absfilm*, *zIdx*)

FBUOY Film buoyancy force [N/m²]

FDEP(*absfilm*, *drop*, *zIdx*)

FDEP Drop deposition shear force [N/m²]

FGRAV(*absfilm*, *zIdx*)

FGRAV Gravitational force on film [N/m²]

FTOT(*absfilm*, *drop*, *zIdx*)

FTOT Total film forces per unit wall area [N/m²]

FVAPOR(*absfilm*, *zIdx*)

FVAPOR Vapor shear stress on film [N/m²]

FWALL(*absfilm*, *zIdx*)

FWALL Film wall shear stress [N/m²]

MENT(*absfilm*, *zIdx*)

MENT Film entrainment mass flux [kg/m²/s]

Based on selected *Inputs.Model.ENTRAINMENT* model.

MTOT(*absfilm*, *drop*, *zIdx*)

MTOT Total film mass transfer [kg/m²/s]

OKAWACOEFS(*absfilm*, *entnum*, *coefs*)

OKAWACOEFS Private method to determine the coefficients used in [InputEnums.ENTRAINMENT](#) = OKAWA entrainment models

Based on the calculated entrainment number

OKAWAMENT(*absfilm*, *zIdx*, *coefs*)

OKAWAMENT Private method to calculate the entrainment mass flux [kg/m²/s]

Based on Okawa models.

RE(*absfilm*, *zIdx*)

RE Film Reynolds number [-]

THICK(*absfilm*, *zIdx*)

THICK Film thickness [m]

UALGEBR(*absfilm*, *zIdx*)

UALGEBR Film velocity using simple algebraic model [m/s]

From Adamsson and Le Corre [2014] (eq. 21 or 23).

UEQUIL(*absfilm*, *drop*, *zIdx*)

UEQUIL Film velocity using full force equilibrium model [m/s]

UEQUILS(*absfilm*, *zIdx*)

UEQUILS Film velocity assuming equilibrium between wall and vapor shear [m/s]

UWALL(*absfilm*, *zIdx*)

UWALL Film wall friction velocity [m/s]

WL(*absfilm*, *zIdx*)

WL Film mass flow rate per unit perimeter [kg/s/m]

X(*absfilm*, *zIdx*)

X Vapor quality in the near-wall region [-]

Vapor quality in the near-wall region, considering only the film for the liquid phase. The near-wall region is defined in the mixture solver.

Inputs:

- drop — [Solvers.ThreeField.Drop](#) object
- zIdx — Axial indices to evaluate (optional)

YPLUS2THICK(*absfilm*, *yplus*, *zIdx*)

YPLUS2THICK Thickness based on wall unit value [m]

class Solvers.AbstractField

Bases: `matlab.mixin.Copyable`

ABSTRACTFIELD Base class for field definitions across solvers

This abstract class defines shared properties and methods for all field-type classes used in solver implementations. It provides mechanisms for transient data extraction, plotting, memoization, struct conversion, and property copying.

Constructor Summary

AbstractField()

ABSTRACTFIELD Constructor for AbstractField class

Property Summary**DT**

Time step size [s]

ITR

Iteration properties

NTIME

Number of time steps [-]

NZ

Number of axial steps [-]

TIDX

Time step index [-]

TIME

Time series [s]

Z

Elevation [m]

Method Summary**copyFlowProperties**(*srcObj, targetObj, opts*)

COPYFLOWPROPERTIES Copies flow properties from source to target field object
Supports full or partial copying depending on ‘opts.all’ flag.

memoizeFunction(*methodStr, methodHandle, varargin*)

MEMOIZEDMETHOD Registers and retrieves memoized function handles
Avoids redundant computation by caching results.
Adapted from <https://stackoverflow.com/a/75037451>

plotzt(*param, ylabelText, ylabelUnit, k, opt, subobj, annular, time*)

PLOTZT 2D Generates a 2D space-time distribution plot for a given parameter

struct()

STRUCT Converts field object to a structured array
Includes TIME, ITR, and flow properties.

transient(*param, subobj, opt*)

TRANSIENT Extracts transient distribution array for a given parameter
Supports wall-wise and axial/time slicing with optional subobject access.

class Solvers.AbstractSolver(*inputSet*)

Bases: handle

ABSTRACTSOLVER Defines the base interface and shared functionality for all solver classes.

Subclasses must implement the abstract methods and properties defined here.

Constructor Summary

AbstractSolver(*inputSet*)

ABSTRACTSOLVER Constructor for AbstractSolver

Initializes the solver with a given *Inputs.InputSet* and applies solver-specific properties.

Property Summary**STATE**

Solver state, defined by *Solvers.SolverState* enumeration

inputSet

Input configuration object of type *Inputs.InputSet*

Method Summary**static CreateITR**(*NZ, ITRFields*)

CreateITR Create and initialize the inner iteration storage structure

Inputs:

- *NZ* — Number of axial nodes (scalar)
- *ITRFields* — (string array) Names of fields to include in the struct

Output:

- *ITR* — Struct with fields initialized to zero vectors of length *NZ*

log(*solver, varargin*)

LOG Log messages to the session log

save(*solver, opts*)

SAVE Save the solver object to a .mat file with a customizable name.

Inputs:

- *solver* — The solver object to be saved.
- *opts.name* — (string) Name of the variable under which to save the solver. Default set to solver name
- *opts.showpath* — (logical) Whether to display the save path. Default: true.

solverName(*solver*)

SOLVERNAME Returns the name of the solver

class Solvers.AbstractPhase

Bases: handle

ABSTRACTPHASE Base class for phase definitions across solvers

This abstract class serves as a common interface for all phase-type classes used in solver implementations. It is intended to provide a foundation for shared behavior and structure.

EXTERNAL PACKAGES

4.1 CoolPropWrapper

The *CoolPropWrapper* package is designed to serve as a MATLAB interface to the CoolProp thermophysical property library, which is a Python-based library for calculating fluid properties.

class CoolPropWrapper.CoolPropWrapper(*fluidName*)

Bases: handle

CoolPropWrapper Summary of this class goes here Detailed explanation goes here

Constructor Summary

CoolPropWrapper(*fluidName*)

CoolPropWrapper Construct an instance of this class *fluidName* defaults to R245fa if not specified

Property Summary

CoolProp

ie. py.CoolProp.AbstactState(...)

CoolPropHandle

ie. py.CoolProp

Method Summary

conductivity(*Atype, A, Btype, B*)

conductivity calculates the conductivity TODO

cp(*Atype, A, Btype, B*)

cp returns the constant pressure specific heat TODO

cv(*Atype, A, Btype, B*)

cv returns the mass constant volume specific heat TODO

density(*Atype, A, Btype, B*)

density calculates the density TODO

enthalpy(*Atype, A, Btype, B*)

enthalpy calculates the enthalpy TODO

entropy(*Atype, A, Btype, B*)

entropy calculates the entropy TODO

static loadobj(*s*)

TODO: there are other ways of loading obj that does not guarantee *s* being obj already.

phase(*Atype, A, Btype, B*)

phase returns the phase TODO: convert enum to string

prandtl(*Atype, A, Btype, B*)

prandtl returns the Prandtl number TODO

pressure(*Atype, A, Btype, B*)

pressure calculates the pressure TODO

quality(*Atype, A, Btype, B*)

qualityPT calculates the quality given P and T TODO, doesn't necessarily work?

saveobj()

Save reference to CoolProp and CoolPropHandle

setFluid(*fluidName*)

setFluid Change fluid

setSpecifyPhase(*phase*)

setSpecifyPhase impose phase

surfaceTension(*Atype, A, Btype, B*)

surfaceTension returns the surfaceTension TODO

temperature(*Atype, A, Btype, B*)

temperature calculates the temperature TODO

viscosity(*Atype, A, Btype, B*)

viscosity calculates the viscosity TODO

class CoolPropWrapper.**CoolPropWrapperHA**

Bases: handle

CoolPropWrapperHA Summary of this class goes here Detailed explanation goes here

Constructor Summary

CoolPropWrapperHA()

CoolPropWrapperHA Construct an instance of this class Detailed explanation goes here

Property Summary

CoolPropHA

ie. py.CoolProp.AbstractState(...)

CoolPropHandle

ie. py.CoolProp

CoolPropWrapper.**sample**

Include CoolPropLib and subdirectories to the Path (MATLAB> Home> Environment> Set Path> Add with subfolders)

BIBLIOGRAPHY

- [1] Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. Openstream: an open-source platform for two-phase flow modeling and simulation. In *Proceedings of the 21st International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-21)*. Busan, Korea, August 31–September 5 2025a. URL: https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025OpenSTREAM.pdf.
- [2] Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. Openstream: a new open-source platform for two-phase flow model development. In *Proceedings of the 12th International Conference on Multiphase Flow (ICMF 2025)*. Toulouse, France, May 12–16 2025b. URL: https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025ICMF.pdf.

MATLAB MODULE INDEX

C

CoolPropWrapper, [117](#)

i

InputEnums, [7](#)

Inputs, [17](#)

S

Session, [33](#)

Solvers, [109](#)

Solvers.FourField, [39](#)

Solvers.FourField.FourFieldSolver, [36](#)

Solvers.Mixture, [57](#)

Solvers.Mixture.MixtureSolver, [53](#)

Solvers.ThreeField, [80](#)

Solvers.ThreeField.ThreeFieldSolver, [75](#)

Solvers.TwoFluid, [89](#)

Solvers.TwoFluid.TwoFluidSolver, [85](#)

A

AbstractField (class in Solvers), 113
 AbstractField() (Solvers.AbstractField method), 113
 AbstractFilm (class in Solvers), 111
 AbstractFilm() (Solvers.AbstractFilm method), 111
 AbstractPhase (class in Solvers), 115
 AbstractSolver (class in Solvers), 114
 AbstractSolver() (Solvers.AbstractSolver method), 114
 addTile() (Solvers.SolverPlotter method), 110
 ADIAM() (Inputs.Geometry method), 28
 AFDISTR() (Solvers.Mixture.Mixture method), 61
 AFDISTR() (Solvers.TwoFluid.Mixture method), 98
 AFFNC() (Solvers.Mixture.Mixture method), 61
 AI() (Solvers.ThreeField.Drop method), 80
 ALPHAF (Inputs.FluidProperties attribute), 30
 ALPHAG (Inputs.FluidProperties attribute), 30
 ALPHAL() (Inputs.FluidProperties method), 32
 ALPHAV() (Inputs.FluidProperties method), 32
 AMPLITUDE() (Solvers.FourField.Wave method), 48
 ANGLE (Inputs.Geometry attribute), 27
 ANGLE (Inputs.Model attribute), 19
 applySolverDependentProperties() (Inputs.InputSet method), 29
 applySolverDependentProps() (Inputs.InputSet method), 29
 AREA (Inputs.Geometry attribute), 27
 AREA() (Solvers.ThreeField.Drop method), 81
 AREA() (Solvers.TwoFluid.Liquid method), 101
 AREA() (Solvers.TwoFluid.Vapor method), 90
 AXIALINTERP (Inputs.Options attribute), 25
 axialInterpolate() (Solvers.Mixture.MixtureSolver.MixtureSolver method), 54

B

Base (class in Solvers.FourField), 42
 base (Solvers.FourField.Film attribute), 41
 Base() (Solvers.FourField.Base method), 42
 BASEEQTHICK (class in InputEnums), 9
 BASEEQTHICK (Inputs.Model attribute), 19
 BASEEQTHICKCOEF (Inputs.Model attribute), 19
 BASEYPLUS (Inputs.Model attribute), 19

bc (Inputs.InputSet attribute), 28
 BETA() (Solvers.FourField.Base method), 43
 BETA() (Solvers.FourField.Wave method), 48
 BETAP() (Solvers.FourField.Base method), 43
 BETAP() (Solvers.FourField.Wave method), 48
 BoundaryConditions (class in Inputs), 17
 boundaryConditions (Solvers.Mixture.MixtureSolver.MixtureSolver attribute), 54
 boundaryConditions (Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver attribute), 77
 boundaryConditions (Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute), 86
 BoundaryConditions() (Inputs.BoundaryConditions method), 17
 BTHTM (class in InputEnums), 14
 BTHTM (Inputs.Model attribute), 19
 BTMTM (class in InputEnums), 9
 BTMTM (Inputs.Model attribute), 19
 BUBBLEDRA (class in InputEnums), 11
 BUBBLEDRA (Inputs.Model attribute), 19
 BUBBLEDRA COEF (Inputs.Model attribute), 19

C

CC() (Solvers.Mixture.Liquid method), 74
 CC() (Solvers.Mixture.Vapor method), 58
 CBT (class in InputEnums), 11
 CBT (Inputs.Model attribute), 19
 cbt (Solvers.Mixture.Mixture attribute), 60
 CBT() (Solvers.Mixture.Mixture method), 61
 CBT() (Solvers.TwoFluid.Mixture method), 98
 CBTELEVATION (Inputs.Model attribute), 19
 CBTELEVATION (Solvers.Mixture.Mixture method), 61
 CBTKEFFECT (Inputs.Model attribute), 19
 CBTMULT (Inputs.Model attribute), 19
 CBTZ() (Solvers.Mixture.Mixture method), 62
 CD() (Solvers.TwoFluid.Liquid method), 101
 CD() (Solvers.TwoFluid.Vapor method), 90
 CHF() (Solvers.Mixture.Mixture method), 62
 CHF() (Solvers.TwoFluid.Mixture method), 98
 closeLog() (Session.Log method), 34
 CONC() (Solvers.ThreeField.Drop method), 81

- conductivity() (*CoolPropWrapper.CoolPropWrapper* method), 117
 convert2JSON() (*Inputs.Input* static method), 30
 CoolProp (*CoolPropWrapper.CoolPropWrapper* attribute), 117
 CoolPropHA (*CoolPropWrapper.CoolPropWrapperHA* attribute), 118
 CoolPropHandle (*CoolPropWrapper.CoolPropWrapper* attribute), 117
 CoolPropHandle (*CoolPropWrapper.CoolPropWrapperHA* attribute), 118
 CoolPropWrapper (class in *CoolPropWrapper*), 117
 CoolPropWrapper (module), 117
 CoolPropWrapper() (*CoolPropWrapper.CoolPropWrapper* method), 117
 CoolPropWrapperHA (class in *CoolPropWrapper*), 118
 CoolPropWrapperHA() (*CoolPropWrapper.CoolPropWrapperHA* method), 118
 copyFlowProperties() (*Solvers.AbstractField* method), 114
 copyFlowProperties() (*Solvers.FourField.Base* method), 46
 copyFlowProperties() (*Solvers.FourField.Film* method), 41
 cp() (*CoolPropWrapper.CoolPropWrapper* method), 117
 CPF (*Inputs.FluidProperties* attribute), 30
 CPG (*Inputs.FluidProperties* attribute), 31
 CPL() (*Inputs.FluidProperties* method), 32
 CPV() (*Inputs.FluidProperties* method), 32
 CreateITR() (*Solvers.AbstractSolver* static method), 115
 CSTWAVEFREQ (*Inputs.Model* attribute), 19
 currentAhIdx (*Solvers.SolverPlotter* attribute), 110
 cv() (*CoolPropWrapper.CoolPropWrapper* method), 117
 CV() (*Solvers.AbstractFilm* method), 112
 CV_WALLLISTHICK_CALC() (*Solvers.AbstractFilm* method), 112
 CW() (*Solvers.AbstractFilm* method), 112
 CW_LAM_CALC() (*Solvers.AbstractFilm* method), 112
 CW_TURB_CALC() (*Solvers.AbstractFilm* method), 112
- ## D
- defaultValueUsedReport() (*Inputs.Input* method), 30
 delete() (*Session.Log* method), 34
 DELTAFREQ() (*Solvers.FourField.Wave* method), 48
 density() (*CoolPropWrapper.CoolPropWrapper* method), 117
 DENSITY() (*Solvers.ThreeField.Drop* method), 81
 DEPENDANCEMENT (class in *InputEnums*), 7
 DEPENDANCEMENT (*Inputs.Model* attribute), 19
 DEPOSITION (class in *InputEnums*), 11
 DEPOSITION (*Inputs.Model* attribute), 19
 DHIN() (*Inputs.BoundaryConditions* method), 18
 DIAM() (*Solvers.ThreeField.Drop* method), 81
 diaryFilePath (*Session.Log* attribute), 34
 diaryIsOn (*Session.Log* attribute), 34
 diaryOff() (*Session.Log* method), 34
 diaryOn() (*Session.Log* method), 34
 directory (*Session.Session* attribute), 35
 dirName (*Session.Session* attribute), 35
 distributeOAFW() (*Solvers.FourField.Film* method), 41
 DITTUSBOELTERCOEF (*Inputs.Model* attribute), 19
 DP (*Solvers.Mixture.Mixture* attribute), 59
 DPACCS() (*Solvers.Mixture.Mixture* method), 62
 DPACCT() (*Solvers.Mixture.Mixture* method), 62
 DPACZ() (*Solvers.Mixture.Mixture* method), 62
 DPDZ() (*Solvers.TwoFluid.Mixture* method), 98
 DPGRAY() (*Solvers.Mixture.Mixture* method), 62
 DPK() (*Solvers.Mixture.Mixture* method), 62
 DPPARTS() (*Solvers.Mixture.Mixture* method), 63
 DPSUM (*Solvers.Mixture.Mixture* attribute), 60
 DPTOT() (*Solvers.Mixture.Mixture* method), 63
 DPWALL() (*Solvers.Mixture.Mixture* method), 63
 DRAG() (*Solvers.ThreeField.Drop* method), 81
 DRAGCOEF() (*Solvers.FourField.Wave* method), 48
 Drop (class in *Solvers.FourField*), 39
 Drop (class in *Solvers.ThreeField*), 80
 drop (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* attribute), 77
 Drop() (*Solvers.ThreeField.Drop* method), 80
 DROPDIAM (*Inputs.Model* attribute), 19
 DROPDRAG (class in *InputEnums*), 10
 DROPDRAG (*Inputs.Model* attribute), 20
 DROPDRAGCOEF (*Inputs.Model* attribute), 20
 dropInit (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* attribute), 77
 DROPSLIP (*Inputs.Model* attribute), 20
 DT (*Solvers.AbstractField* attribute), 114
 DT (*Solvers.FourField.Base* attribute), 42
 DT (*Solvers.FourField.Film* attribute), 40
 DT (*Solvers.FourField.Wave* attribute), 47
 DT (*Solvers.Mixture.Mixture* attribute), 60
 DT (*Solvers.Mixture.MixtureSolver.MixtureSolver* attribute), 54
 DT (*Solvers.ThreeField.Drop* attribute), 80
 DT (*Solvers.ThreeField.Film* attribute), 84
 DT (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* attribute), 76
 DT (*Solvers.TwoFluid.Liquid* attribute), 100
 DT (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* attribute), 86
 DT (*Solvers.TwoFluid.Vapor* attribute), 89
 DTIN() (*Inputs.BoundaryConditions* method), 18
 DTMFB (*Inputs.Model* attribute), 20
 DZ (*Solvers.AbstractFilm* attribute), 111

- DZ (*Solvers.Mixture.Mixture attribute*), 60
 DZ (*Solvers.Mixture.MixtureSolver.MixtureSolver attribute*), 54
 DZ (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver attribute*), 76
 DZ (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute*), 86
- ## E
- ECT() (*Solvers.Mixture.Mixture method*), 63
 enthalpy() (*CoolPropWrapper.CoolPropWrapper method*), 117
 ENTNUM() (*Solvers.AbstractFilm method*), 112
 ENTRAINMENT (*class in InputEnums*), 14
 ENTRAINMENT (*Inputs.Model attribute*), 20
 entropy() (*CoolPropWrapper.CoolPropWrapper method*), 117
 EPSILON() (*Solvers.FourField.Base method*), 43
 EPSILON() (*Solvers.FourField.Wave method*), 48
 EQFREQUENCY() (*Solvers.FourField.Wave method*), 48
 EQPERIOD() (*Solvers.FourField.Wave method*), 49
 EQSTROUHAL (*class in InputEnums*), 13
 EQSTROUHAL (*Inputs.Model attribute*), 20
 EQSTROUHAL() (*Solvers.FourField.Wave method*), 49
 EQSTROUHALCOEF (*Inputs.Model attribute*), 20
 EQTHICK() (*Solvers.FourField.Base method*), 43
 EQUIL() (*Solvers.FourField.Film method*), 41
 EQUIL() (*Solvers.FourField.FourFieldSolver.FourFieldSolver method*), 37
 EQUIL() (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver method*), 77
 ERRORFW (*Inputs.Options attribute*), 25
 ERRORH (*Inputs.Options attribute*), 25
 ERRORP (*Inputs.Options attribute*), 25
 ERRORU (*Inputs.Options attribute*), 25
 ERRORUD (*Inputs.Options attribute*), 25
 ERRORUF (*Inputs.Options attribute*), 25
 ERRORW (*Inputs.Options attribute*), 25
 ERRORWF (*Inputs.Options attribute*), 25
 ETA() (*Solvers.FourField.Base method*), 43
 ETA() (*Solvers.FourField.Wave method*), 49
 extra (*Inputs.Input attribute*), 29
- ## F
- FBASE() (*Solvers.FourField.Wave method*), 49
 FBASEMASS() (*Solvers.FourField.Wave method*), 49
 FBASEVAPOR() (*Solvers.FourField.Base method*), 43
 FBUOY() (*Solvers.AbstractFilm method*), 112
 FBUOY() (*Solvers.ThreeField.Drop method*), 81
 FBUOY() (*Solvers.TwoFluid.Liquid method*), 101
 FBUOY() (*Solvers.TwoFluid.Vapor method*), 90
 FDEP() (*Solvers.AbstractFilm method*), 112
 FDEP() (*Solvers.FourField.Base method*), 44
 FDEP() (*Solvers.FourField.Wave method*), 49
 FDRAG() (*Solvers.FourField.Wave method*), 50
 FDRAG() (*Solvers.ThreeField.Drop method*), 81
 FDRAG() (*Solvers.TwoFluid.Liquid method*), 101
 FDRAG() (*Solvers.TwoFluid.Vapor method*), 90
 FDRY() (*Solvers.FourField.Base method*), 44
 FENT() (*Solvers.ThreeField.Drop method*), 82
 FGRAV() (*Solvers.AbstractFilm method*), 112
 FGRAV() (*Solvers.ThreeField.Drop method*), 82
 FGRAV() (*Solvers.TwoFluid.Liquid method*), 102
 FGRAV() (*Solvers.TwoFluid.Vapor method*), 90
 fieldName2plotStyle() (*Solvers.SolverPlotter static method*), 110
 Film (*class in Solvers.FourField*), 39
 Film (*class in Solvers.ThreeField*), 84
 film (*Solvers.FourField.Wave attribute*), 47
 film (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver attribute*), 77
 Film() (*Solvers.FourField.Film method*), 40
 filmInit (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver attribute*), 77
 FINTCOND() (*Solvers.TwoFluid.Liquid method*), 102
 FINTEVAP() (*Solvers.TwoFluid.Vapor method*), 90
 FLOWID() (*Solvers.TwoFluid.Liquid method*), 102
 FLOWREGIME() (*Solvers.TwoFluid.Liquid method*), 102
 FLOWREGIME() (*Solvers.TwoFluid.Vapor method*), 91
 FLUID (*Inputs.FluidProperties attribute*), 31
 FLUID (*Inputs.Model attribute*), 20
 fluid (*Solvers.AbstractFilm attribute*), 111
 fluid (*Solvers.Mixture.Mixture attribute*), 60
 fluid (*Solvers.Mixture.MixtureSolver.MixtureSolver attribute*), 54
 fluid (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver attribute*), 77
 fluid (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute*), 86
 FLUIDPROPERTIES (*class in InputEnums*), 10
 FluidProperties (*class in Inputs*), 30
 FluidProperties() (*Inputs.FluidProperties method*), 30
 FontSize (*Solvers.SolverPlotter attribute*), 110
 FourFieldSolver (*class in Solvers.FourField.FourFieldSolver*), 36
 FourFieldSolver() (*Solvers.FourField.FourFieldSolver.FourFieldSolver method*), 37
 FOURIERCOND() (*Solvers.Mixture.Mixture method*), 63
 FOURIEREVAP() (*Solvers.Mixture.Mixture method*), 64
 FREQUENCY (*Solvers.FourField.Wave attribute*), 47
 FRICTION (*Inputs.Model attribute*), 20
 FSHEAR() (*Solvers.FourField.Wave method*), 50
 FTOT() (*Solvers.AbstractFilm method*), 112
 FTOT() (*Solvers.FourField.Base method*), 44
 FTOT() (*Solvers.FourField.Wave method*), 50
 FTOT() (*Solvers.ThreeField.Drop method*), 82
 FTOT() (*Solvers.TwoFluid.Liquid method*), 102

FTOT() (*Solvers.TwoFluid.Vapor method*), 91
 FVAPOR() (*Solvers.AbstractFilm method*), 112
 FVAPOR() (*Solvers.FourField.Wave method*), 50
 FW() (*Solvers.Mixture.Mixture method*), 64
 FW() (*Solvers.Mixture.Vapor method*), 58
 FWALEVAP() (*Solvers.TwoFluid.Vapor method*), 91
 FWALL() (*Solvers.AbstractFilm method*), 112
 FWALL() (*Solvers.FourField.Wave method*), 50
 FWALL() (*Solvers.TwoFluid.Liquid method*), 102
 FWALL() (*Solvers.TwoFluid.Vapor method*), 91
 FWAVE() (*Solvers.FourField.Base method*), 44
 FWAVEMASS() (*Solvers.FourField.Base method*), 44
 FWL() (*Solvers.Mixture.Mixture method*), 64

G

G (*Inputs.Model attribute*), 20
 gca() (*Solvers.SolverPlotter method*), 110
 gcf() (*Solvers.SolverPlotter method*), 110
 Geometry (*class in Inputs*), 27
 geometry (*Inputs.InputSet attribute*), 29
 Geometry() (*Inputs.Geometry method*), 27
 Grid (*Solvers.SolverPlotter attribute*), 110

H

H (*Solvers.FourField.Base attribute*), 42
 H (*Solvers.FourField.Film attribute*), 40
 H (*Solvers.FourField.Wave attribute*), 47
 H (*Solvers.Mixture.Mixture attribute*), 60
 H (*Solvers.ThreeField.Drop attribute*), 80
 H (*Solvers.ThreeField.Film attribute*), 84
 H (*Solvers.TwoFluid.Liquid attribute*), 100
 H (*Solvers.TwoFluid.Vapor attribute*), 89
 HC() (*Inputs.FluidProperties method*), 32
 HC() (*Solvers.Mixture.Liquid method*), 74
 HC() (*Solvers.Mixture.Vapor method*), 58
 HC() (*Solvers.TwoFluid.Mixture method*), 98
 HDIAM() (*Inputs.Geometry method*), 28
 HF (*Inputs.FluidProperties attribute*), 31
 HF() (*Inputs.BoundaryConditions method*), 18
 HFG (*Inputs.FluidProperties attribute*), 31
 HFLUX (*Solvers.FourField.Base attribute*), 42
 HFLUX (*Solvers.FourField.Film attribute*), 40
 HFLUX (*Solvers.FourField.Wave attribute*), 47
 HFLUX (*Solvers.Mixture.Mixture attribute*), 60
 HFLUX (*Solvers.ThreeField.Film attribute*), 84
 HFLUX() (*Solvers.Mixture.Liquid method*), 74
 HFLUX() (*Solvers.Mixture.Vapor method*), 58
 HFLUX() (*Solvers.TwoFluid.Liquid method*), 103
 HFLUX() (*Solvers.TwoFluid.Mixture method*), 98
 HFLUX() (*Solvers.TwoFluid.Vapor method*), 91
 HFLUXWALEVAP() (*Solvers.Mixture.Vapor method*), 58
 HFLUXWALEVAP() (*Solvers.TwoFluid.Liquid method*), 103
 HFLUXWALEVAP() (*Solvers.TwoFluid.Vapor method*), 92

HG (*Inputs.FluidProperties attribute*), 31
 HG() (*Inputs.BoundaryConditions method*), 18
 HIN (*Inputs.BoundaryConditions attribute*), 17
 HINT() (*Solvers.Mixture.Mixture method*), 64
 HINT() (*Solvers.TwoFluid.Liquid method*), 103
 HINT() (*Solvers.TwoFluid.Vapor method*), 92
 HINTCOND() (*Solvers.Mixture.Mixture method*), 65
 HINTCOND() (*Solvers.TwoFluid.Liquid method*), 103
 HINTCOND() (*Solvers.TwoFluid.Vapor method*), 92
 HINTEVAP() (*Solvers.Mixture.Mixture method*), 65
 HINTEVAP() (*Solvers.TwoFluid.Liquid method*), 103
 HINTEVAP() (*Solvers.TwoFluid.Vapor method*), 92
 HM() (*Solvers.Mixture.Mixture method*), 65
 HTOT() (*Solvers.Mixture.Mixture method*), 65
 HTOT() (*Solvers.TwoFluid.Liquid method*), 104
 HTOT() (*Solvers.TwoFluid.Vapor method*), 92
 HVTOT() (*Solvers.Mixture.Mixture method*), 65
 HWALEVAP() (*Solvers.Mixture.Mixture method*), 66
 HWALEVAP() (*Solvers.TwoFluid.Liquid method*), 104
 HWALEVAP() (*Solvers.TwoFluid.Vapor method*), 92
 HWALHEAT() (*Solvers.Mixture.Mixture method*), 66
 HWALHEAT() (*Solvers.TwoFluid.Liquid method*), 104
 HWALHEAT() (*Solvers.TwoFluid.Vapor method*), 93
 HWALL() (*Solvers.Mixture.Liquid method*), 74
 HWALL() (*Solvers.Mixture.Mixture method*), 66
 HWALL() (*Solvers.Mixture.Vapor method*), 58
 HWALLBOIL() (*Solvers.Mixture.Mixture method*), 66
 HWALLPOSTCBT() (*Solvers.Mixture.Mixture method*), 66

I

ID (*Inputs.Geometry attribute*), 27
 ID (*Inputs.Model attribute*), 20
 ID (*Inputs.Options attribute*), 25
 initializeBaseAndWave() (*Solvers.FourField.Film method*), 41
 initializeSolver() (*Solvers.FourField.FourFieldSolver.FourFieldSolver method*), 37
 initializeSolver() (*Solvers.Mixture.MixtureSolver.MixtureSolver method*), 54
 initializeSolver() (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver method*), 77
 initializeSolver() (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver method*), 87
 Input (*class in Inputs*), 29
 Input() (*Inputs.Input method*), 29
 InputEnums (*module*), 7
 Inputs (*module*), 17
 InputSet (*class in Inputs*), 28
 inputSet (*Solvers.AbstractFilm attribute*), 111
 inputSet (*Solvers.AbstractSolver attribute*), 115
 inputSet (*Solvers.Mixture.Mixture attribute*), 60
 inputSet (*Solvers.Mixture.MixtureSolver.MixtureSolver attribute*), 54

- inputSet (Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver attribute), 77
- inputSet (Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute), 86
- InputSet() (Inputs.InputSet method), 28
- INTAREA (class in InputEnums), 16
- INTAREA (Inputs.Model attribute), 20
- INTAREA() (Solvers.TwoFluid.Liquid method), 104
- INTAREA() (Solvers.TwoFluid.Vapor method), 93
- interpBoundaryConditions() (Solvers.Mixture.MixtureSolver.MixtureSolver method), 55
- INTHFLUX() (Solvers.TwoFluid.Liquid method), 104
- INTHFLUX() (Solvers.TwoFluid.Vapor method), 93
- INTLENGTH (class in InputEnums), 14
- INTLENGTH (Inputs.Model attribute), 20
- INTLENGTHLCST (Inputs.Model attribute), 20
- INTLENGTHVCST (Inputs.Model attribute), 20
- INTNU (class in InputEnums), 8
- INTNU (Inputs.Model attribute), 20
- INTNU() (Solvers.TwoFluid.Liquid method), 105
- INTNU() (Solvers.TwoFluid.Vapor method), 93
- INTNULCST (Inputs.Model attribute), 20
- INTNUVCST (Inputs.Model attribute), 20
- INTTRANSH (class in InputEnums), 16
- INTTRANSH (Inputs.Model attribute), 20
- isLegalPath() (in module Session), 34
- ITR (Solvers.AbstractField attribute), 114
- ITR (Solvers.FourField.Base attribute), 42
- ITR (Solvers.FourField.Film attribute), 40
- ITR (Solvers.FourField.Wave attribute), 47
- ITR (Solvers.Mixture.Mixture attribute), 60
- ITR (Solvers.ThreeField.Drop attribute), 80
- ITR (Solvers.ThreeField.Film attribute), 84
- ITR (Solvers.TwoFluid.Liquid attribute), 100
- ITR (Solvers.TwoFluid.Vapor attribute), 89
- ## J
- J() (Solvers.TwoFluid.Liquid method), 105
- J() (Solvers.TwoFluid.Vapor method), 93
- JG() (Solvers.Mixture.Mixture method), 67
- JL() (Solvers.Mixture.Mixture method), 67
- ## K
- KBLOCKRATIO (Inputs.Model attribute), 21
- KDIST() (Solvers.Mixture.Mixture method), 67
- KENH() (Solvers.ThreeField.Drop method), 82
- KF (Inputs.FluidProperties attribute), 31
- KG (Inputs.FluidProperties attribute), 31
- KIDX() (Solvers.Mixture.Mixture method), 67
- KL() (Inputs.FluidProperties method), 32
- KLOC (Inputs.Model attribute), 21
- KLOCZ() (Solvers.Mixture.Mixture method), 67
- KLOSS (Inputs.Model attribute), 21
- KLOSS() (Solvers.Mixture.Mixture method), 67
- KTRELAX (Inputs.Model attribute), 21
- KTRELAX() (Solvers.Mixture.Mixture method), 68
- KTUNING (Inputs.Model attribute), 21
- KV() (Inputs.FluidProperties method), 32
- ## L
- L() (Solvers.TwoFluid.Liquid method), 105
- L() (Solvers.TwoFluid.Vapor method), 94
- legend() (Solvers.SolverPlotter method), 110
- LENGTH (Inputs.Geometry attribute), 27
- LHGR() (Solvers.Mixture.Mixture method), 68
- LHGR() (Solvers.TwoFluid.Mixture method), 98
- Liquid (class in Solvers.Mixture), 73
- Liquid (class in Solvers.TwoFluid), 100
- liquid (Solvers.Mixture.Mixture attribute), 61
- liquid (Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute), 86
- Liquid() (Solvers.Mixture.Liquid method), 74
- Liquid() (Solvers.TwoFluid.Liquid method), 100
- liquidInit (Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute), 86
- listInputProperties() (Inputs.Input method), 30
- loadobj() (CoolPropWrapper.CoolPropWrapper static method), 117
- LOCLEVEL (class in InputEnums), 7
- LOCLEVEL (Inputs.Model attribute), 21
- Log (class in Session), 33
- log (Session.Session attribute), 35
- Log() (Session.Log method), 33
- log() (Session.Log method), 34
- log() (Solvers.AbstractSolver method), 115
- LOGFID (Session.Log attribute), 33
- logFileName (Session.Log attribute), 34
- logFilePath (Session.Log attribute), 34
- LogMode (class in Session), 35
- LOGMODE (Session.Log attribute), 33
- ## M
- makeSessionDirectory() (Session.Session method), 35
- MAXITER (Inputs.Options attribute), 25
- MBASE() (Solvers.FourField.Wave method), 50
- MCT() (Solvers.Mixture.Mixture method), 68
- MDEP() (Solvers.FourField.Base method), 44
- MDEP() (Solvers.FourField.Wave method), 50
- MDEP() (Solvers.ThreeField.Drop method), 82
- MDER (Solvers.Mixture.Mixture attribute), 60
- memoizeFunction() (Solvers.AbstractField method), 114
- MENT() (Solvers.AbstractFilm method), 112
- MENT() (Solvers.FourField.Base method), 45
- MENT() (Solvers.FourField.Wave method), 51
- MEVAP (Solvers.FourField.Film attribute), 40

- MEVAP (*Solvers.ThreeField.Film* attribute), 84
 MEVAP() (*Solvers.FourField.Base* method), 45
 MEVAP() (*Solvers.FourField.Wave* method), 51
 MFBT (*class in InputEnums*), 9
 MFBT (*Inputs.Model* attribute), 21
 mfbt (*Solvers.Mixture.Mixture* attribute), 61
 MFBT() (*Solvers.Mixture.Mixture* method), 68
 MFLOW (*Inputs.BoundaryConditions* attribute), 17
 MFLUX() (*Solvers.Mixture.Liquid* method), 74
 MFLUX() (*Solvers.Mixture.Mixture* method), 68
 MFLUX() (*Solvers.Mixture.Vapor* method), 58
 MFLUX() (*Solvers.TwoFluid.Mixture* method), 98
 MINT() (*Solvers.Mixture.Mixture* method), 68
 MINT() (*Solvers.TwoFluid.Liquid* method), 105
 MINT() (*Solvers.TwoFluid.Vapor* method), 94
 MINTCOND() (*Solvers.Mixture.Mixture* method), 69
 MINTCOND() (*Solvers.TwoFluid.Liquid* method), 105
 MINTCOND() (*Solvers.TwoFluid.Vapor* method), 94
 MINTEVAP() (*Solvers.Mixture.Mixture* method), 69
 MINTEVAP() (*Solvers.TwoFluid.Liquid* method), 106
 MINTEVAP() (*Solvers.TwoFluid.Vapor* method), 94
 mix (*Solvers.AbstractFilm* attribute), 111
 mix (*Solvers.ThreeField.Drop* attribute), 80
 mix (*Solvers.TwoFluid.Liquid* attribute), 101
 mix (*Solvers.TwoFluid.Vapor* attribute), 89
 mixSolver (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* attribute), 77
 mixSolver (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* attribute), 86
 Mixture (*class in Solvers.Mixture*), 59
 Mixture (*class in Solvers.TwoFluid*), 97
 mixture (*Solvers.Mixture.MixtureSolver.MixtureSolver* attribute), 54
 Mixture() (*Solvers.Mixture.Mixture* method), 59
 Mixture() (*Solvers.TwoFluid.Mixture* method), 97
 mixtureInit (*Solvers.Mixture.MixtureSolver.MixtureSolver* attribute), 54
 MixtureSolver (*class in Solvers.Mixture.MixtureSolver*), 53
 MixtureSolver() (*Solvers.Mixture.MixtureSolver.MixtureSolver* method), 53
 Model (*class in Inputs*), 18
 model (*Inputs.InputSet* attribute), 29
 Model() (*Inputs.Model* method), 18
 MOECKCOEF (*Inputs.Model* attribute), 21
 MOMENTBASE (*class in InputEnums*), 14
 MOMENTBASE (*Inputs.Model* attribute), 21
 MOMENTDROP (*class in InputEnums*), 11
 MOMENTDROP (*Inputs.Model* attribute), 21
 MOMENTFILM (*class in InputEnums*), 13
 MOMENTFILM (*Inputs.Model* attribute), 21
 MOMENTGAS (*class in InputEnums*), 10
 MOMENTGAS (*Inputs.Model* attribute), 21
 MOMENTLIQUID (*class in InputEnums*), 12
 MOMENTLIQUID (*Inputs.Model* attribute), 21
 MOMENTWAVE (*class in InputEnums*), 12
 MOMENTWAVE (*Inputs.Model* attribute), 21
 MRM (*Solvers.Mixture.Mixture* attribute), 60
 MRTIMEINT (*class in InputEnums*), 16
 MRTIMEINT (*Inputs.Options* attribute), 25
 MTOT() (*Solvers.AbstractFilm* method), 112
 MTOT() (*Solvers.FourField.Base* method), 45
 MTOT() (*Solvers.FourField.Wave* method), 51
 MTOT() (*Solvers.Mixture.Mixture* method), 69
 MTOT() (*Solvers.TwoFluid.Liquid* method), 106
 MTOT() (*Solvers.TwoFluid.Vapor* method), 94
 MTRANSV() (*Solvers.Mixture.Mixture* method), 69
 MTURB() (*Solvers.FourField.Base* method), 45
 MTURB() (*Solvers.FourField.Wave* method), 51
 MU() (*Solvers.Mixture.Mixture* method), 69
 MUF (*Inputs.FluidProperties* attribute), 31
 MUG (*Inputs.FluidProperties* attribute), 31
 MUL() (*Inputs.FluidProperties* method), 32
 MUV() (*Inputs.FluidProperties* method), 32
 MWALEVAP() (*Solvers.Mixture.Mixture* method), 69
 MWALEVAP() (*Solvers.TwoFluid.Liquid* method), 106
 MWALEVAP() (*Solvers.TwoFluid.Vapor* method), 95
 MWAVE() (*Solvers.FourField.Base* method), 45
- ## N
- N (*class in Solvers*), 51
 N() (*Solvers.FourField.Wave* method), 51
 Name (*Session.Session* attribute), 35
 NEARWALL (*Solvers.Mixture.Mixture* attribute), 60
 NEARWALLEQOAF (*Inputs.Model* attribute), 21
 NEARWALLH (*class in InputEnums*), 8
 NEARWALLH (*Inputs.Model* attribute), 21
 NEARWALLHRATIO (*Inputs.Model* attribute), 21
 NEARWALLRATIO (*Inputs.Model* attribute), 21
 NEARWALLRELAX (*class in InputEnums*), 12
 NEARWALLRELAX (*Inputs.Model* attribute), 21
 NEARWALLRELAXCOEF (*Inputs.Model* attribute), 22
 NEARWALLRELAXT (*Inputs.Model* attribute), 22
 NEARWALLRELAXX (*Inputs.Model* attribute), 22
 newTile() (*Solvers.SolverPlotter* method), 110
 NNODES (*Inputs.Model* attribute), 22
 NTIME (*Solvers.AbstractField* attribute), 114
 NTIME (*Solvers.FourField.Base* attribute), 42
 NTIME (*Solvers.FourField.Film* attribute), 40
 NTIME (*Solvers.FourField.Wave* attribute), 47
 NTIME (*Solvers.Mixture.Mixture* attribute), 60
 NTIME (*Solvers.Mixture.MixtureSolver.MixtureSolver* attribute), 54
 NTIME (*Solvers.ThreeField.Drop* attribute), 80
 NTIME (*Solvers.ThreeField.Film* attribute), 84
 NTIME (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* attribute), 76
 NTIME (*Solvers.TwoFluid.Liquid* attribute), 100

- NTIME (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* attribute), 86
 NTIME (*Solvers.TwoFluid.Vapor* attribute), 89
 NU() (*Solvers.Mixture.Liquid* method), 74
 NU() (*Solvers.Mixture.Vapor* method), 58
 NWALL() (*Inputs.Geometry* method), 28
 NZ (*Solvers.AbstractField* attribute), 114
 NZ (*Solvers.FourField.Base* attribute), 42
 NZ (*Solvers.FourField.Film* attribute), 40
 NZ (*Solvers.FourField.Wave* attribute), 47
 NZ (*Solvers.Mixture.Mixture* attribute), 60
 NZ (*Solvers.Mixture.MixtureSolver.MixtureSolver* attribute), 54
 NZ (*Solvers.ThreeField.Drop* attribute), 80
 NZ (*Solvers.ThreeField.Film* attribute), 84
 NZ (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* attribute), 76
 NZ (*Solvers.TwoFluid.Liquid* attribute), 100
 NZ (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* attribute), 86
 NZ (*Solvers.TwoFluid.Vapor* attribute), 89
- ## O
- OAF (class in *InputEnums*), 9
 OAF (*Inputs.Model* attribute), 22
 OAFBASERATIO (*Inputs.Model* attribute), 22
 OAFDROPRATIO (*Inputs.Model* attribute), 22
 OAFENTRAINED (class in *InputEnums*), 9
 OAFENTRAINED (*Inputs.Model* attribute), 22
 OAFFILMSPLIT (class in *InputEnums*), 10
 OAFFILMSPLIT (*Inputs.Model* attribute), 22
 OAFIDX() (*Solvers.Mixture.Mixture* method), 70
 OAFTRANSITION (*Inputs.Model* attribute), 22
 OAFWL() (*Solvers.Mixture.Mixture* method), 70
 OAFX() (*Solvers.Mixture.Mixture* method), 70
 OAFX() (*Solvers.TwoFluid.Mixture* method), 98
 OAFZ() (*Solvers.Mixture.Mixture* method), 70
 OKAWACOEFS (*Inputs.Model* attribute), 22
 OKAWACOEFS() (*Solvers.AbstractFilm* method), 112
 OKAWAMENT() (*Solvers.AbstractFilm* method), 113
 openLog() (*Session.Log* method), 34
 Options (class in *Inputs*), 24
 options (*Inputs.InputSet* attribute), 29
 Options() (*Inputs.Options* method), 24
 overwriteFiles (*Session.Session* attribute), 35
- ## P
- P (*Solvers.Mixture.Mixture* attribute), 60
 parentDir (*Session.Session* attribute), 35
 PCRT (*Inputs.FluidProperties* attribute), 31
 PERIM (*Inputs.Geometry* attribute), 28
 PERIOD() (*Solvers.FourField.Wave* method), 51
 phase() (*CoolPropWrapper.CoolPropWrapper* method), 117
- PHI2F() (*Solvers.Mixture.Mixture* method), 70
 PHI2K() (*Solvers.Mixture.Mixture* method), 70
 plot() (*Inputs.BoundaryConditions* method), 18
 plot() (*Inputs.FluidProperties* method), 33
 plotK() (*Solvers.SolverPlotter* method), 110
 plotOAF() (*Solvers.SolverPlotter* method), 111
 plott() (*Solvers.FourField.FourFieldSolver.FourFieldSolver* method), 38
 plott() (*Solvers.Mixture.MixtureSolver.MixtureSolver* method), 55
 plott() (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* method), 78
 plott() (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* method), 87
 plotz() (*Solvers.FourField.FourFieldSolver.FourFieldSolver* method), 38
 plotz() (*Solvers.Mixture.MixtureSolver.MixtureSolver* method), 56
 plotz() (*Solvers.SolverPlotter* method), 111
 plotz() (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* method), 78
 plotz() (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* method), 87
 plotzt() (*Solvers.AbstractField* method), 114
 plotzt() (*Solvers.FourField.FourFieldSolver.FourFieldSolver* method), 38
 plotzt() (*Solvers.Mixture.MixtureSolver.MixtureSolver* method), 56
 plotzt() (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* method), 79
 plotzt() (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* method), 88
 POSFILM (*Inputs.Model* attribute), 22
 POWER (*Inputs.BoundaryConditions* attribute), 17
 prandtl() (*CoolPropWrapper.CoolPropWrapper* method), 118
 PRANDTLF (*Inputs.FluidProperties* attribute), 31
 PRANDTLG (*Inputs.FluidProperties* attribute), 31
 PRANDTLL() (*Inputs.FluidProperties* method), 32
 PRANDTLV() (*Inputs.FluidProperties* method), 32
 PRESSURE (*Inputs.BoundaryConditions* attribute), 17
 PRESSURE (*Inputs.FluidProperties* attribute), 31
 pressure() (*CoolPropWrapper.CoolPropWrapper* method), 118
 PROPERTIES (*Inputs.FluidProperties* attribute), 31
 PROPERTIES (*Inputs.Model* attribute), 22
 PTRIPLE (*Inputs.FluidProperties* attribute), 31
- ## Q
- QC() (*Solvers.Mixture.Liquid* method), 74
 QC() (*Solvers.Mixture.Vapor* method), 59
 QK (*Inputs.FluidProperties* attribute), 31
 quality() (*CoolPropWrapper.CoolPropWrapper* method), 118

R

RANZMARSHALLCST (*Inputs.Model* attribute), 22
 RANZMARSHALLVCST (*Inputs.Model* attribute), 22
 RE() (*Solvers.AbstractFilm* method), 113
 RE() (*Solvers.Mixture.Liquid* method), 75
 RE() (*Solvers.Mixture.Mixture* method), 71
 RE() (*Solvers.Mixture.Vapor* method), 59
 RE() (*Solvers.ThreeField.Drop* method), 82
 RE() (*Solvers.TwoFluid.Liquid* method), 106
 RE() (*Solvers.TwoFluid.Vapor* method), 95
 REGIMES (*class in Solvers.TwoFluid*), 108
 REL() (*Solvers.Mixture.Mixture* method), 71
 REL() (*Solvers.TwoFluid.Vapor* method), 95
 RELAXCONDCOEFF (*Inputs.Model* attribute), 22
 RELAXCONDFO (*Inputs.Model* attribute), 22
 RELAXCONDFOCOEF (*Inputs.Model* attribute), 22
 RELAXCONDT (*Inputs.Model* attribute), 22
 RELAXEVAPCOEF (*Inputs.Model* attribute), 23
 RELAXEVAPFO (*Inputs.Model* attribute), 23
 RELAXEVAPFOCOEF (*Inputs.Model* attribute), 23
 RELAXEVAPT (*Inputs.Model* attribute), 23
 RELAXFW (*Inputs.Options* attribute), 25
 RELAXHL (*Inputs.Options* attribute), 25
 RELAXHM (*Inputs.Options* attribute), 25
 RELAXHV (*Inputs.Options* attribute), 25
 RELAXPM (*Inputs.Options* attribute), 25
 RELAXT() (*Solvers.Mixture.Mixture* method), 71
 RELAXTB (*Inputs.Model* attribute), 23
 RELAXTCOND() (*Solvers.Mixture.Mixture* method), 71
 RELAXTCOND() (*Solvers.TwoFluid.Mixture* method), 99
 RELAXTEVAP() (*Solvers.Mixture.Mixture* method), 71
 RELAXTEVAP() (*Solvers.TwoFluid.Mixture* method), 99
 RELAXTW (*Inputs.Model* attribute), 23
 RELAXUB (*Inputs.Options* attribute), 26
 RELAXUD (*Inputs.Options* attribute), 26
 RELAXUF (*Inputs.Options* attribute), 26
 RELAXUL (*Inputs.Options* attribute), 26
 RELAXUV (*Inputs.Options* attribute), 26
 RELAXUW (*Inputs.Options* attribute), 26
 RELAXWB (*Inputs.Options* attribute), 26
 RELAXWF (*Inputs.Options* attribute), 26
 RELAXWL (*Inputs.Options* attribute), 26
 RELAXWM (*Inputs.Options* attribute), 26
 RELAXWV (*Inputs.Options* attribute), 26
 RELAXWW (*Inputs.Options* attribute), 26
 RELAXX (*Inputs.Model* attribute), 23
 RELVELCST (*Inputs.Model* attribute), 23
 resizeFigure() (*Solvers.SolverPlotter* method), 111
 REV() (*Solvers.FourField.Wave* method), 52
 REV() (*Solvers.ThreeField.Drop* method), 83
 REV() (*Solvers.TwoFluid.Liquid* method), 106
 RHO() (*Solvers.Mixture.Mixture* method), 72
 RHO() (*Solvers.TwoFluid.Mixture* method), 99
 RHOF (*Inputs.FluidProperties* attribute), 31

RHOG (*Inputs.FluidProperties* attribute), 31
 RHOL() (*Inputs.FluidProperties* method), 32
 RHOV() (*Inputs.FluidProperties* method), 33
 RWALL() (*Inputs.Geometry* method), 28

S

SC() (*Solvers.TwoFluid.Liquid* method), 106
 SC() (*Solvers.TwoFluid.Vapor* method), 95
 save() (*Solvers.AbstractSolver* method), 115
 saveobj() (*CoolPropWrapper.CoolPropWrapper* method), 118
 Session (*class in Session*), 34
 session (*Inputs.InputSet* attribute), 29
 Session (*module*), 33
 Session() (*Session.Session* method), 35
 setFluid() (*CoolPropWrapper.CoolPropWrapper* method), 118
 setSpecifyPhase() (*CoolPropWrapper.CoolPropWrapper* method), 118
 setupLog() (*Session.Session* method), 35
 setZs() (*Solvers.SolverPlotter* method), 111
 SHAPEFACTOR() (*Solvers.FourField.Wave* method), 52
 SHAPEFACTORCOEF (*Inputs.Model* attribute), 23
 showWarnings (*Session.Log* attribute), 34
 showWarnings (*Session.Session* attribute), 35
 SIGMA (*Inputs.FluidProperties* attribute), 31
 SLIP (*Inputs.Model* attribute), 23
 solve() (*in module Solvers.FourField.FourFieldSolver*), 39
 solve() (*in module Solvers.Mixture.MixtureSolver*), 57
 solve() (*in module Solvers.ThreeField.ThreeFieldSolver*), 79
 solve() (*in module Solvers.TwoFluid.TwoFluidSolver*), 88
 SOLVERDEPENDENTPROPS (*Inputs.Model* attribute), 23
 solverName() (*Solvers.AbstractSolver* method), 115
 SolverPlotter (*class in Solvers*), 109
 SolverPlotter() (*Solvers.SolverPlotter* method), 110
 Solvers (*module*), 109
 Solvers.FourField (*module*), 39
 Solvers.FourField.FourFieldSolver (*module*), 36
 Solvers.Mixture (*module*), 57
 Solvers.Mixture.MixtureSolver (*module*), 53
 Solvers.ThreeField (*module*), 80
 Solvers.ThreeField.ThreeFieldSolver (*module*), 75
 Solvers.TwoFluid (*module*), 89
 Solvers.TwoFluid.TwoFluidSolver (*module*), 85
 SolverState (*class in Solvers*), 109
 SPACING() (*Solvers.FourField.Wave* method), 52
 SPHTM (*class in InputEnums*), 15
 SPHTM (*Inputs.Model* attribute), 23
 SPMTM (*class in InputEnums*), 16
 SPMTM (*Inputs.Model* attribute), 23

SSCONVFW (*Inputs.Options* attribute), 26
 SSCONVH (*Inputs.Options* attribute), 26
 SSCONVP (*Inputs.Options* attribute), 26
 SSCONVU (*Inputs.Options* attribute), 26
 SSCONVUD (*Inputs.Options* attribute), 26
 SSCONVUF (*Inputs.Options* attribute), 26
 SSCONVW (*Inputs.Options* attribute), 26
 SSCONVWF (*Inputs.Options* attribute), 27
 SSMAXITER (*Inputs.Options* attribute), 27
 SSTSTEP (*Inputs.Options* attribute), 27
 STATE (*Solvers.AbstractSolver* attribute), 115
 struct() (*Solvers.AbstractField* method), 114
 surfaceTension() (*CoolPropWrapper.CoolPropWrapper* method), 118

T

T() (*Inputs.FluidProperties* method), 33
 T() (*Solvers.Mixture.Liquid* method), 75
 T() (*Solvers.Mixture.Mixture* method), 72
 T() (*Solvers.Mixture.Vapor* method), 59
 T() (*Solvers.TwoFluid.Liquid* method), 107
 T() (*Solvers.TwoFluid.Vapor* method), 95
 TAUW() (*Solvers.Mixture.Mixture* method), 72
 TAUW() (*Solvers.Mixture.Vapor* method), 59
 TAUW() (*Solvers.TwoFluid.Liquid* method), 107
 TAUW() (*Solvers.TwoFluid.Mixture* method), 99
 TAUW() (*Solvers.TwoFluid.Vapor* method), 95
 TBASE() (*Solvers.FourField.Base* method), 45
 TDRY() (*Solvers.FourField.Base* method), 46
 temperature() (*CoolPropWrapper.CoolPropWrapper* method), 118
 THERMALNONEQ (*class in InputEnums*), 10
 THERMALNONEQ (*Inputs.Model* attribute), 23
 THERMALRELAX (*class in InputEnums*), 8
 THERMALRELAX (*Inputs.Model* attribute), 23
 THICK() (*Solvers.AbstractFilm* method), 113
 THICK() (*Solvers.FourField.Wave* method), 52
 THICKMIN() (*Solvers.FourField.Base* method), 46
 THINFILMFRIC (*class in InputEnums*), 7
 THINFILMFRIC (*Inputs.Model* attribute), 23
 THINFILMTHICK (*Inputs.Model* attribute), 23
 THINWAVETHICK (*Inputs.Model* attribute), 23
 ThreeFieldSolver (*class in Solvers.ThreeField.ThreeFieldSolver*), 75
 ThreeFieldSolver() (*Solvers.ThreeField.ThreeFieldSolver* method), 76
 TIDX (*Solvers.AbstractField* attribute), 114
 TIDX (*Solvers.FourField.Base* attribute), 42
 TIDX (*Solvers.FourField.Film* attribute), 40
 TIDX (*Solvers.FourField.Wave* attribute), 47
 TIDX (*Solvers.Mixture.Mixture* attribute), 60
 TIDX (*Solvers.ThreeField.Drop* attribute), 80
 TIDX (*Solvers.ThreeField.Film* attribute), 84
 TIDX (*Solvers.TwoFluid.Liquid* attribute), 100

TIDX (*Solvers.TwoFluid.Vapor* attribute), 89
 TIME (*Inputs.BoundaryConditions* attribute), 17
 TIME (*Solvers.AbstractField* attribute), 114
 TIME (*Solvers.FourField.Base* attribute), 42
 TIME (*Solvers.FourField.Film* attribute), 40
 TIME (*Solvers.FourField.Wave* attribute), 47
 TIME (*Solvers.Mixture.Mixture* attribute), 60
 TIME (*Solvers.Mixture.MixtureSolver.MixtureSolver* attribute), 54
 TIME (*Solvers.ThreeField.Drop* attribute), 80
 TIME (*Solvers.ThreeField.Film* attribute), 84
 TIME (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver* attribute), 76
 TIME (*Solvers.TwoFluid.Liquid* attribute), 101
 TIME (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* attribute), 86
 TIME (*Solvers.TwoFluid.Vapor* attribute), 89
 TIME() (*Solvers.Mixture.Liquid* method), 75
 TIME() (*Solvers.Mixture.Vapor* method), 59
 TIME() (*Solvers.TwoFluid.Mixture* method), 99
 TIMEINTERP (*Inputs.Options* attribute), 27
 timeInterpolate() (*Solvers.Mixture.MixtureSolver.MixtureSolver* method), 57
 TIN() (*Inputs.BoundaryConditions* method), 18
 Title (*Solvers.SolverPlotter* attribute), 110
 TPFM (*class in InputEnums*), 15
 TPFM (*Inputs.Model* attribute), 23
 TPHTM (*class in InputEnums*), 12
 TPHTM (*Inputs.Model* attribute), 24
 TPKM (*class in InputEnums*), 15
 TPKM (*Inputs.Model* attribute), 24
 transient() (*Inputs.FluidProperties* method), 33
 transient() (*Solvers.AbstractField* method), 114
 Ts (*Solvers.SolverPlotter* attribute), 110
 TSAT (*Inputs.FluidProperties* attribute), 32
 TSAT() (*Inputs.BoundaryConditions* method), 18
 TSTEP (*Inputs.Options* attribute), 27
 TWALL() (*Solvers.Mixture.Mixture* method), 72
 TwoFluidSolver (*class in Solvers.TwoFluid.TwoFluidSolver*), 85
 TwoFluidSolver() (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver* method), 86

U

U (*Solvers.FourField.Base* attribute), 42
 U (*Solvers.FourField.Film* attribute), 40
 U (*Solvers.FourField.Wave* attribute), 47
 U (*Solvers.ThreeField.Drop* attribute), 80
 U (*Solvers.ThreeField.Film* attribute), 84
 U (*Solvers.TwoFluid.Liquid* attribute), 101
 U (*Solvers.TwoFluid.Vapor* attribute), 89
 U() (*Solvers.Mixture.Liquid* method), 75
 U() (*Solvers.Mixture.Mixture* method), 72
 U() (*Solvers.Mixture.Vapor* method), 59

U() (*Solvers.TwoFluid.Mixture method*), 99
 UALGEBR() (*Solvers.AbstractFilm method*), 113
 UALGEBR() (*Solvers.ThreeField.Drop method*), 83
 UC (*Inputs.FluidProperties attribute*), 32
 UEQUIL() (*Solvers.AbstractFilm method*), 113
 UEQUIL() (*Solvers.ThreeField.Drop method*), 83
 UEQUILS() (*Solvers.AbstractFilm method*), 113
 UP() (*Solvers.Mixture.Mixture method*), 72
 USLIP() (*Solvers.ThreeField.Drop method*), 83
 USLIP() (*Solvers.TwoFluid.Liquid method*), 107
 USLIP() (*Solvers.TwoFluid.Vapor method*), 96
 UWALL() (*Solvers.AbstractFilm method*), 113

V

validateFunctionHandleInput() (*Inputs.Input static method*), 30
 validateInputEntry() (*Inputs.Input method*), 30
 Vapor (*class in Solvers.Mixture*), 57
 Vapor (*class in Solvers.TwoFluid*), 89
 vapor (*Solvers.Mixture.Mixture attribute*), 61
 vapor (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute*), 86
 Vapor() (*Solvers.Mixture.Vapor method*), 58
 Vapor() (*Solvers.TwoFluid.Vapor method*), 89
 VAPORFRIC (*class in InputEnums*), 13
 VAPORFRIC (*Inputs.Model attribute*), 24
 VAPORFRICCSST (*Inputs.Model attribute*), 24
 vaporInit (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute*), 87
 VF() (*Solvers.Mixture.Liquid method*), 75
 VF() (*Solvers.Mixture.Mixture method*), 73
 VF() (*Solvers.Mixture.Vapor method*), 59
 VF() (*Solvers.TwoFluid.Liquid method*), 107
 VF() (*Solvers.TwoFluid.Mixture method*), 99
 VF() (*Solvers.TwoFluid.Vapor method*), 96
 VISC() (*Solvers.TwoFluid.Liquid method*), 107
 VISC() (*Solvers.TwoFluid.Vapor method*), 96
 viscosity() (*CoolPropWrapper.CoolPropWrapper method*), 118
 VOID (*class in InputEnums*), 13
 VOID (*Inputs.Model attribute*), 24
 VOLUME() (*Solvers.ThreeField.Drop method*), 83
 VR() (*Solvers.ThreeField.Drop method*), 83
 VR() (*Solvers.TwoFluid.Liquid method*), 107

W

W (*Solvers.FourField.Base attribute*), 42
 W (*Solvers.FourField.Film attribute*), 41
 W (*Solvers.FourField.Wave attribute*), 47
 W (*Solvers.Mixture.Mixture attribute*), 60
 W (*Solvers.ThreeField.Drop attribute*), 80
 W (*Solvers.ThreeField.Film attribute*), 85
 W (*Solvers.TwoFluid.Liquid attribute*), 101
 W (*Solvers.TwoFluid.Vapor attribute*), 89

W() (*Solvers.Mixture.Liquid method*), 75
 W() (*Solvers.Mixture.Vapor method*), 59
 W() (*Solvers.TwoFluid.Mixture method*), 99
 WALEVAPRATIO() (*Solvers.Mixture.Mixture method*), 73
 WALEVAPRATIO() (*Solvers.TwoFluid.Liquid method*), 108
 WALEVAPRATIO() (*Solvers.TwoFluid.Mixture method*), 100
 WALEVAPRATIO() (*Solvers.TwoFluid.Vapor method*), 96
 WallIdx (*Solvers.SolverPlotter attribute*), 110
 warning() (*Session.Log method*), 34
 warnings (*Inputs.Input attribute*), 29
 Wave (*class in Solvers.FourField*), 46
 wave (*Solvers.FourField.Film attribute*), 41
 Wave() (*Solvers.FourField.Wave method*), 47
 WAVEBASEINT (*class in InputEnums*), 8
 WAVEBASEINT (*Inputs.Model attribute*), 24
 WAVEDRAGCOEF (*Inputs.Model attribute*), 24
 WAVEFREQUENCY (*class in InputEnums*), 15
 WAVEFREQUENCY (*Inputs.Model attribute*), 24
 WAVEMIXCOEF (*Inputs.Model attribute*), 24
 WBOILINGN (*Inputs.Model attribute*), 24
 WBOILINGXSAT (*Inputs.Model attribute*), 24
 WBOILINGXSUB (*Inputs.Model attribute*), 24
 WIDTH() (*Solvers.FourField.Wave method*), 52
 WL() (*Solvers.AbstractFilm method*), 113
 WL() (*Solvers.FourField.Wave method*), 52
 WMESH (*Inputs.BoundaryConditions attribute*), 17
 WMIN() (*Solvers.FourField.Base method*), 46
 WMINL() (*Solvers.FourField.Base method*), 46
 WPOWER (*Inputs.BoundaryConditions attribute*), 17
 writeInputFile() (*Inputs.BoundaryConditions static method*), 18
 writeInputFile() (*Inputs.Geometry static method*), 28
 writeInputFile() (*Inputs.Input static method*), 30
 writeInputFile() (*Inputs.Model static method*), 24
 writeInputFile() (*Inputs.Options static method*), 27
 WWALL() (*Solvers.Mixture.Mixture method*), 73

X

X() (*Solvers.AbstractFilm method*), 113
 X() (*Solvers.Mixture.Liquid method*), 75
 X() (*Solvers.Mixture.Mixture method*), 73
 X() (*Solvers.Mixture.Vapor method*), 59
 X() (*Solvers.ThreeField.Drop method*), 84
 X() (*Solvers.TwoFluid.Liquid method*), 108
 X() (*Solvers.TwoFluid.Vapor method*), 96
 XEQ() (*Solvers.Mixture.Mixture method*), 73
 XEQ() (*Solvers.TwoFluid.Mixture method*), 100
 XIN() (*Inputs.BoundaryConditions method*), 18
 xlim() (*Solvers.SolverPlotter method*), 111
 XTR_ANN() (*Solvers.TwoFluid.Liquid method*), 108
 XTR_ANN() (*Solvers.TwoFluid.Vapor method*), 96
 XTR_CBT() (*Solvers.TwoFluid.Vapor method*), 96

[XTR_ITM\(\)](#) (*Solvers.TwoFluid.Liquid method*), 108
[XTR_ITM\(\)](#) (*Solvers.TwoFluid.Vapor method*), 97
[XTR_SAT\(\)](#) (*Solvers.TwoFluid.Liquid method*), 108
[XTR_SAT\(\)](#) (*Solvers.TwoFluid.Vapor method*), 97
[XTR_SUB\(\)](#) (*Solvers.TwoFluid.Liquid method*), 108
[XTR_SUB\(\)](#) (*Solvers.TwoFluid.Vapor method*), 97

Y

[ylabelIs\(\)](#) (*Solvers.SolverPlotter method*), 111
[ylim\(\)](#) (*Solvers.SolverPlotter method*), 111
[YPLUS2THICK\(\)](#) (*Solvers.AbstractFilm method*), 113

Z

[Z](#) (*Solvers.AbstractField attribute*), 114
[Z](#) (*Solvers.FourField.Base attribute*), 42
[Z](#) (*Solvers.FourField.Film attribute*), 41
[Z](#) (*Solvers.FourField.Wave attribute*), 47
[Z](#) (*Solvers.Mixture.Mixture attribute*), 60
[Z](#) (*Solvers.Mixture.MixtureSolver.MixtureSolver attribute*), 54
[Z](#) (*Solvers.ThreeField.Drop attribute*), 80
[Z](#) (*Solvers.ThreeField.Film attribute*), 85
[Z](#) (*Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver attribute*), 77
[Z](#) (*Solvers.TwoFluid.Liquid attribute*), 101
[Z](#) (*Solvers.TwoFluid.TwoFluidSolver.TwoFluidSolver attribute*), 86
[Z](#) (*Solvers.TwoFluid.Vapor attribute*), 89
[Z\(\)](#) (*Solvers.Mixture.Liquid method*), 75
[Z\(\)](#) (*Solvers.Mixture.Vapor method*), 59
[Z\(\)](#) (*Solvers.TwoFluid.Mixture method*), 100
[Zs](#) (*Solvers.SolverPlotter attribute*), 110